

Файл: /home/sergey/Documents/configurate/FracSin.py (относительно: FracSin.py)

```
import numpy as np
import matplotlib.pyplot as plt

# Параметры
# Амплитуды: A_k = A0 * (0.3)^k
A0 = 0.32
decay = 0.32
f0 = 1.0
# Генерируем массивы амплитуд и частот
f = [f0 * (8 ** k) for k in range(levels)]
# Параметрическая кривая: x(t) = t, y(t) = Σ A_k * sin(2π f_k t + φ_k) (но это не даст обвивания)
# Для обвивания нужно на каждом уровне смещать по нормали, но при 7 уровнях это крайне тяжело
# Поэтому я предлагаю упрощённую модель: кривая задаётся как (X(t), Y(t)) = (t, F(t)), где F(t) = Σ A_k * sin(2π f_k t)
# Это даст сильную осцилляцию, но без самопересечений. Для самопересечений нужно параметрическое обвивание.
# Для демонстрации сложности построим график функции y = Σ A_k sin(2π f_k t) (уровни накладываются)
# Это не совсем то, что мы рисовали раньше, но зато вычислимо для 7 уровней.
t = np.linspace(0, 2*np.pi, 50000)
for k in range(levels):
    y[k] = A[k] * np.sin(f[k] * t)

plt.figure(figsize=(14, 6))
plt.plot(t, 'g', 'Сумма синусоид: 7 уровней, частота растёт в 8 раз, амплитуда умножается на {decay}')
```

Теперь увеличенный фрагмент, чтобы увидеть мелкие колебания

```
fragment = np.zeros_like(t_fragment)
for k in range(levels):
    y_fragment += A[k] * np.sin(f[k] * t_fragment)

plt.figure(figsize=(14, 6))
plt.plot(t_fragment, y_fragment, 'b-', lw=0.5)
plt.plot([0, 0.05], 'r', 'Фрагмент [0, 0.05] — видны высокочастотные осцилляции')
plt.show()
```

Файл: /home/sergey/Documents/configurate/base_manager.py (относительно: base_manager.py)

```
# base_manager.py
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QTableWidgetItem, QTableWidgetItemEvent, QMessageBox, QHeaderView
from PyQt5.QtCore import Qt
from database import DatabaseService

class BaseEntityManager(QWidget):
    """Базовый класс для объектов управления справочниками.
    Предоставляет таблицу и виртуальные методы для CRUD.
    """
    def __init__(self, parent=None, db: DatabaseService = None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.table_headers = [] # заголовки колонок
        self.key_column = 0 # первичный ключ
        self.data = [] # кэш данных
        layout = QVBoxLayout(self)
        layout.setContentsMargins(0, 0, 0, 0)
        # Кнопки CRUD (будут подключены в подклассе)
        self.crud_buttons = None # будет установлен в подклассе
        # Таблица
        self.table = QTableWidgetItem()
        self.table.setSelectionBehavior(QTableWidgetItem.SelectRows)
        self.table.setSelectionMode(QTableWidgetItem.SingleSelection)
        self.table.horizontalHeader().setStretchLastSection(True)
        self.table.itemDoubleClicked.connect(self.on_item_double_clicked)
        layout.addWidget(self.table)
        self.setup_table()

    def setup_table(self):
        if self.table.rowCount() > 0:
            self.table.setHeaderCount(len(self.table.headers))
            self.table.setHorizontalHeaderLabels(self.table_headers)
        def set_crud_buttons(self, crud_widget):
            """Создаём кнопки CRUD и подключаем сигналы."""
            self.crud_buttons = crud_widget
            self.crud_buttons.add_clicked.connect(self.add_item)
            self.crud_buttons.delete_clicked.connect(self.delete_item)
        def load_data(self):
            """Загрузить данные из БД и обновить таблицу. Переопределяется в подклассах."""
            pass
        def refresh_table(self):
            """Обновить отображение таблицы на основе self.data."""
            if not self.data:
                return
            self.table.setRowCount(len(self.data))
            for row_idx, row_data in enumerate(self.data):
                for col_idx, col_data in enumerate(self.table_headers):
                    item = QTableWidgetItem(str(row_data[col_idx]))
                    self.table.setItem(row_idx, col_idx, item)
        def get_selected_id(self):
            """Возвращает ID выбранной строки или None."""
            current_row = self.table.currentRow()
            if current_row >= 0:
                item = self.table.item(current_row, 0)
                if item:
                    return item.data(Qt.UserRole)
            return None
        def add_item(self):
            """Добавление нового элемента. Переопределяется."""
            pass
        def edit_item(self):
            """Редактирование выбранного элемента. Переопределяется."""
            pass
        def delete_item(self):
            """Удаление выбранного элемента. Переопределяется."""
            pass
        def on_item_double_clicked(self, item):
            """По двойному клику вызываем редактирование."""
            self.edit_item()
```

Файл: /home/sergey/Documents/configurate/composition_manager.py (относительно: composition_manager.py)

```
# composition_manager.py
from PyQt5.QtWidgets import QMessageBox
```

```
from database import DatabaseService
```

```
class CompositionManager:
    def __init__(self, db = DatabaseService(), parent_widget=None):
        self.parent = parent_widget

    def get_entity_cname(self, entity_id: int) -> str:
        """Возвращает имя системы или имя (cname) сущности, приводит к нижнему регистру. """
        res = self.db.execute_query(" SELECT cname FROM meta.entitytypes WHERE id=%s", (entity_id,))
        if not res:
            return ValueError(f"Сущность с id {entity_id} не найдена")
        return res[0][0].lower()

    def ensure_ext_table_and_field(self, child_entity_id, link_field_name):
        child_cname = self.get_entity_cname(child_entity_id)
        ext_table = f"ext_ext_{child_cname}"
        # Проверим существование таблицы
        if not check_table_or_not_check_table[0][0]:
            sql_create_table = f"CREATE TABLE IF NOT EXISTS {ext_table} (
                versionid INTEGER NOT NULL REFERENCES data.entity_versions(id) ON DELETE CASCADE
            )"
            self.db.execute_query(sql_create, fetch=False)
        # Проверим наличие поля в meta.fields
        check_field_id = f"SELECT id FROM meta.fields WHERE entitytypeid=%s AND cfieldname=%s",
        (child_entity_id, link_field_name)
        if not check_field:
            sql_insert = f"INSERT INTO meta.fields (entitytypeid, cfieldname, cfieldtype, calias, lisindexed, status_id)
                VALUES (%s, %s, %s, %s, %s, %s)"
            (child_entity_id, link_field_name, "FK: {link_field_name}")
            if ins_id == ins[0][0]:
                self.db.execute_procedure("meta.p_field_sync", (field_id,))
        return True

    def add_composition(self, parent_entityid, child_entityid, role_id, link_field, sortorder=10):
        if self.ensure_ext_table_and_field(child_entityid, link_field):
            if self.parent:
                QMessageBox.warning(self.parent, "Ошибка", "Не удалось подготовить ext-таблицу/поле")
            return self.db.execute_query(
                "WHERE parent_entityid=%s AND child_entityid=%s AND role_id=%s",
                (parent_entityid, child_entityid, role_id)
            )
        if check:
            QMessageBox.warning(self.parent, "Ошибка", "Такая связь уже существует")
            return
        sql = f"INSERT INTO meta.entity_composition (parent_entityid, child_entityid, role_id, link_fieldname, sortorder)
            VALUES (%s, %s, %s, %s, %s)"
        self.db.execute_query(sql, (parent_entityid, child_entityid, role_id, link_field, sortorder), fetch=False)
        return True

    def update_composition(self, comp_id, parent_entityid, child_entityid, role_id, link_field, sortorder=10):
        if self.ensure_ext_table_and_field(child_entityid, link_field):
            if self.parent:
                QMessageBox.warning(self.parent, "Ошибка", "Не удалось подготовить ext-таблицу/поле")
            return self.db.execute_query(
                "WHERE parent_entityid=%s AND child_entityid=%s AND role_id=%s AND id!=%s",
                (parent_entityid, child_entityid, role_id, comp_id)
            )
        if check:
            QMessageBox.warning(self.parent, "Ошибка", "Такая связь уже существует")
            return
        sql = f"UPDATE meta.entity_composition SET
            parent_entityid=%s, child_entityid=%s, role_id=%s,
            link_fieldname=%s, sortorder=%s"
        self.db.execute_query(sql, (parent_entityid, child_entityid, role_id, link_field, sortorder, comp_id), fetch=False)
        return True

    def delete_composition(self, comp_id):
        self.db.execute_query("DELETE FROM meta.entity_composition WHERE id=%s", (comp_id,), fetch=False)
```

Файл: /home/sergey/Documents/configurate/create_import_structure.py (относительно: crud_buttons.py)

```
# create_import_structure.py
import os
ROOT = "/home/sergey/Documents/configurate"
STRUCTURE = {
    "mozart_import/graph": [
        "init.py",
        "graphics_items.py",
        "relation_scale.py",
        "link_line.py",
    ],
    "mozart_import/wizards": [
        "init.py",
        "source_page.py",
        "selection_page.py",
        "mapping_detail_page.py",
        "execute_page.py",
    ],
    "mozart_import/core": [
        "init.py",
        "models.py",
        "mapping_analyzer.py",
        "mapping_manager.py",
    ],
    "mozart_import/utills": [
        "field_creator.py",
    ],
    "mozart_import/connectors": [], # уже есть, оставляем
}

def create_structure():
    for full_path, files in STRUCTURE.items():
        os.makedirs(os.path.join(ROOT, full_path), exist_ok=True)
        for filename in files:
            file_path = os.path.join(full_path, filename)
            with open(file_path, "w", encoding='utf-8') as f:
                f.write(f"Created: {file_path}\n")
            else:
                print(f"Exists: {file_path}")
if __name__ == "__main__":
    create_structure()
    print("Done.")
```

Файл: /home/sergey/Documents/configurate/crud_buttons.py (относительно: crud_buttons.py)

```
# crud_buttons.py
from PyQt5.QtWidgets import QWidget, QHBoxLayout, QPushButton
from PyQt5.QtCore import pyqtSignal
from lang.local_translator import LocalTranslator
class CrudButtons(QWidget):
    add_clicked = pyqtSignal()
```

```

edit_clicked = pyqtSignal()
delete_clicked = pyqtSignal()
def __init__(self, parent=None, with_add=True, with_edit=True, with_delete=True):
    self.translator = QtCore.QTranslator()
    self.with_add = with_add
    self.with_edit = with_edit
    self.with_delete = with_delete
    layout = QtWidgets.QVBoxLayout()
    layout.setContentsMargins(0, 0, 0, 0)
    if with_add:
        self.btn_add = QPushButton()
        self.btn_add.clicked.connect(self.add_clicked.emit)
        layout.addWidget(self.btn_add)
    if with_edit:
        self.btn_edit = QPushButton()
        self.btn_edit.clicked.connect(self.edit_clicked.emit)
        layout.addWidget(self.btn_edit)
    if with_delete:
        self.btn_del = QPushButton()
        self.btn_del.clicked.connect(self.delete_clicked.emit)
        layout.addWidget(self.btn_del)
    layout.addStretch(1)
    self.retranslate_ui()
def retranslate_ui(self):
    if self.with_add and hasattr(self, 'btn_add'):
        self.btn_add.setText(self.translator.tr('btn_add'))
    if self.with_edit and hasattr(self, 'btn_edit'):
        self.btn_edit.setText(self.translator.tr('btn_edit'))
    if self.with_delete and hasattr(self, 'btn_del'):
        self.btn_del.setText(self.translator.tr('btn_delete'))

```

Файл: /home/sergey/Documents/configurate/database.py (относительно: database.py)

```

#database.py
import psycopg2
from ide_core import goProject

class DatabaseService:
    """Единичный класс для взаимодействия с БД, обёртка над goProject."""
    def __init__(self):
        self.conn = goProject.db_conn
    def execute_query(self, sql, params=None, fetch=True):
        """Выполнить SQL-запрос (SELECT или модифицирующий)."""
        if fetch:
            return goProject.f_Execute(sql, params)
        else:
            try:
                with self.conn.cursor() as cur:
                    cur.execute(sql, params)
            except Exception as e:
                raise DBError(e)
    def execute_procedure(self, proc_name, params=None):
        """Выполнить процедуру"""
        placeholders = ','.join(['%s'] * len(params))
        sql = f"CALL {proc_name}({placeholders})"
        return self.execute_query(sql, params, fetch=False)
    def get_connection(self):
        return self.conn
    def refresh_connection(self):
        self.conn = goProject.db_conn

```

Файл: /home/sergey/Documents/configurate/ide_core.py (относительно: ide_core.py)

```

# ide_core.py
import psycopg2
import os

class GoProject:
    def __init__(self):
        # Залочка 1: БАЗА ДАННЫХ (Layer 1) ---
        self.db_host = "localhost"
        self.db_port = "5432"
        self.db_name = "postgres_erp"
        self.db_pass = "postgres"
        self.db_conn = None # Активное соединение psycopg2
        self.current_user_id = None # ID текущего пользователя (конфигуратора)
        # Залочка 2: СРЕДНИЙ СЛОЙ (Layer 2) ---
        self.z1_path = os.getcwd() + "/layer2_server"
        self.z1_process = None
        # Залочка 3: КЛИЕНТ (Layer 3) ---
        self.z3_path = os.getcwd() + "/layer3_client"
        self.z3_process = None
    def p_Connect(self):
        """Установка физического соединения с PostgreSQL.
        Проверка, что пользователь имеет роль конфигуратора.
        """
        try:
            # Закрываем старое соединение, если оно было
            if self.db_conn:
                self.db_conn.close()
            self.db_conn = psycopg2.connect(
                host=self.db_host,
                port=self.db_port,
                dbname=self.db_name,
                user=self.db_user,
                password=self.db_pass,
                connect_timeout=5
            )
            self.db_conn.autocommit = True
            # Проверяем, является ли текущий пользователь конфигуратором
            with self.db_conn.cursor() as cur:
                cur.execute("SELECT id FROM auth.users WHERE login = %s", (self.db_user,))
                row = cur.fetchone()
                if not row:
                    self.db_conn.close()
                    return False, f"Пользователь {self.db_user} не найден в системе."
            user_id = row[0]
            # Проверяем наличие роли с типом 'CONFIG'
            cur.execute(
                """SELECT u.id
                FROM auth.users u
                JOIN auth.roles r ON u.id = r.user_id
                JOIN auth.role_types t ON r.role_type = t.id
                WHERE u.id = %s AND t.role_type = 'CONFIG'"""
            )
            if not cur.fetchone():
                self.db_conn.close()

```

```

        return False, "У вас нет прав configurатора. Доступ запрещён."
    self.current_user_id = user_id
except Exception:
    return False, "Слоном установлено успешно. Добро пожаловать, configurатор!"
return False, "Ошибка подключения: {str(e)}"
def f.Execute(self, sql, params=None):
    """Выполнение SQL-запроса (SELECT)
    Возвращает коллекцию строк (toResult). [0..3]
    if not self.db_conn:
        return []
    try:
        with self.db_conn.cursor() as cur:
            cur.execute(sql, params)
            if cur.description: # Если запрос возвращает данные (SELECT)
                return cur.fetchall()
    except Exception as e:
        return ["SQL Error: {e}"]
def p.SaveProject(self, file_path):
    """Сохранение манифеста проекта в .mzt файл (JSON) """
    data = {
        "host": self.db_host, "port": self.db_port,
        "name": self.db_name, "user": self.db_user, "pass": self.db_pass,
        "l2": {"path": self.l2_path, "port": self.l2_port},
        "l3": {"path": self.l3_path, "starter": self.l3_starter}
    }
    with open(file_path, 'w', encoding='utf-8') as f:
        json.dump(data, f, indent=4, ensure_ascii=False)
def p.LoadProject(self, file_path):
    """Загрузка параметров проекта из .mzt файла """
    if not os.path.exists(file_path):
        return False, "Файл проекта не найден."
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            data = json.load(f)
            self.db_host = data['host'].get('host', 'localhost')
            self.db_name = data['host'].get('name', '432')
            self.db_pass = data['host'].get('pass', 'postgres')
            self.l2_path = data['l2'].get('path', '8000')
            self.l3_path = data['l3'].get('path', '')
            self.l3_starter = data['l3'].get('starter', 'starter.py')
        return True, "Проект загружен."
    except Exception as e:
        return False, "Ошибка чтения файла: {str(e)}"
# Создаем единственный экземпляр объекта управления проектом
goProject = GoProject()

```

Файл: /home/sergey/Documents/configurate/ide_db_work.py (относительно: ide_db_work.py)

```

# ide_db_work.py
# ide_db_work.py
from PyQt5.QtWidgets import QDialog, QVBoxLayout, QWidget, QPushButton
from PyQt5.QtCore import Qt
from database import DatabaseService
from tabs import BalanceUnitsTab, UsersRolesTab, EntitiesTab, DocumentsTab, DataViewTab, RelationsTab
from lang.local_translator import LocalTranslator
class DBWorkDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('window_title_db_work'))
        self.resize(1300, 800)
        self.db = DatabaseService()
        layout = QVBoxLayout(self)
        self.tabs = QWidget()
        self.tabs.addTab(BalanceUnitsTab(self, db=self.db, self.translator.tr('tab_balance_units')))
        self.tabs.addTab(UsersRolesTab(self, db=self.db, self.translator.tr('tab_users_roles')))
        self.tabs.addTab(EntitiesTab(self, db=self.db, self.translator.tr('tab_entities')))
        self.tabs.addTab(DocumentsTab(self, db=self.db, self.translator.tr('tab_documents')))
        self.tabs.addTab(DataViewTab(self, db=self.db, self.translator.tr('tab_data_view')))
        self.tabs.addTab(RelationsTab(self, db=self.db, self.translator.tr('tab_relations')))
        layout.addWidget(self.tabs)
        self.btn_close = QPushButton(self.translator.tr('btn_close'))
        layout.addWidget(self.btn_close, alignment=Qt.AlignRight)
        # Применяем текущий язык ко всем элементам
        self.retranslate_ui()
    def retranslate_ui(self):
        self.setWindowTitle(self.translator.tr('window_title_db_work'))
        self.tabs.setTabText(1, self.translator.tr('tab_balance_units'))
        self.tabs.setTabText(2, self.translator.tr('tab_users_roles'))
        self.tabs.setTabText(3, self.translator.tr('tab_entities'))
        self.tabs.setTabText(4, self.translator.tr('tab_documents'))
        self.tabs.setTabText(5, self.translator.tr('tab_data_view'))
        self.tabs.setTabText(6, self.translator.tr('tab_relations'))
        self.btn_close.setText(self.translator.tr('btn_close'))
        # Обновляем содержимое вкладок, если они имеют метод retranslate_ui
        for i in range(self.tabs.count()):
            widget = self.tabs.widget(i)
            if hasattr(widget, 'retranslate_ui'):
                widget.retranslate_ui()

```

Файл: /home/sergey/Documents/configurate/ide_field_edit.py (относительно: ide_field_edit.py)

```

#ide_field_edit.py
import sys
from PyQt5.QtWidgets import (QDialog, QVBoxLayout, QFormLayout, QLineEdit,
                             QComboBox, QCheckBox, QPushButton, QHBoxLayout, QLabel,
                             QTextEdit,
                             Qt)
from PyQt5.QtCore import Qt
from widgets import ReferenceEditor
class FieldEditDialog(QDialog):
    def __init__(self, meta_data=None, status_id=None, read_only_mode=False, db=None):
        super().__init__()
        self.setWindowTitle("Свойства поля (meta.fields)")
        self.setFixedSize(500, 550)
        # Данные: [Имя, ТипКод, Алиас, Описание, Len, Dec, Idx, RefID]
        if not meta_data:
            self.data = ["new_field", "C", "Новое поле", "", 255, 0, False, None]
        else:
            self.data = field_data
        self.status_id = status_id
        self.read_only_mode = read_only_mode
        self.db = db or DatabaseService()
        self.init_ui()

```



```

def init_ui(self):
    layout = QVBoxLayout(self)
    form = QFormLayout()
    self.edit_name = QLineEdit(str(self.data[0]))
    form.addRow("Имя поля (SQL):", self.edit_name)
    self.validate_field_name()
    self.combo_type = QComboBox()
    types = goProject.r.Execute("SELECT cname, ccode FROM meta.field_types ORDER BY cname")
    for name, code in types:
        self.combo_type.addItem(name, code)
    idx = self.combo_type.findData(str(self.data[1]))
    self.combo_type.setCurrentIndex(idx)
    form.addRow("Тип данных:", self.combo_type)
    self.edit_alias = QLineEdit(str(self.data[2]))
    form.addRow("Бизнес-алиас:", self.edit_alias)
    self.edit_comment = QTextEdit()
    self.edit_comment.setText(str(self.data[3]) if self.data[3] else "")
    form.addRow("Описание:", self.edit_comment)
    # Ссылка на другую сущность (ReferenceEditor)
    self.ref_editor = ReferenceEditor(
        table_name="meta.entitytypes",
        display_field="cname",
        db=self.db
    )
    if len(self.data) > 7 and self.data[7] and str(self.data[7]).isdigit():
        ref_id = int(self.data[7])
        res = self.db.execute_query("SELECT cname FROM meta.entitytypes WHERE id=%s", (ref_id,))
        self.ref_editor.set_value(ref_id, res[0][0])
    form.addRow("Ссылается на (Ref):", self.ref_editor)
    self.edit_len = QLineEdit(str(self.data[4]))
    form.addRow("Длина (Len):", self.edit_len)
    self.edit_dec = QLineEdit(str(self.data[5]))
    form.addRow("Точность (Dec):", self.edit_dec)
    self.chk_idx = QCheckBox("Создать индекс в БД")
    self.chk_idx.setChecked(str(val_idx).lower() == 'true' or val_idx is True)
    form.addRow(self.chk_idx)
    # Статус
    self.combo_status = QComboBox()
    statuses = self.db.execute_query("SELECT id, name FROM meta.object_statuses ORDER BY id")
    for sid, sname in statuses:
        self.combo_status.addItem(sname, sid)
    if self.status_id:
        idx_status = self.combo_status.findData(self.status_id)
        if idx_status >= 0:
            self.combo_status.setCurrentIndex(idx_status)
    form.addRow("Статус:", self.combo_status)
    layout.addLayout(form)
    btns = QHBoxLayout()
    btn_ok = QPushButton("Принять")
    btn_ok.clicked.connect(self.accept)
    btn_cancel = QPushButton("Отмена")
    btn_cancel.clicked.connect(self.reject)
    btns.addWidget(btn_ok)
    btns.addWidget(btn_cancel)
    layout.addLayout(btns)
    self.p_CheckType()
    # Блокировка для PROD
    if self.read_only_mode or self.status_id == 2: # PROD
        self.edit_name.setEnabled(False)
        self.combo_type.setEnabled(False)
        self.edit_len.setEnabled(False)
        self.edit_dec.setEnabled(False)
        self.edit_comment.setEnabled(False)
        self.chk_idx.setEnabled(False)
        self.combo_status.setEnabled(True)
    def validate_field_name(self, text):
        if not text:
            self.edit_name.setStyleSheet("")
        if re.match(r'^[a-zA-Z][a-zA-Z0-9_]*$', text):
            self.edit_name.setStyleSheet("")
        else:
            self.edit_name.setStyleSheet("border: 1px solid red;")
    def p_CheckType(self):
        self.combo_type.setCurrentData(1)
        self.edit_dec.setEnabled((self.combo_type.currentData() in ('C', 'M')))
        self.ref_editor.setEnabled(not in ('G', 'V', 'F', 'M'))
        self.chk_idx.setEnabled(not in ('G', 'V', 'F', 'M'))
    def get_data(self):
        return [
            self.edit_name.text(),
            self.combo_type.currentData(),
            self.edit_alias.text(),
            self.edit_comment.toPlainText(),
            int(self.edit_len.text()) if self.edit_len.text().isdigit() else 0,
            self.chk_idx.isChecked() if self.edit_dec.text().isdigit() else 0,
            self.ref_editor.get_value()
        ]

```

Файл: /home/sergey/Documents/configurate/ide_main.py (относительно: ide_main.py)

```

# ide_main.py
import sys
import os
from PyQt5.QtWidgets import (QApplication, QMainWindow, QAction, QFileDialog,
                             QMessageBox, QLabel, QGroupBox, QSplitter, QDialog)
from PyQt5.QtCore import Qt
from ide_core import goProject
from ide_db_work import DBWorkDialog
from database_translator import LocalTranslator

class MozartIDE(QMainWindow):
    def __init__(self):
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('window_title_main'))
        self.resize(1100, 700)
        # словарь для хранения действий меню
        self.actions = {}
        # Центральный виджет
        central = QWidget()
        self.setCentralWidget(central)
        main_layout.setContentsMargins(5, 5, 5, 5)
        # Сплиттер
        splitter = QSplitter(Qt.Horizontal)
        main_layout.addWidget(splitter)
        # Левая панель (настройка, подключение)
        left = self._create_left_panel()

```

```

splitter.addWidget(left)
# Правая панель (кнопки управления)
splitter.addWidget(right_panel())
splitter.setSizes([450, 650])
# Статус бар
self.statusbar = QStatusBar()
self.statusbar.showMessage(self.translator.tr('status_ready'))
# Меню
self.init_menu()
# Переменные состояния
self.current_project_file = None
self.buttons_enabled = False
self.db_work_dialog = None # ссылка на диалог, если открыт
self.retranslate_ui()
def init_menu(self):
    menubar = self.menuBar()
    # Меню "Файл"
    file_menu = menubar.addMenu(self.translator.tr('menu_file'))
    self.actions['file_menu'] = file_menu
    new_act = QAction(self.translator.tr('action_new_project'), self)
    file_menu.triggered.connect(self.new_project)
    self.actions['new_project'] = new_act
    open_act = QAction(self.translator.tr('action_open_project'), self)
    file_menu.addAction(open_act)
    self.actions['open_project'] = open_act
    save_act = QAction(self.translator.tr('action_save_project'), self)
    file_menu.addAction(save_act)
    self.actions['save_project'] = save_act
    file_menu.addSeparator()
    exit_act = QAction(self.translator.tr('action_exit'), self)
    file_menu.addAction(exit_act)
    self.actions['exit'] = exit_act
    # Меню "Язык"
    lang_menu = menubar.addMenu(self.translator.tr('menu_language'))
    self.actions['lang_menu'] = lang_menu
    languages = ('ru', 'Русский'),
    ('en', 'English'),
    ('de', 'Deutsch'),
    ('tt', 'Татарча')
    for code, name in languages:
        action = QAction(name, self)
        action.triggered.connect(lambda checked, lang=code: self.change_language(lang))
        lang_menu.addAction(action)
    # Меню "Помощь"
    help_menu = menubar.addMenu(self.translator.tr('menu_help'))
    self.actions['help_menu'] = help_menu
    about_act = QAction(self.translator.tr('action_about'), self)
    help_menu.addAction(about_act)
    self.actions['about'] = about_act
def change_language(self, lang_code):
    self.translator.set_language(lang_code)
    self.retranslate_ui()
    if self.db_work_dialog and self.db_work_dialog.isVisible():
        self.db_work_dialog.retranslate_ui()
def retranslate_ui(self):
    self.setWindowTitle(self.translator.tr('window_title_main'))
    # Левая панель
    self.group_project_file.setTitle(self.translator.tr('group_project_file'))
    self.btn_open.setText(self.translator.tr('btn_open_project'))
    self.btn_new.setText(self.translator.tr('btn_new_project'))
    self.btn_save.setText(self.translator.tr('btn_save_project'))
    self.btn_connect.setText(self.translator.tr('btn_connect'))
    if self.project_db_conn:
        self.lbl_conn_status.setText(self.translator.tr('status_connected'))
    else:
        self.lbl_conn_status.setText(self.translator.tr('status_disconnected'))
    # Правая панель
    self.group_db_work.setTitle(self.translator.tr('group_db_work'))
    self.client_group.setTitle(self.translator.tr('group_server'))
    self.btn_db_work.setText(self.translator.tr('btn_db_work'))
    self.btn_srv_start.setText(self.translator.tr('btn_start_server'))
    self.btn_client_start.setText(self.translator.tr('btn_start_client'))
    # Меню (ОMenu использует setTitle, QAction - setText)
    self.actions['new_project'].setText(self.translator.tr('action_new_project'))
    self.actions['open_project'].setText(self.translator.tr('action_open_project'))
    self.actions['exit'].setText(self.translator.tr('action_exit'))
    self.actions['save_project'].setText(self.translator.tr('action_save_project'))
    self.actions['lang_menu'].setTitle(self.translator.tr('menu_language'))
    self.actions['about'].setText(self.translator.tr('action_about'))
    # Статус бар
    self.statusbar.showMessage(self.translator.tr('status_ready'))
def create_left_panel(self):
    panel = QWidget()
    layout = QVBoxLayout(panel)
    layout.setAlignment(Qt.AlignTop)
    # Группа "Файл проекта"
    file_group = QGroupBox(self.translator.tr('group_project_file'))
    self.btn_open = QPushButton()
    self.btn_new = QPushButton()
    self.btn_save = QPushButton()
    self.btn_connect = QPushButton()
    file_group.addWidget(self.btn_new)
    file_group.addWidget(self.btn_save)
    file_group.addWidget(self.btn_connect)
    layout.addWidget(file_group)
    # Разделитель
    line = QFrame.HLine()
    line.setFrameShape(QFrame.HLine)
    line.setFrameStyle(QFrame.Sunken)
    layout.addWidget(line)
    # Группа "Подключение к базе данных"
    db_group = QGroupBox(self.translator.tr('group_db_connection'))
    form = QFormLayout()
    form.addRow(self.translator.tr('field_host'), self.edit_host)
    form.addRow(self.translator.tr('field_port'), self.edit_port)
    form.addRow(self.translator.tr('field_user'), self.edit_user)
    form.addRow(self.translator.tr('field_pass'), self.edit_pass)
    self.edit_pass.setEchoMode(QLineEdit.Password)

```

```

form.addRow(self.translator.tr('field_password'), self.edit_pass)
self.btn_connect = QPushButton(self.translator.tr('btn_connect'))
self.btn_connect.setEnabled(False)
form.addRow(self.btn_connect)
layout.addWidget(self.conn_group)
# Информация о статусе подключения
self.lbl_conn_status = QLabel(self.translator.tr('status_disconnected'))
layout.addWidget(self.lbl_conn_status)
layout.addStretch()
return panel

def create_right_panel(self):
    panel = QVBoxLayout()
    layout.setAlignment(Qt.AlignTop)
    # Группы "Работа с БД"
    db_layout = QVBoxLayout()
    self.btn_db_work = QPushButton(self.translator.tr('btn_db_work'))
    self.btn_db_work.clicked.connect(self.open_db_work)
    db_layout.addWidget(self.btn_db_work)
    self.db_group.setLayout(db_layout)
    layout.addWidget(self.db_group)
    # Группы "Сервер приложений"
    srv_layout = QVBoxLayout()
    self.btn_srv_start = QPushButton(self.translator.tr('btn_start_server'))
    self.btn_srv_start.setEnabled(False)
    self.btn_srv_stop = QPushButton(self.translator.tr('btn_stop_server'))
    self.btn_srv_stop.setEnabled(False)
    srv_layout.addWidget(self.btn_srv_start)
    self.srv_group.setLayout(srv_layout)
    layout.addWidget(self.srv_group)
    # Группы "Клиент"
    client_layout = QVBoxLayout()
    self.btn_client_start = QPushButton(self.translator.tr('btn_start_client'))
    self.btn_client_start.setEnabled(False)
    self.client_group.setLayout(client_layout)
    layout.addWidget(self.client_group)
    layout.addStretch()
    return panel

# Методы работы с проектом ---
def new_project(self):
    self.current_project_file = None
    self.clear_connection_fields()
    self.btn_connect.setEnabled(True)
    self.statusbar.showMessage(self.translator.tr('status_new_project'))

def open_project(self):
    path = self.openFileDialog.getOpenFileName(
        self.translator.tr('open_dialog_open_project'), "",
        Mozart Project (*.mzt), All files (*)
    )
    if not path:
        return
    ok, msg = goProject.p_LoadProject(path)
    self.current_project_file = path
    self.btn_connect.setEnabled(False)
    self.statusbar.showMessage(self.translator.tr('status_project_loaded') + f" {os.path.basename(path)}")
    QMessageBox.critical(self, self.translator.tr('error_loading'), msg)

def save_project(self):
    if not self.current_project_file:
        return
    path = self.saveFileDialog.getSaveFileName(
        self.translator.tr('save_dialog_save_project'), "",
        Mozart Project (*.mzt)
    )
    if path:
        if not path.lower().endswith(".mzt"):
            path = path + ".mzt"
        self.current_project_file = path
        goProject.p_SaveProject(self.current_project_file)
        self.statusbar.showMessage(self.translator.tr('status_project_saved') + f" {os.path.basename(self.current_project_file)}")

# Работа с подключением
def clear_connection_fields(self):
    self.edit_host.setText("")
    self.edit_port.setText("5432")
    self.edit_db.setText("")
    self.edit_user.setText("")
    self.edit_pass.setText("")

def update_connection_fields(self):
    self.edit_host.setText(goProject.db_host)
    self.edit_port.setText(str(goProject.db_port))
    self.edit_db.setText(goProject.db_name)
    self.edit_pass.setText(goProject.db_pass)

def connect_to_db(self):
    goProject.db_host = self.edit_host.text().strip()
    goProject.db_port = self.edit_port.text().strip()
    goProject.db_name = self.edit_db.text().strip()
    goProject.db_pass = self.edit_pass.text().strip()
    ok, msg = goProject.p_Connect()
    if ok:
        # Синхронизируем локальную базу переводов с PostgreSQL
        db_service = DatabaseService() # используем тот же класс, что и в других частях
        self.translator.sync_from_postgres(db_service)
        # переводим интерфейс на текущий язык (например, русский)
        self.retranslate_ui()
        self.lbl_conn_status.setText(self.translator.tr('status_connected'))
        self.statusbar.showMessage(self.translator.tr('status_connected'))
        self.btn_db_work.setEnabled(True)
        self.btn_client_start.setEnabled(True)
    else:
        self.lbl_conn_status.setText(self.translator.tr('status_disconnected'))
        QMessageBox.warning(self, self.translator.tr('error'), msg)
        self.btn_db_work.setEnabled(False)
        self.btn_client_start.setEnabled(False)

def open_db_work(self):
    if self.db_work_dialog is None or not self.db_work_dialog.isVisible():
        self.db_work_dialog = DBWorkDialog(self)
        self.db_work_dialog.show()
        self.db_work_dialog.activateWindow()

def about(self):
    QMessageBox.about(self, self.translator.tr('action_about'),
        MOZART ERP Configurator (Версия 0.1) (Разработано на PyQt5)")

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window.show()
    sys.exit(app.exec_())

```



```

'field_status': 'Статус',
'info_add_as_child_not_supported': 'Добавление как ребенка не поддерживается (создаётся в родителе)',
'confirm_rename_message': 'Изменение системного имени с \'{0}\'' на \'{1}\'' переименует таблицу
расширения. Продолжить?',
'info_table_renamed': 'Таблица ext.{0} успешно переименована в ext.{1}',
'warning_table_not_found': 'Таблица ext.{0} не найдена. Переименование не требуется',
'error_rename_prod': 'Cannot change system name for entity in PROD status',
'error_delete_prod': 'Entities in PROD status cannot be deleted. {0}',
'error_save_field': 'Ошибка сохранения поля.',
},
{
'window_title_main': 'MOZART ERP - Configurator',
'language': '&Language',
'help': '&Help',
'new_project': 'New Project...',
'save_project': 'Save Settings',
'about': 'About',
'project_file': 'Project File',
'db_connection': 'Database Connection',
'db_work': 'Database Work',
'verver': 'Application Server',
'open_project': 'Open Project...',
'new_project': 'New Project',
'save_project': 'Save Settings',
'db_work': 'Database Work',
'start_server': 'Start Server',
'stop_server': 'Stop Client',
'close': 'Close',
'add': 'Add',
'cancel': 'Cancel',
'delete': 'Delete',
'cancel': 'Cancel',
'connect_db': 'Database connection successful',
'disconnect': 'Not connected',
'host_select_record': 'Select a record',
'port': 'Database',
'user': 'User',
'password': 'Password',
'alias': 'Alias',
'id': 'Active',
'information': 'Information',
'warning': 'Warning',
'error_handling_error': 'Error handling error',
'open_project': 'Open Project',
'save_project': 'Save Project',
'new_project': 'New project created. Fill in database parameters and connect.',
'project_created': 'Project created',
'edit_balance_unit': 'Edit Balance Unit',
'is_balance_unit': 'Is balance unit',
'parent': 'Parent',
'roles': 'Roles',
'context': 'Context',
'roles & context': 'Roles & Context',
'role_edit': 'Edit Role',
'context_assign': 'Assign Context',
'password_hash': 'Password hash',
'person_instance': 'Person instance',
'interface_role': 'Interface role',
'interface_path': 'Interface path',
'comment': 'Comment',
'role': 'Role',
'balance_unit': 'Balance unit',
'role': 'Role',
'context': 'Context',
'context_select': 'Select context entry',
'context_required': 'Context required. Users, roles and balance units are required',
'confirm_delete_role': 'Confirm deletion?',
'confirm_delete_context': 'Delete context?',
'interface': 'Interface',
'parent': 'Parent',
'as_parent': 'As child',
'default': 'Default',
'master': 'Master',
'role_field': 'Link field',
'add_visibility': 'Add BU visibility',
'individual': 'Individual',
'all_visible': 'All visible',
'is_already_exists': 'Record already exists',
'confirm_delete_field': 'Confirm delete field',
'confirm_delete_fields': 'Delete selected records (%d items)?',
'keys_for_entities_and_tabs': 'Keys for entities and tabs',
'entity_type': 'Entity Type',
'entity_name': 'System name',
'description': 'Description',
'legacy_class': 'Legacy class (legacy)',
'hierarchical': 'Hierarchical',
'versioned': 'Versioned',
'security_strategy': 'Security strategy',
'fields': 'Fields',
'visibility': 'BU Visibility',
'composition': 'Subordinate tables',
'routes': 'Routes',
'decimal_type': 'Decimals',
'leaf_type': 'Leaf type',
'routes_in_development': 'Section under development: document routes',
'warning_select_record': 'Warning select record',
'add_composition': 'Add subordinate table',
'info_add_as_child_not_supported': 'Adding as child is not supported (create in parent)',
'confirm_rename_message': 'Changing system name from \'{0}\'' to \'{1}\'' will rename the extension table.
Continue?',
'info_table_renamed': 'Table ext.{0} successfully renamed to ext.{1}',
'warning_table_not_found': 'Table ext.{0} not found, rename not required',
'error_rename_prod': 'Cannot change system name for entity in PROD status',
'error_delete_prod': 'Entities in PROD status cannot be deleted. {0}',
'error_save_field': 'Error saving field.'
}

```

```

    } }
def load_fallback(self, conn):
    for lang_data in self.get_fallback().items():
        for key, text in lang_data.items():
            cur.execute("INSERT OR IGNORE INTO translations (label_key, language, translation) VALUES (?, ?, ?)",
                        conn.commit())
def set_language(self, lang_code):
    self.current_language = lang_code
def tr(self, key):
    conn = sqlite3.connect(self.db_path)
    cur.execute("SELECT translation FROM translations WHERE label_key=? AND language=?", (key,
self.current_language))
    if row := cur.fetchone():
        conn.close()
        return row[0]
    cur.execute("SELECT translation FROM translations WHERE label_key=? AND language='en'", (key,))
    row = cur.fetchone()
    conn.close()
    return row[0] if row else key
def sync_from_postgres(self, db_service):
    if not rows := db_service.execute_query("SELECT label_key, language, translation FROM lang.translations"):
        return
    conn = sqlite3.connect(self.db_path)
    cur = conn.cursor()
    for key, lang, text in rows:
        cur.execute("INSERT OR REPLACE INTO translations (label_key, language, translation) VALUES (?, ?, ?)",
                    conn.commit())
    conn.close()

```

Файл: /home/sergey/Documents/configurate/lang/translator.py (относительно: lang/translator.py)

```

# lang/translator.py
import threading
from database import DatabaseService
class Translator:
    instance = None
    _lock = threading.Lock()
    def __new__(cls):
        if cls.instance is None:
            with cls._lock:
                if cls.instance is None:
                    cls.instance = cls.__new__(cls)
                    cls.instance._initialized = False
            return cls.instance
    def __init__(self):
        if self._initialized:
            return
        self._db = DatabaseService()
        self._current_language = ru
        self._cache = {}
        self._loaded = False
    def load_all(self):
        rows = self._db.execute_query("SELECT label_key, language, translation FROM lang.translations")
        for key, lang, text in rows:
            self._cache[(key, lang)] = text
        self._loaded = True
    def set_language(self, lang_code):
        self.current_language = lang_code
    def tr(self, key):
        if not self._loaded:
            if (key, self.current_language) in self._cache:
                return self._cache[(key, self.current_language)]
            if (key, 'en') in self._cache:
                return self._cache[(key, 'en')]
        return key
    def refresh(self):
        self._cache.clear()
        self._loaded = False
        # переводы будут перезагружены при следующем вызове tr()

```

Файл: /home/sergey/Documents/configurate/layer2_server/server.py (относительно: layer2_server/server.py)

Файл: /home/sergey/Documents/configurate/layer3_client/starter.py (относительно: layer3_client/starter.py)

Файл: /home/sergey/Documents/configurate/makejson.py (относительно: makejson.py)

```

#!/usr/bin/env python3
# -*- coding: utf-8 -*-
Скрипт для выгрузки структуры и содержимого текстовых файлов проекта в JSON.
Корень сканирования - директория, в которой находится сам скрипт.
Результат - project_dump.json в той же директории.

```

```

import os
import json
from pathlib import Path
# Расширения текстовых файлов, которые включаем
TEXT_EXTENSIONS = {
    '.py', '.pyw', '.pyx', '.pxd', '.pyi',
    '.bat', '.sqlite', '.pdx',
    '.html', '.htm', '.xhtml',
    '.css', '.scss', '.sass', '.vue',
    '.json', '.yaml', '.yml', '.toml', '.ini', '.cfg', '.conf',
    '.md', '.markdown', '.rst', '.txt',
    '.sh', '.bash', '.zsh', '.bat', '.cmd', '.ps1',
    '.csv',
    '.java', '.kt', '.h', '.hpp', '.cc', '.cxx',
    '.go', '.r', '.swift', '.scala',
    '.tex', '.latex',
    '.log',
    '.desktop', '.service', '.timer',
    '.ps1', '.psm1', '.psd1',
}
# Расширения, которые игнорируем целиком (бинарные, архивы, медиа)
IGNORE_EXTENSIONS = {
    '.pyc', '.pyo', '.so', '.dll', '.exe', '.bin',
    '.mp4', '.mpeg', '.wav', '.ogg', '.bmp', '.ico', '.webp',
    '.zip', '.tar', '.gz', '.bz2', '.bz', '.7z', '.rar',
    '.doc', '.docx', '.xls', '.pptx',
}
# Имена папок, которые не нужно обходить
IGNORE_DIRS = {
    '.git', '.svn', '.hg', '.idea', '.vscode',
    '.node_modules', '.venv', '.env', '.virtualenv',
    'dist', 'build', '.mypy_cache', '.pytest_cache', '.tox',
}

```

```

'ipynb, checkpoints', '.DS_Store', 'Thumbs.db', '.trash', '.tmp',
}
# Если у вас есть папка с большими временными файлами, добавьте сюда
def should_include_file(file_path):
    """Проверяет, нужно ли включить файл в дамп."""
    ext = os.path.splitext(file_path)[1].lower()
    if ext in IGNORE_EXTENSIONS:
        if ext in TEXT_EXTENSIONS:
            # Если расширение не в списке, можно добавить проверку на текстовый файл с риском
            # Но лучше перечислить нужные расширения явно)
            return False
    return True
def read_file_content(file_path):
    """Пытается прочитать файл как текст (UTF-8 или cp1251)."""
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    except UnicodeDecodeError:
        try:
            with open(file_path, 'r', encoding='cp1251') as f:
                return f.read()
        except:
            return f"[Binary or unsupported encoding: {file_path}]"
    except Exception as e:
        return f"[Error reading file: {e}]"
def collect_directory(root_dir, base_path):
    """Рекурсивно обходит директорию и возвращает словарь:
    - если элемент файл -> записывает содержимое
    - если директория -> рекурсивный вызов
    result = {}
    # Продолжим для стабильности порядка
    for entry in sorted(os.listdir(root_dir)):
        full_path = os.path.join(root_dir, entry)
        if entry in IGNORE_DIRS:
            continue
        if os.path.isdir(full_path):
            if sub: collect_directory(full_path, base_path)
            if sub: result[entry] = sub
        else:
            if should_include_file(entry):
                content = read_file_content(full_path)
                # Сохраняем как строку (можно также добавить мета-информацию)
                result[entry] = content
    return result
def main():
    """определяем корень сканирования: директорию, где находится скрипт
    script_dir = os.path.dirname(os.path.abspath(__file__))
    root_path = script_dir
    # Проверяем, что директория существует
    if not os.path.isdir(root_path):
        print(f"Ошибка: директория {root_path} не существует")
        sys.exit(1)
    output_file = os.path.join(root_path, "project_dump.json")
    print(f"Сканируем: {root_path}")
    print(f"Ждите, это может занять некоторое время...")
    structure = collect_directory(root_path, root_path)
    print(f"Запись в файл: {output_file}")
    with open(output_file, 'w', encoding='utf-8') as f:
        json.dump(structure, f, ensure_ascii=False, indent=2)
    size = os.path.getsize(output_file)
    print(f"Объем: Размер файла {size} байт ({size / 1024:2f}, КБ)")
    print(f"Файл {output_file} можно отправлять ассистенту.")
if __name__ == "__main__":
    main()

```

Файл: /home/sergey/Documents/configurate/makejsonDB.py (относительно: makejsonDB.py)

```

#!/usr/bin/env python3
# coding: utf-8
Работает как из командной строки, так и из IDE (задать параметры в секции DEFAULT_RUNTIME_CONFIG).

import json
import argparse
import psycopg2
from datetime import date, datetime, time
# ===== НАСТРОЙКИ ПО УМОЛЧАНИЮ (для запуска из IDE)
# ПЕРЕМЕННЫЕ, если запускаете скрипт без аргументов командной строки.
DEFAULT_RUNTIME_CONFIG = {
    'schema': 'pg_schema', # 1 - обрабатывать схему, 0 - пропустить (None - использовать значение по умолчанию)
    'meta': None, # None означает: обрабатывать, т.к. схема в ALL_SCHEMAS
    'data': None,
    'auth': None,
    'autodoc': None,
    'public': None,
    'with_data_tables': [] # список строк 'schema.table' для принудительной выгрузки данных
}
output_file = 'pg_schema_dump_full.json'
# =====
# Все возможные схемы
ALL_SCHEMAS = ['ext', 'meta', 'data', 'auth', 'autodoc', 'lang', 'public']
# Конфигурация подключения (можно переопределить через аргумент config_file)
DEFAULT_CONFIG = {
    'host': 'localhost',
    'port': 5432,
    'dbname': 'mozart_erp',
    'password': 'SQLSerlra'
}
def get_connection(config):
    return psycopg2.connect(**config)
def json_serializer(obj):
    if isinstance(obj, (datetime, date)):
        return obj.isoformat()
    if isinstance(obj, time):
        return obj.strftime('%H:%M:%S')
    if isinstance(obj, decimal.Decimal):
        return float(obj)
    raise TypeError(f"Object of type {type(obj)} is not JSON serializable")
def safe_fetch(cursor, query, params=None):
    try:
        cursor.execute(query, params)
    except Exception as e:
        print(f"Ошибка Запроса: {e}")
        return []
def get_schema_comment(cursor, schema_name):
    rows = safe_fetch(cursor, f"SELECT obj_description(oid) FROM pg_namespace WHERE nsname = %s",
                        (schema_name,))
    return rows[0][0] if rows else None

```

```

def get_tables_metadata(cursor, schema_name):
    tables = []
    rows = safe_fetch(cursor, """
    SELECT obj_description('%s.%s'::regclass),
    FROM information_schema.tables
    WHERE table_schema = %s AND table_type = 'BASE TABLE'
    ORDER BY table_name
    """, (schema_name,))
    for (table_name, comment) in rows:
        if comment is None:
            try:
                cursor.execute("SELECT obj_description('%s.%s'::regclass)", (schema_name, table_name))
            except:
                pass
            row = cursor.fetchone()
            if row:
                comment = row[0]
            else:
                columns = []
                rows = safe_fetch(cursor, """
                SELECT column_name, data_type, is_nullable, column_default
                FROM information_schema.columns
                WHERE table_schema = %s AND table_name = %s
                ORDER BY ordinal_position
                """, (schema_name, table_name))
                for (col_name, data_type, is_nullable, col_default) in rows:
                    col_comment = None
                    try:
                        cursor.execute("""
                        SELECT obj_description('%s.%s'::regclass), ordinal_position
                        FROM information_schema.columns
                        WHERE table_schema = %s AND table_name = %s
                        AND column_name = %s
                        """, (schema_name, table_name, schema_name, table_name, col_name))
                        cc = cursor.fetchone()
                        if cc:
                            col_comment = cc[0]
                    except:
                        pass
                    columns.append({
                        "name": col_name,
                        "data_type": data_type,
                        "is_nullable": is_nullable,
                        "default": col_default,
                        "comment": col_comment
                    })
            pk_rows = safe_fetch(cursor, """
            SELECT column_name, ccu.table_schema, ccu.table_name, ccu.column_name
            FROM information_schema.table_constraints tc
            JOIN information_schema.key_column_usage kc
            ON tc.constraint_name = kc.constraint_name
            WHERE tc.table_schema = %s AND tc.constraint_type = 'PRIMARY KEY'
            ORDER BY kc.ordinal_position()
            """, (schema_name,))
            fk_rows = safe_fetch(cursor, """
            SELECT kc.column_name, ccu.table_schema, ccu.table_name, ccu.column_name
            FROM information_schema.table_constraints tc
            JOIN information_schema.key_column_usage kc
            ON tc.constraint_name = kc.constraint_name
            JOIN information_schema.constraint_column_usage ccu
            ON tc.constraint_name = ccu.constraint_name
            WHERE tc.table_schema = %s AND tc.constraint_type = 'FOREIGN KEY'
            """, (schema_name,))
            unique_rows = safe_fetch(cursor, """
            SELECT column_name, tc.table_schema, tc.table_name, tc.constraint_name
            FROM information_schema.table_constraints tc
            JOIN information_schema.key_column_usage kc
            ON tc.constraint_name = kc.constraint_name
            WHERE tc.table_schema = %s AND tc.constraint_type = 'UNIQUE'
            ORDER BY kc.ordinal_position()
            """, (schema_name,))
            unique_rows = [row for row in unique_rows]
            tables.append({
                "name": table_name,
                "comment": comment,
                "columns": columns,
                "primary_key": pk,
                "foreign_keys": fk,
                "unique_constraints": unique
            })
    return tables

def get_views_metadata(cursor, schema_name):
    views = []
    rows = safe_fetch(cursor, """
    SELECT obj_description('%s.%s'::regclass),
    FROM information_schema.views
    WHERE table_schema = %s
    ORDER BY table_name
    """, (schema_name,))
    for (view_name, comment) in rows:
        if comment is None:
            try:
                cursor.execute("SELECT obj_description('%s.%s'::regclass)", (schema_name, view_name))
            except:
                pass
            row = cursor.fetchone()
            if row:
                comment = row[0]
            else:
                columns = []
                rows = safe_fetch(cursor, """
                SELECT column_name, data_type
                FROM information_schema.columns
                WHERE table_schema = %s AND table_name = %s
                ORDER BY ordinal_position
                """, (schema_name, view_name))
                for (col_name, data_type) in rows:
                    columns.append({
                        "name": col_name,
                        "data_type": data_type
                    })
            views.append({
                "name": view_name,
                "comment": comment,
                "columns": columns
            })
    return views

def get_functions_metadata(cursor, schema_name):
    functions = []
    rows = safe_fetch(cursor, """
    SELECT pg_get_functiondef(p.oid), pg_get_function_result(p.oid),
    FROM pg_proc p
    JOIN pg_namespace n ON p.pronamespace = n.oid
    ORDER BY p.proname
    """, (schema_name,))
    for (row) in rows:
        func_name = row[0]
        arguments = row[1]
        result_type = row[2]
        kind = row[5]
        if kind == 'p':
            kind = 'procedure'
        else:
            kind = 'function'
        functions.append({
            "name": func_name,
            "arguments": arguments,
            "result_type": result_type,
            "kind": kind
        })
    return functions

def get_triggers_metadata(cursor, schema_name):
    triggers = []
    rows = safe_fetch(cursor, """
    SELECT tgname, tgtype, tgoid::regproc, pg_get_triggerdef(t.oid)
    FROM pg_trigger t
    JOIN pg_class c ON t.tgrelid = c.oid
    WHERE c.relnamespace = %s AND NOT tgisinternal
    ORDER BY tgname
    """, (schema_name,))
    for (tgname, tgtype, func_name, definition) in rows:
        if tgtype & 1:
            event_parts.append(("BEFORE"))
        if tgtype & 4:
            event_parts.append(("UPDATE"))
        if tgtype & 8:
            event_parts.append(("INSERT"))

```

```

if tctype & 16: event_parts.append("DELETE")
if tctype & 64: event_parts.append("UPDATE")
if tctype & 128: event_parts.append("TRUNCATE")
trigger_str.append("(" + event_parts + ") if event_parts else \"UNKNOWN\"")
trigger_name.append(
    "table": f"{schema_name}.{relname}",
    "event": event_str,
    "definition": definition,
)
return triggers

def get_indexes_metadata(cursor, schema_name):
    rows = safe_fetch(cursor, """
    SELECT i.relname, pg_get_indexdef(idx.indexrelid), idx.indisunique, am.amname
    JOIN pg_class c ON i.indexrelid = c.oid
    JOIN pg_namespace n ON c.relnamespace = n.oid
    JOIN pg_am a ON i.amname = a.oid
    ORDER BY relname
    """)
    for idx_name, table_name, definition, is_unique, method in rows:
        indexes.append(
            "name": f"{schema_name}.{table_name}.{idx_name}",
            "definition": definition,
            "unique": is_unique,
            "method": method,
        )
    return indexes

def get_rules_metadata(cursor, schema_name):
    rows = safe_fetch(cursor, """
    SELECT r.relname, relname, pg_get_ruledef(r.oid)
    JOIN pg_class c ON r.ev_class = c.oid
    JOIN pg_namespace n ON c.relnamespace = n.oid
    WHERE r.relname NOT LIKE '_RETURN'
    ORDER BY relname
    """)
    for rule_name, relname, definition in rows:
        rules.append(
            "name": f"{schema_name}.{rule_name}",
            "definition": definition,
        )
    return rules

def get_table_data(cursor, schema_name, table_name, limit=500):
    try:
        cursor.execute(sql.SQL("SELECT * FROM {f}.{f} LIMIT %s").format(
            sql.Identifier(schema_name),
            sql.Identifier(table_name)
        ))
        rows = cursor.fetchall()
        if not rows:
            return []
        columns = [desc[0] for desc in cursor.description]
        for row in rows:
            row_dict = {}
            for i, col in enumerate(columns):
                val = row[i]
                if isinstance(val, (datetime, date, time)):
                    row_dict[col] = val.isoformat()
                elif isinstance(val, Decimal):
                    row_dict[col] = str(val)
            result.append(row_dict)
    except Exception as e:
        print(f"Ошибка выгрузки данных из {schema_name}.{table_name}: {e}")
    return []

def get_translations_data(cursor):
    rows = safe_fetch(cursor, "SELECT label key, language, translation FROM lang.translations ORDER BY language, label key")
    return [{"label_key": r[0], "language": r[1], "translation": r[2]} for r in rows]

def main():
    # Определяем, откуда брать параметры: из командной строки или из DEFAULT_RUNTIME_CONFIG
    if len(sys.argv) == 1:
        print("Запуск без аргументов: используем DEFAULT_RUNTIME_CONFIG")
        schemas_config = DEFAULT_RUNTIME_CONFIG['schemas']
        with_data_tables = DEFAULT_RUNTIME_CONFIG['with_data_tables']
        config_file = None
    else:
        # Парсим аргументы командной строки
        parser = argparse.ArgumentParser(description='Расширенный дамп структуры PostgreSQL и выборочно')
        parser.add_argument('config_file', nargs='?', help='JSON файл с параметрами подключения (опционально)')
        parser.add_argument('--type', type=int, choices=[0, 1], default=0, help='тип структуры, 0: данные из meta/auth/autodoc/lang/public, 1: все данные')
        for schema in AP - SCHEMAS:
            parser.add_argument(f'--schema {schema}', type=int, choices=[0, 1], default=None, help=f'Обработка схемы {schema} (0: данные из meta/auth/autodoc/lang/public, 1: все данные)')
        parser.add_argument('--table', type=str, help='Имя таблицы для выгрузки (можно несколько)')
        parser.add_argument('--table_out', type=str, help='Имя выходного JSON файла')
        args = parser.parse_args()

        # Формируем словарь схем из аргументов
        schemas_config = SCHEMAS
        for schema in args.schema:
            if val is not None:
                schemas_config[schema] = val
            else:
                schemas_config[schema] = None
        with_data_tables = args.with_data_table
        config_file = args.config_file

    # Загрузка конфигурации подключения
    with open(config_file, 'r', encoding='utf-8') as f:
        config_data = json.load(f)
        db_cfg = config_data.get('db', {})
        host = db_cfg.get('host', DEFAULT_DB_CONFIG['host'])
        port = db_cfg.get('port', DEFAULT_DB_CONFIG['port'])
        dbname = db_cfg.get('name', DEFAULT_DB_CONFIG['dbname'])
        user = db_cfg.get('user', DEFAULT_DB_CONFIG['user'])
        password = db_cfg.get('pass', DEFAULT_DB_CONFIG['password'])
    else:
        db_config = DEFAULT_DB_CONFIG

    # Определяем, какие схемы обрабатывать
    schemas_to_process = SCHEMAS
    for schema in AP - SCHEMAS:
        if schema in schemas_config.get(schema):
            if val is not None:
                schemas_to_process.append(schema)
            else:
                # флаг не задан - обрабатываем схему (по умолчанию все схемы)
                schemas_to_process.append(schema)

    # Определяем, для каких схем выгружать данные таблиц
    schemas_to_load_data = set()
    if all(mode == 1):
        schemas_to_load_data = set(schemas_to_process)
    else:
        # as mode == 0
        schemas_to_load_data = set(schemas_to_process) - {'data', 'ext'}

    # добавляем таблицы, явно указанные в with_data_tables
    for tab in with_data_tables:

```



```

if ' ' in tab:
    sch, tbl = tab.split(' ', 1)
    if sch in schemas_to_process:
        extra_tables_info.append((sch, tbl))
    else:
        print(f"Предупреждение: таблица {tab} не указана в формате schema.table, пропускаем")
try:
    conn = get_connection(db_config)
    conn.autocommit = True
    result = {}
    for schema in schemas_to_process:
        schema_data = {}
        schema_comment = get_schema_comment(cursor, schema),
        tables = get_tables_metadata(cursor, schema),
        views = get_views_metadata(cursor, schema),
        functions = get_functions_metadata(cursor, schema),
        indexes = get_indexes_metadata(cursor, schema),
        rules = get_rules_metadata(cursor, schema)
    }
    if schema in schemas_to_load_data:
        data_tables = get_data_tables_for_schema(schema)
        if extra_tables_info:
            tables_to_load = [tbl for (sch, tbl) in extra_tables_info if sch == schema]
        else:
            tables_to_load = [t["name"] for t in schema_data["tables"]]
        for table_name in tables_to_load:
            rows = get_table_data(cursor, schema, table_name, limit=500)
            data_tables[table_name] = rows
        if data_tables:
            result[schema] = schema_data
        result[schema] = schema_data
    if 'lang' in schemas_to_process:
        result["lang_translations_data"] = get_translations_data(cursor)
    cursor.close()
    with open(output_file, 'w', encoding='utf-8') as f:
        json.dump(result, f, ensure_ascii=False, indent=2, default=json_serializer)
    print(f"Готово. Результат сохранён в {output_file}")
except Exception as e:
    print(f"Критическая ошибка: {e}")
    sys.exit(1)
if __name__ == "__main__":
    main()

```

Файл: [/home/sergey/Documents/configurate/makePdf.py](#) (относительно: [makePdf.py](#))

```

#!/usr/bin/env python3
# coding: utf-8
Собирает все .py файлы из указанной директории (рекурсивно) и формирует PDF.
Между файлами - разделитель.

import os
from pathlib import Path
from reportlab.lib.pagesizes import A4
from reportlab.lib.units import mm
from reportlab.pdfbase import pdfmetrics
from reportlab.pdfbase.ttfonts import TTFont
from reportlab.lib.colors import black, blue

# ПУТИ И НАСТРОЙКИ
ROOT_DIR = os.path.dirname(os.path.abspath(__file__)) # по умолчанию - папка скрипта
OUTPUT_PDF = os.path.join(ROOT_DIR, "all_python_files.pdf")
# Расширения, которые включаем - только .py
EXTENSIONS = {'.py'}
# Игнорируемые папки
IGNORE_DIRS = ['.git', '.svn', '.hg', '.idea', '.vscode',
               '.pycache', '.venv', '.env', '.pyenv', '.virtualenv',
               '.build', '.cache', '.DS_Store', 'thumbs.db', '.trash', '.tmp',
               'logs', 'backup', 'old']

# Попробуем загрузить шрифт с поддержкой кириллицы (если есть)
CYRILLIC_FONT = None
for font_path in [
    "/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf",
    "/usr/share/fonts/truetype/liberation/LiberationSans-Regular.ttf",
]:
    if os.path.exists(font_path):
        try:
            pdfmetrics.registerFont(TTFont('CyrillicFont', font_path))
            CYRILLIC_FONT = CyrillicFont
            break
        except:
            pass

if not CYRILLIC_FONT:
    print("Предупреждение: шрифт с поддержкой кириллицы не найден, используется стандартный")
    CYRILLIC_FONT = 'Helvetica'

def should_include(file_path):
    ext = os.path.splitext(file_path)[1].lower()
    return ext in EXTENSIONS

def collect_py_files(root_dir):
    files = []
    for dirpath, dirnames, filenames in os.walk(root_dir):
        for filename in filenames:
            if dirpath not in IGNORE_DIRS:
                file_path = os.path.join(dirpath, filename)
                if should_include(file_path):
                    files.append(file_path)
    files.sort()
    return files

def read_file_content(file_path):
    try:
        with open(file_path, 'r', encoding='utf-8') as f:
            return f.read()
    except UnicodeDecodeError:
        try:
            with open(file_path, 'r', encoding='cp1251') as f:
                return f.read()
        except:
            return f"[Не удалось прочитать файл {file_path}]"
    except Exception as e:
        return f"[Ошибка чтения: {e}]"

def add_text_to_canvas(canvas_obj, text, font_name, font_size,
                      x, y, max_width, page_height, margin_top, margin_bottom, leading):
    Разбивает текст на строки, рисует на canvas с учётом ширины страницы.
    Возвращает новую позицию y.
    canvas_obj.setFont(font_name, font_size)
    for line in text.splitlines():
        if canvas_obj.stringWidth(line, font_name, font_size) > max_width:
            words = line.split()
            cur_line = ""

```

```

        for w in words:
            test_line = cur_line + (' ' if cur_line else '') + w
            if canvas.obj.stringWidth(test_line, font_name, font_size) <= max_width:
                cur_line = test_line
            else:
                if cur_line:
                    lines.append(cur_line)
                if cur_line == w:
                    lines.append(cur_line)
                else:
                    lines.append(line)
        for line in lines:
            if y + leading < margin_bottom:
                canvas.obj.drawString(font_name, font_size)
                y = page_height - margin_bottom
            canvas.obj.drawString(x, y, line)
            return y
        leading = 4.5

def create_pdf(files, output_pdf):
    page_width = canvas.width
    page_height = canvas.height
    margin_bottom = 20 * mm
    margin_top = 15 * mm
    text_width = page_width - margin_left - margin_right
    leading = 4.5
    font_name = CYRILLIC_FONT
    heading_font = 'Helvetica-Bold' if font_name == 'Helvetica' else font_name
    heading_font_size = 12
    y = page_height - margin_top
    for file_path in files:
        file_path = os.path.realpath(file_path, ROOT_DIR)
        # добавляем в файл пути (относительно: {rel_path})"
        display = f'{full_path_display} (относительно: {rel_path})"'
        if y - heading_font_size - leading < margin_bottom:
            c.showPage()
            c.setFillColor(font, heading_font_size)
            c.drawString(margin_left, y, display)
            c.setFillColor(black)
            content = read_file_content(file_path)
            y = add_text_to_canvas(content, font_name, code_font_size,
                                   page_height, margin_top, margin_bottom, leading)
            if y - 10 < margin_bottom:
                y = page_height - margin_top
            else:
                c.setStrokeColor(black)
                y = 10
            c.fillRect(margin_left, y, page_width - margin_right, y)
        c.save()
    print(f'PDF создан: {output_pdf}')

def main():
    if not os.path.isdir(ROOT_DIR):
        print(f'Ошибка: директория {ROOT_DIR} не существует')
        sys.exit(1)
    py_files = collect_py_files(ROOT_DIR)
    print(f'Найдено Python-файлов: {len(py_files)}')
    if not py_files:
        return
    create_pdf(py_files, OUTPUT_PDF)

if __name__ == '__main__':
    main()

```

[Файл: /home/sergey/Documents/configurate/mozart_import/__init__.py \(относительно: мо](#)

```

# -*- coding: utf-8 -*-
from core.models import (
    SourceField, SourceTable, SourceSchema,
    MatchedField, MatchedEntity, MappingProposal,
    ImportProfile
)
from core.mozart.matcher import MozartMatcher
from core.metadata_creator import MetadataCreator
from core.import_profile import ImportProfileManager
from core.schema_analyzer import SchemaAnalyzer
from designer.designer_v2 import ImportDesignerV2
from designer.entity_creator import EntityCreator
from connectors.file_connector import FileConnector
from connectors.db_connector import DBConnector
from connectors.url_connector import URLConnector
from utils.ext_helper import ExtHelper

all = [Type, 'FileFormat', 'FieldType',
       'SourceField', 'SourceTable', 'SourceSchema',
       'MatchedField', 'MatchedEntity', 'MappingProposal',
       'ImportProfile',
       'MetadataCreator',
       'ImportProfileManager',
       'SchemaAnalyzer',
       'ImportDesignerV2',
       'EntityCreator',
       'FileConnector',
       'DBConnector',
       'URLConnector',
       'ExtHelper']

```

[Файл: /home/sergey/Documents/configurate/mozart_import/connectors/__init__.py \(относит](#)

```

# moztart.import/connectors/__init__.py
# (пустой файл, обозначает пакет)

```

[Файл: /home/sergey/Documents/configurate/mozart_import/connectors/base_connector.py](#)

```

# moztart.import/connectors/base_connector.py
from abc import ABC, abstractmethod
from typing import Any, Dict, List, Optional, Generator
from core.models import SourceSchema, SourceTable, SourceField, FieldType, SourceType
class BaseConnector(ABC):
    """Абстрактный базовый класс для всех коннекторов к источникам данных."""

```

```

def __init__(self, source_path: str, **kwargs):
    self.source_path = source_path
    self.options = kwargs
    @abstractmethod
    def connect(self) -> bool:
        pass
    """Устанавливает соединение с источником. Возвращает True при успехе."""
    @abstractmethod
    def disconnect(self) -> None:
        pass
    """Закрывает соединение."""
    @abstractmethod
    def get_schema(self) -> SourceSchema:
        pass
    """Извлекает схему источника (таблицы, поля, типы, связи)."""
    @abstractmethod
    def get_table_sample(self, table_name: str, limit: int = 100) -> List[Dict[str, Any]]:
        pass
    """Возвращает первые limit записей из указанной таблицы."""
    @abstractmethod
    def iter_records(self, table_name: str, batch_size: int = 1000) -> Generator[List[Dict[str, Any]], None, None]:
        pass
    """Генератор, возвращающий записи пакетами для итеративной обработки."""
    @abstractmethod
    def get_row_count(self, table_name: str) -> Optional[int]:
        pass
    """Возвращает приблизительное количество записей в таблице."""

```

Файл: /home/sergey/Documents/configurate/mozart_import/connectors/db_connector.py (0

```

# mozart_import/connectors/db_connector.py
import sys
from typing import List, Dict, Any, Optional, Generator
from ..core.connector import BaseConnector, SourceTable, SourceField, FieldType, SourceType
try:
    import psycopg2
    PSYCOPG2_AVAILABLE = True
except ImportError:
    PSYCOPG2_AVAILABLE = False
try:
    import pymysql
    PYMYSQL_AVAILABLE = True
except ImportError:
    PYMYSQL_AVAILABLE = False
try:
    import pyodbc
    PYODBC_AVAILABLE = True
except ImportError:
    PYODBC_AVAILABLE = False
try:
    import cx_Oracle
    CX_ORACLE_AVAILABLE = True
except ImportError:
    CX_ORACLE_AVAILABLE = False
class DBConnector(BaseConnector):
    """Коннектор для реляционных баз данных: PostgreSQL, MySQL, MSSQL, Oracle, SQLite.
    Параметры подключения передаются через kwargs:
    host, port, database, user, password (для SQLite только database — путь к файлу)
    """
    def __init__(self, source_path: str, **kwargs):
        self.db_type = kwargs.get('db_type', 'postgresql')
        self.host = kwargs.get('host', 'localhost')
        self.port = kwargs.get('port', 5432)
        self.database = kwargs.get('database')
        self.user = kwargs.get('user')
        self.password = kwargs.get('password')
        self.cursor = None
    def connect(self) -> bool:
        if not PSYCOPG2_AVAILABLE:
            raise ImportError("Установите psycopg2: pip install psycopg2-binary")
        self.conn = psycopg2.connect(
            host=self.host or 5432,
            database=self.database,
            user=self.user,
            password=self.password
        )
        self.cursor = self.conn.cursor()
        elif self.db_type == 'mysql':
            if not PYMYSQL_AVAILABLE:
                raise ImportError("Установите pymysql: pip install pymysql")
            self.conn = pymysql.connect(
                host=self.host or 3306,
                database=self.database,
                user=self.user,
                password=self.password,
                charset='utf8mb4'
            )
            self.cursor = self.conn.cursor()
        elif self.db_type == 'mssql':
            raise ImportError("Установите pyodbc: pip install pyodbc")
            driver = 'ODBC Driver 17 for SQL Server'
            conn_str = 'DRIVER={driver};SERVER={self.host};PORT={self.port or 1433};DATABASE={self.database};UID={self.user};PWD={self.password}'
            self.conn = pyodbc.connect(conn_str)
            self.cursor = self.conn.cursor()
        elif self.db_type == 'oracle':
            if not CX_ORACLE_AVAILABLE:
                raise ImportError("Установите cx_Oracle: pip install cx-Oracle")
            self.conn = cx_Oracle.connect(dsn=self.host, self.port or 1521, service_name=self.database)
            self.cursor = self.conn.cursor()
        elif self.db_type == 'sqlite':
            # для SQLite database - путь к файлу
            if not self.database:
                raise ValueError("Для SQLite необходимо указать database (путь к файлу)")
            self.conn = sqlite3.connect(self.database)
            self.cursor = self.conn.cursor()
        else:
            raise ValueError(f"Неподдерживаемый тип БД: {self.db_type}")
        return True
    def disconnect(self) -> None:
        if self.cursor:
            self.cursor.close()
        if self.conn:
            self.conn.close()
        self.cursor = None
        self.conn = None
    def get_schema(self) -> SourceSchema:
        # получаем список таблиц (исключаем служебные)
        self.cursor.execute("SELECT table_name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%'")
        tables = [row[0] for row in self.cursor.fetchall()]
        source_tables = []
        self.cursor.execute("PRAGMA table_info('{t}')"
        for table in tables:
            cols = self.cursor.fetchall()
            col_names = [col[1] for col in cols]
            # определяем тип поля

```

```

        if 'INT' in col_type:
            type = FieldType.INTEGER
        elif 'VARCHAR' in col_type or 'FLOAT' in col_type or 'DOUBLE' in col_type:
            type = FieldType.FLOAT
        elif 'CHAR' in col_type or 'TEXT' in col_type or 'CLOB' in col_type:
            type = FieldType.STRING
        elif 'BLOB' in col_type:
            type = FieldType.BINARY
        else:
            type = FieldType.STRING
        length = None
        fields.append(SourceField(
            name=col[1],
            type=type,
            length=length,
            nullable=not col[3]
        ))
    source_tables.append(SourceTable(name=t, fields=fields))
    return SourceSchema(
        source_type=SourceType.DATABASE,
        tables=source_tables
    )

def get_schema(self) -> SourceSchema:
    if self.db_type == 'postgresql':
        return self.get_postgresql_schema()
    elif self.db_type == 'mysql':
        return self.get_mysql_schema()
    # TODO: реализовать для MSSQL, Oracle аналогично
    raise NotImplementedError("Получение схемы для {self.db_type} пока не реализовано")

def get_table_sample(self, table_name: str, limit: int = 100) -> List[Dict[str, Any]]:
    if self.db_type == 'postgresql':
        self._cursor.execute(f"SELECT * FROM \"{table_name}\" LIMIT ?", (limit,))
        rows = self._cursor.fetchall()
        return [dict(zip(columns, row)) for row in rows]
    else:
        self._cursor.execute(f"SELECT * FROM \"{table_name}\" LIMIT %s", (limit,))
        columns = self._cursor.fetchall()
        rows = [dict(zip(columns, row)) for row in rows]
    return [dict(zip(columns, row)) for row in rows]

def iter_records(self, table_name: str, batch_size: int = 1000) -> Generator[List[Dict[str, Any]], None, None]:
    if self.db_type == 'sqlite':
        offset = 0
        while True:
            self._cursor.execute(f"SELECT * FROM \"{table_name}\" LIMIT ? OFFSET ?", (batch_size, offset))
            rows = self._cursor.fetchall()
            if not rows:
                break
            columns = [desc[0] for desc in self._cursor.description]
            batch = [dict(zip(columns, row)) for row in rows]
            yield batch
            offset += batch_size
    else:
        offset = 0
        while True:
            self._cursor.execute(f"SELECT * FROM \"{table_name}\" LIMIT %s OFFSET %s", (batch_size, offset))
            rows = self._cursor.fetchall()
            if not rows:
                break
            columns = [desc[0] for desc in self._cursor.description]
            batch = [dict(zip(columns, row)) for row in rows]
            yield batch
            offset += batch_size
    break

def get_row_count(self, table_name: str) -> Optional[int]:
    self._cursor.execute(f"SELECT COUNT(*) FROM \"{table_name}\"")
    return self._cursor.fetchone()[0]

```

Файл: /home/sergey/Documents/configurate/mozart_import/connectors/file_connector.py (0

```

# mozart_import/connectors/file_connector.py
import os
from typing import List, Dict, Optional, Generator
from base.connector import BaseConnector
from ..core.models import SourceSchema, SourceTable, SourceField, FieldType, SourceType, FileFormat

try:
    from dbfread import DBF
    DBF_AVAILABLE = True
except ImportError:
    DBF_AVAILABLE = False

try:
    import pandas as pd
    PANDAS_AVAILABLE = True
except ImportError:
    PANDAS_AVAILABLE = False

try:
    from lxml import etree as lxml_etree
    LXML_AVAILABLE = True
except ImportError:
    LXML_AVAILABLE = False

def clean_key(key: str) -> str:
    Очищает ключ от пространств имён XML и заменяет проблемные символы.
    Например: {turn:cb,r:ed.v2.0;participaninfo} -> 'participaninfo'
    if '\\' in key:
        key = key.split('\\')[1][1:]
    # Замена дефисов и точек на подчёркивания (опционально)
    return key.replace('-', '_').replace('.', '_')

def flatten_xml_element(elem, prefix=""):
    Рекурсивно преобразует XML-элемент lxml в плоский словарь.
    Ключи очищаются от пространств имён.
    result = {}
    for attr, value in elem.attrib.items():
        full_key = f"{prefix}{clean_attr}" if prefix else clean_attr
        result[full_key] = value
    # Текстовое содержимое, если нет дочерних элементов
    if not (len(elem) and elem.text and elem.text.strip()):
        result[text_key] = elem.text.strip()
    # Дочерние элементы
    for child in elem:
        child_data = flatten_xml_element(child, child_prefix)
        for k, v in child_data.items():
            if k not in result:
                result[k] = v
    return result

class FileConnector(BaseConnector):
    def __init__(self, source_path: str, file_format: FileFormat, **kwargs):
        super().__init__(source_path, **kwargs)
        self.data = None
        self.file_format = file_format
    def connect(self) -> bool:
        if not os.path.exists(self.source_path):

```



```

        raise FileNotFoundError(f"Файл не найден: {self.source_path}")
    self._load_data()
    return True
def disconnect(self) -> None:
    self._data = None
def _load_data(self):
    if self._file_format == FileFormat.JSON:
        with open(self.source_path, encoding='utf-8') as f:
            self._data = (json.loads(f.read()))
    elif self._file_format == FileFormat.XML:
        if not XML_AVAILABLE:
            raise ImportError("Установите lxml; pip install lxml")
        tree = etree.parse(self.source_path)
        root = tree.getroot()
        records = []
        for elem in root:
            records.append(record(elem))
        self._data = records
    elif self._file_format == FileFormat.DBF:
        if not DBF_AVAILABLE:
            raise ImportError("Установите dbfread; pip install dbfread")
        table = DBF(self.source_path, ignore_missing_memory=True)
        self._data = list(table)
    elif self._file_format == FileFormat.CSV, FileFormat.EXCEL:
        if not PANDAS_AVAILABLE:
            raise ImportError("Установите pandas; pip install pandas")
        if self._file_format == FileFormat.CSV:
            self._data = pd.read_csv(self.source_path)
        else:
            self._data = pd.read_excel(self.source_path)
        self._data = df.to_dict(orient='records')
    else:
        raise ValueError(f"Неподдерживаемый формат: {self.file_format}")
def get_schema(self) -> SourceSchema:
    if self._data is None:
        self._load_data()
    if not self._data:
        return SourceSchema(source_type=SourceType.FILE, source_path=self.source_path)
    sample = self._data[0]
    fields = []
    for name, value in sample.items():
        if isinstance(value, str):
            length = len(value)
        elif isinstance(value, int):
            length = None
        elif isinstance(value, float):
            length = None
        elif isinstance(value, bool):
            length = None
        else:
            length = None
        type = FieldType.STRING
        fields.append(SourceField(
            name=name,
            type=type,
            length=length,
            nullable=False,
            sample_values=[value]
        ))
    table = SourceTable(
        name=self._path.basename(self.source_path),
        sample_rows=self._data[:10]
    )
    return SourceSchema(
        source_type=SourceType.FILE,
        source_path=self.source_path,
        tables=[table]
    )
def get_table_sample(self, table_name: str = None, limit: int = 100) -> List[Dict[str, Any]]:
    if self._data is None:
        self._load_data()
    return self._data[:limit]
def iter_records(self, table_name: str = None, batch_size: int = 1000) -> Generator[List[Dict[str, Any]], None, None]:
    if self._data is None:
        self._load_data()
    for i in range(0, len(self._data), batch_size):
        yield self._data[i:i+batch_size]
def get_row_count(self, table_name: str = None) -> Optional[int]:
    if self._data is None:
        self._load_data()
    return len(self._data)

```

Файл: /home/sergey/Documents/configurate/mozart_import/connectors/url_connector.py (0

```

# mozart_import/connectors/url_connector.py
import requests
from typing import List, Dict, Any, Optional, Generator, Callable, Union
from ..base.connector import BaseConnector
from ..core.models import SourceSchema, SourceTable, SourceField, FieldType, SourceType
import os
from ..core.models import SourceSchema, SourceTable, SourceField, FieldType, SourceType, FileFormat
class URLConnector(BaseConnector):
    """Коннектор для HTTP/HTTPS источников (REST API, RSS, веб-страницы)."""
    headers: dict с HTTP-заголовками
    pagination: dict с настройками пагинации
    : page: имя параметра page, offset, cursor)
    : page_size: количество записей на странице
    : data_path: путь к файлу со списком до URL следующей страницы
    data: list с данными (например, data.items())
    body: dict для POST-запроса
    def __init__(self, source_path: str, **kwargs):
        self._method = kwargs.get('method', 'GET')
        self._headers = kwargs.get('headers', {})
        self._data_path = kwargs.get('data_path', None)
        self._body = kwargs.get('body', None)
        self.session = None
    def connect(self) -> bool:
        self.session = requests.Session()
        self.session.headers.update(self.headers)
        return True
    def disconnect(self) -> None:
        if self.session:
            self.session.close()
    def _get_json_path(self, data: Any, path: str) -> Any:
        if not data:
            return None
        parts = path.split('.')
        for part in parts:
            if isinstance(data, dict) and part in data:
                data = data[part]
            elif isinstance(data, list) and part.isdigit() and int(part) < len(data):
                data = data[int(part)]
            else:
                return None
        return data
    def _extract_records(self, response_json: Dict) -> List[Dict]:

```



```

if self.data_path:
    data = self._get_json_path(response_json, self.data_path)
    if isinstance(data, list):
        return data
    else:
        return [data]
# Если data_path не задан, предполагаем, что JSON сам является массивом
# Или сможем найти ключ, который содержит список
for value in response_json.values():
    if isinstance(value, list):
        return value
def get_next_url(self, response_json: Dict) -> Optional[str]:
    if self.data_path:
        next_path = self.pagination.get('next_url_path')
        if next_path:
            return self._get_json_path(response_json, next_path)
    return None
def fetch_all(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
    url = self.source_path
    page = 1
    offset = 0:
    while True:
        paginated_url = self._build_paginated_url(url, page, offset)
        response = self.session.request(self.method, paginated_url, json=self.body)
        response.raise_for_status()
        data = response.json()
        results.extend(records(data))
        if len(results) == limit:
            return results
        next_url = self._get_next_url(data)
        if next_url:
            url = next_url
            if self.pagination.get('type') == 'page':
                page += 1
            if self.pagination.get('type') == 'offset':
                offset = self.pagination.get('page_size', 100)
            continue
        break
    return results
def _build_paginated_url(self, base_url: str, page: int, offset: int) -> str:
    param_type = self.pagination.get('type')
    param = self.pagination.get('param_name', 'page')
    separator = '&' if param in base_url else '?'
    elif param_type == 'offset':
        param = self.pagination.get('param_name', 'offset')
        page_size = self.pagination.get('page_size', 100)
        return '{base_url}{separator}{param}={offset}&limit={page_size}'
    else:
        return base_url
def get_schema(self) -> SourceSchema:
    # Для URL-источников схему получаем из первой страницы
    sample_data = self.fetch_all(limit=10)
    if not sample_data:
        return SourceSchema(source_type=SourceType.URL, source_path=self.source_path)
    sample = sample_data[0]
    for name, value in sample.items():
        if isinstance(value, list):
            if isinstance(value, str):
                ftype = FieldType.STRING
            elif isinstance(value, int):
                ftype = FieldType.INTEGER
            elif isinstance(value, float):
                ftype = FieldType.FLOAT
            elif isinstance(value, bool):
                ftype = FieldType.BOOLEAN
            elif isinstance(value, dict) or isinstance(value, list):
                ftype = FieldType.STRING
            else:
                ftype = FieldType.STRING
        fieldtype.append(name=name, field_type=ftype, nullable=True))
    source_table = SourceTable(
        name='data',
        sample_rows=sample_data[:5]
    )
    return SourceSchema(
        source_type=SourceType.URL,
        source_path=self.source_path,
        tables=[source_table]
    )
def get_table_sample(self, table_name: str = "data", limit: int = 100) -> List[Dict[str, Any]]:
    return self.fetch_all(limit=limit)
def iter_records(self, table_name: str = "data", batch_size: int = 1000) -> Generator[List[Dict[str, Any]], None, None]:
    all_data = self.fetch_all()
    for i in range(0, len(all_data), batch_size):
        yield all_data[i:i+batch_size]
def get_row_count(self, table_name: str = "data") -> Optional[int]:
    # Для API точное количество получить сложно, возвращаем None
    return None

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/__init__.py (относительно

```

# mozart_import/core/__init__.py
from models import Format, FieldType, Schema
from matched_field import MatchedEntity, MappingProposal,
import Profile
)
from :mozart_matcher import MozartMatcher
from :metadata_creator import MetadataCreator
from :import_engine import ImportEngine
from :schema_analyzer import SchemaAnalyzer
__all__ = ['Type', 'FileFormat', 'FieldType',
'MatchedField', 'MatchedEntity', 'MappingProposal',
'ImportProfile',
'MetadataCreator',
'ImportEngine',
'SchemaAnalyzer']

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/import_engine.py (относи

```

# mozart_import/core/import_engine.py
# -*- coding: utf-8 -*-
from typing import List, Dict, Any, Optional, Callable
from datetime import datetime
from database import DatabaseService
from :metadata_creator import MetadataCreator
from :connectors.base_connector import BaseConnector

class ImportEngine:
    def __init__(self, db: DatabaseService, connector: BaseConnector, profile: ImportProfile):
        self.db = db
        self.connector = connector
        self.profile = profile
        self.metadata_creator = MetadataCreator(db)

```

```

self.stats = {'inserted': 0, 'updated': 0, 'errors': 0, 'skipped': 0}
self._source_id_field = 'sourceid'
# ----- Работа с ext-таблицами (используем cname, а не alias) -----
def ensure_ext_table(self, entity_id: int, entity_cname: str) -> Optional[str]:
    """Создает ext-таблицу, если ее нет, и возвращает полное имя."""
    ext_table = self.get_ext_table(entity_id, entity_cname)
    if not ext_table:
        return self.create_ext_table(entity_id, entity_cname)
    return ext_table

def create_ext_table(self, entity_id: int, entity_cname: str) -> Optional[str]:
    """Создаю отсутствующую таблицу: {ext_table}"""
    sql = f'''CREATE TABLE {ext_table} (
        versionid INTEGER NOT NULL REFERENCES data.entity_versions(id) ON DELETE CASCADE
    )'''
    self.db.execute_query(sql, fetch=False)
    self.db.execute_query(f'ALTER TABLE {ext_table} ADD COLUMN {self._source_id_field} varchar(100)',
        fetch=False)
    # Синхронизируем все поля сущности
    for field_id in fields:
        try:
            self.db.execute_query("SELECT id FROM meta.fields WHERE entitytypeid = %s", (entity_id,))
            self.db.execute_procedure("meta.p_field_sync", (field_id,))
            ext_table = self.get_ext_table(entity_id, entity_cname)
            return ext_table
        except:
            # Проблема синхронизации поля {field_id}: {e}"""
            return None

def get_ext_table(self, entity_id: int) -> Optional[str]:
    """Возвращает имя ext-таблицы, используя cname сущности."""
    if not self.db.execute_query("SELECT cname FROM meta.entitytypes WHERE id=%s", (entity_id,)) or not res[0][0]:
        return None
    return self.ensure_ext_table(entity_id, cname)

def ensure_sourceid_field(self, entity_id: int):
    """Создает поле sourceid в meta.fields, если его нет."""
    exists = self.db.execute_query("SELECT 1 FROM meta.fields WHERE entitytypeid=%s AND cfieldname=%s",
        (entity_id, self._source_id_field))
    if not exists:
        res = self.db.execute_query(
            f'INSERT INTO meta.fields (entitytypeid, cfieldname, cfieldtype, alias, nlength, lisindexed, status_id)
            VALUES (%s, %s, 255, true, 1) RETURNING id',
            (entity_id, self._source_id_field))
        if res:
            self.db.execute_procedure("meta.p_field_sync", (res[0][0],))

# ----- Основные операции с инстансами и версиями -----
def create_instance(self, entity_id: int) -> Optional[int]:
    """Создает инстанс сущности."""
    res = self.db.execute_query(
        f'INSERT INTO data.entity_instances (entitytypeid, tdt_birth) VALUES (%s, now()) RETURNING id',
        (entity_id,))
    return res[0][0]

def create_version(self, instance_id: int, entity_id: int, values: Dict[str, Any],
    cname: str, fields_map: Dict[str, str]) -> Optional[int]:
    """Создает версию инстанса сущности."""
    res = self.db.execute_query(
        f'INSERT INTO data.entity_versions (instanceid, cname, cversionstatus, tdt_start)
        VALUES (%s, %s, 'Active', now()) RETURNING id',
        (instance_id, cname))
    version_id = res[0][0]
    ext_table = self.get_ext_table(entity_id)
    if not ext_table:
        raise RuntimeError(f"Не удалось получить/создать ext-таблицу для entity_id={entity_id}")
    columns = ['versionid']
    placeholders = ['versionid']
    params = [version_id]
    if source_id is not None:
        columns.append(self._source_id_field)
        params.append(source_id)
    for mozart_field in fields_map.items():
        if mozart_field == self._source_id_field:
            continue
        columns.append(mozart_field)
        placeholders.append('%s')
        params.append(values.get(source_field))
    sql = f'INSERT INTO {ext_table} ({', '.join(columns)}) VALUES ({', '.join(placeholders)})'
    self.db.execute_query(sql, params, fetch=False)

def update_version(self, instance_id: int, entity_id: int, values: Dict[str, Any], fields_map: Dict[str, str]):
    """Обновляет версию инстанса сущности."""
    self.db.execute_query(
        f'UPDATE data.entity_versions SET tdt_end = now(), cversionstatus = 'History' WHERE instanceid = %s AND
        cversionstatus = 'Active',
        (instance_id,))
    self.create_version(instance_id, entity_id, values, fields_map)

def find_instance_by_sourceid(self, entity_id: int, source_id: str) -> Optional[int]:
    """Находит инстанс по source_id."""
    ext_table = self.get_ext_table(entity_id)
    if not ext_table:
        return None
    res = self.db.execute_query(
        f'SELECT v.instanceid FROM data.entity_versions v
        JOIN {ext_table} e ON v.versionid = e.versionid
        WHERE e.{self._source_id_field} = %s AND v.cversionstatus = 'Active'',
        (source_id,))
    return res[0][0] if res else None

def get_or_create_instance(self, entity_id: int, source_id: Optional[str],
    values: Dict[str, Any], fields_map: Dict[str, str]) -> Optional[int]:
    """Получает или создает инстанс сущности по source_id."""
    if source_id:
        instance_id = self.find_instance_by_sourceid(entity_id, source_id)
        if instance_id:
            self.update_version(instance_id, entity_id, values, fields_map)
            return instance_id
    instance_id = self.create_instance(entity_id)
    self.create_version(instance_id, entity_id, values, fields_map, source_id)
    return instance_id

# ----- Публичный метод для оркестратора -----
def import_single_entity(self, dict: Dict[str, Dict[str, int]]) -> (Dict[str, int], Dict[str, int]):
    """Импортирует одну сущность."""
    entity_id = dict['entity_id']
    if not entity_id:
        return {'inserted': 0, 'updated': 0, 'errors': 0, 'skipped': 0}, {}
    self.ensure_sourceid_field(entity_id)
    fields_map = {}
    ref_resolvers = {}
    for field in dict['fields']:
        if field['type'] == 'source':
            fields_map[field['name']] = field['source_id']
        elif field['type'] == 'ref':
            rule = dict['rules'].get(field['name'])
            if rule:
                ref_resolvers.append((field['name'], rule['ref_entity_id'], rule['ref_key_field']))
    source_id_field = dict['source_id_field']
    stats = {'inserted': 0, 'updated': 0, 'errors': 0, 'skipped': 0, 'details': []}
    print(f"Импортирую сущность {entity_id} (source_id_field={source_id_field})")
    for batch_num, batch in enumerate(self.connector.iter_records(dict['records'], batch_size=100)):
        for record in batch:
            # Добавляю ссылку
            key = record.get(ref_key_field)
            if key:
                ref_resolvers.append((key, dict['ref_entity_id'], dict['ref_key_field']))
    self.db.execute_query(
        f'INSERT INTO data.entity_instances (entitytypeid, tdt_birth) VALUES (%s, now()) RETURNING id',
        (entity_id,))
    instance_id = res[0][0]
    self.create_version(instance_id, entity_id, dict['values'], dict['fields_map'], dict['source_id'])
    return instance_id, stats

```

```

        if key:
            record[fields_map[tgt_field]] = instance_cache.get(ref_entity, {}).get(str(key))
        else:
            record[fields_map[tgt_field]] = None
        source_id = None
        if source_id_field:
            raw = record.get(source_id_field)
            if raw:
                source_id = str(raw)
            if source_id.lower() == 'none':
                source_id = None
        try:
            instance_id = self.get_or_create_instance(entity_id, source_id, record, fields_map)
            if source_id:
                instance_cache[source_id] = instance_id
        except Exception as e:
            stats['errors'] += 1
            print(f"Ошибка {entity.source_table}, source_id={source_id}: {e}")
            stats['details'].append(err)
        stats[details].append(err)
print(f"Батч {batch_num+1}: вставлено {stats['inserted']}, обновлено {stats['updated']}, ошибок {stats['errors']}")
print(f"=== Итого: вставлено {stats['inserted']}, обновлено {stats['updated']}, ошибок {stats['errors']}")
return stats, mapping, cache
def run(self, progress_callback=Optional[Callable[[int, int], None]] = None):
    import_single_entity()
    raise NotImplementedError("Используйте ImportOrchestrator.run() или import_single_entity()")

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/import_orchestrator.py (открыть)

```

# -*- coding: utf-8 -*-
from collections import deque
from typing import Dict, List, Optional, Any
from database import DatabaseService
from models import ImportProfile, MatchedEntity, MatchedField
from . import engine import ImportEngine

class ImportOrchestrator:
    """Управляет порядком импорта таблиц источника на основе их зависимостей.
    Сначала загрузятся таблицы без ссылок на другие таблицы (корневые),
    затем зависимые."""
    def __init__(self, db: DatabaseService, connector, profile: ImportProfile):
        self.db = db
        self.connector = connector
        self.engine = ImportEngine(db, connector, profile)
        # {} - словарь: таблица -> Dict[str, Dict[str, int]] == {}
        self.instance_cache = Dict[str, Dict[str, int]] == {}
        self.errors: List[str] = []
    def build_dependency_graph(self) -> Dict[str, List[str]]:
        """Строит граф зависимостей. Ключ - имя таблицы, которая зависит от списка целевых таблиц.
        Возвращает словарь {таблица: [таблицы от которых зависит]}
        # Собираем все таблицы, участвующие в импорте
        for link in self.profile.import_links:
            tables.add(link.source_table)
        graph = {}
        for t in tables:
            graph[t] = []
        # Используем source.references, если они есть
        for ref in self.profile.references:
            src_table = ref.get('source_table')
            dst_table = ref.get('target_table')
            if src_table and dst_table and src_table in graph and dst_table in graph:
                # dst_table -> src_table
                graph[dst_table].append(src_table)
        return graph
    def order_tables(self, graph: Dict[str, List[str]]) -> List[str]:
        """Возвращает топологически отсортированный список таблиц, начиная с корневых.
        in_degree = {t: 0 for t in graph}
        for dep in graph.items():
            for src in dep:
                in_degree[src] += 1
        # src зависит от deg -> увеличиваем степень src
        # Корневые - те, у кого нет зависимостей (in_degree == 0)
        order = []
        while queue:
            current = queue.popleft()
            # Уменьшаем степень у таблиц, которые зависят от current
            for src in graph[current]:
                if current in src:
                    in_degree[src] -= 1
                    if in_degree[src] == 0:
                        queue.append(src)
        if len(order) != len(graph):
            remaining = set(graph.keys()) - set(order)
            raise ValueError("Циклическая зависимость между таблицами: {}".format(' '.join(remaining)))
        return order
    def run(self, progress_callback=None) -> Dict[str, int]:
        """Выполняет импорт всех таблиц в правильном порядке.
        Возвращает статистику (aggregated stats).
        # 1. Построить порядок
        graph = self.build_dependency_graph()
        ordered_tables = self.order_tables(graph)
        except ValueError as e:
            return {'inserted': 0, 'updated': 0, 'errors': len(self.errors), 'skipped': 0}
        # 2. Для каждой таблицы в порядке очереди
        for link in self.profile.import_links:
            table = link.source_table
            total_stats = {'inserted': 0, 'updated': 0, 'errors': 0, 'skipped': 0}
            for table_name in ordered_tables:
                if table_name == table:
                    # Таблица не связана с Mozart - пропускаем
                    continue
                # 3. Создаём частичный профиль для одной таблицы
                entity_id = link.target_entity_id
                # Безопасно получаем имя сущности
                target_entity_name = self.db.get_entity_name(entity_id)
                # Получаем данные из БД
                res = self.db.execute_query("SELECT cname FROM meta.entitytypes WHERE id=%s", (entity_id,))
                if target_entity_name == res[0][0]:
                    target_entity_name = f"Entity_{entity_id}"
                else:
                    target_entity_name = f"Entity_{entity_id}"
                matched_fields = []
                source_id_field = None
                ref_rules = [] # локальные правила разрешения ссылок для этой таблицы
                # Мappings из профиля
                for m in link.get('mappings', []):
                    m['source_field'] = m['source_field'],

```

```

        target_field=mf['target_field']
        is_reference=m.get('is-reference', False),
        matched_fields.append(mf)
    if source_id_field == mf['source_field']:
        # Если поле ссылочное, находим правило для разрешения
        # Упавшим правило в source_references, ""
        ref_src_table=m.get('ref-source-table', "")
        if not ref_key_field or not m.get('ref-source-key-field', "")
        target_entity_id=None
        for link in self.possible_import_links:
            if link['source-table']==ref_src_table:
                target_entity_id=link['target-entity-id']
            if target_entity:
                ref_target.append({
                    'target-entity-id': target_entity_id,
                    'ref-key-field': ref_src_key_field
                })
    entity = MatchedEntity(
        source_table=table_name,
        target_entity_id=entity_id,
        confidence=1,
        fields=matched_fields,
        source_id_field=source_id_field
    )
# 4. Вызвать импорт одной сущности
try:
    stats, sourceid, mapping = self.engine.import_single_entity(
        entity, ref_rules, self.instance_cache
    )
    total_stats['inserted'] += stats['inserted']
    total_stats['errors'] += stats['errors']
    total_stats['skipped'] += stats.get('skipped', 0)
    # 5. Сохранить sourceid -> instanceid в кэш для этой таблицы
    cache_key = table_name
    self.instance_cache[cache_key] = sourceid, mapping
except Exception as e:
    total_stats['errors'] += 1
    total_stats['errors'] += "Ошибка импорта {table_name}: {str(e)}"
return total_stats

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/mapping_manager.py (открыть)

-*- coding: utf-8 -*-

Файл: /home/sergey/Documents/configurate/mozart_import/core/metadata_creator.py (открыть)

```

# mozart_import/core/metadata_creator.py
from database import DatabaseService
from models import MatchedEntity, MatchedField, ImportProfile
from composition_manager import CompositionManager

class MetadataCreator:
    def __init__(self, db: DatabaseService):
        self.comp_manager = CompositionManager(db=db)
        self.dict_class_id = self._get_class_id('DICT')

    def _get_class_id(self, class_alias: str) -> Optional[int]:
        res = self.db.execute_query("SELECT id FROM meta.entity_classes WHERE calias = %s", (class_alias,))
        return res[0][0] if res else None

    def _create_entity_type(self, table_name: str, is_document: bool = False) -> int:
        cname = table_name.capitalize()
        class_id = self._get_class_id('DICT') if is_document else self.dict_class_id
        INSERT INTO meta.entity_types (cname, calias, class_id, cbasclass, lisdependent, lishierarchy, lisversioned,
        security, status) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)
        RETURNING id
        res = self.db.execute_query(sql, (
            cname, is_document, class_id,
            False, False, True, 1, 1
        ))
        return res[0][0]

    def _create_field(self, entity_id: int, field_name: str, field_type: str, calias: str, ref_entity_id: Optional[int] = None) -> int:
        INSERT INTO meta.fields (entitytypeid, cfieldname, cfieldtype, calias, lisindexed, ref_entitytypeid, status_id)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
        RETURNING id
        res = self.db.execute_query(sql, (entity_id, field_name, field_type, calias, True, ref_entity_id, 1))
        field_id = res[0][0]
        self.db.execute_procedure("meta.p_field_sync", (field_id,))
        return field_id

    def ensure_entity_and_fields(self, matched_entity: MatchedEntity) -> int:
        if matched_entity.target_entity_id is not None and not matched_entity.is_new:
            entity_id = matched_entity.target_entity_id
        else:
            is_doc = matched_entity.is_document
            entity_id = self._create_entity_type(matched_entity.source_table, is_doc)
            matched_entity.is_new = True
        for field in matched_entity.fields:
            if not field.is_new:
                source_field_info = field.target_field == ":":
                for src_field in getattr(matched_entity, 'source_table_fields', []):
                    if src_field.name == field.source_field:
                        break
                if source_field_info is None:
                    if field.is_reference:
                        ref_entity_id = field.ref_entitytype_id
                        if source_field_info.field_type.name == 'INTEGER':
                            ref_entity_id = None
                        elif source_field_info.field_type.name == 'FLOAT':
                            ref_entity_id = None
                        else:
                            ref_entity_id = None
                        mozart_type = 'C'
                    else:
                        mozart_type = 'I'
                    new_calias = field.source_field.lower()
                    self._create_field(entity_id, new_calias, mozart_type, new_calias, ref_entity_id)
                    field.confidence = 1.0
            return entity_id

    def create_composition_if_needed(self, parent_entity_id: int, child_entity_id: int, link_field: str, role_id: int = None):
        if role_id is None:
            roles = self.db.execute_query("SELECT id FROM meta.composition_roles WHERE calias = 'DEFAULT'")
            if roles:
                role_id = roles[0][0]
            else:
                roles = self.db.execute_query(
                    "INSERT INTO meta.composition_roles (calias, cname) VALUES ('DEFAULT', 'Основная') RETURNING id"
                )
                role_id = res[0][0]
            exists = self.db.execute_query(

```

```

"SELECT id FROM meta.entity_composition WHERE parent_entityid=%s AND child_entityid=%s AND
role_id=%s
    (parent_entity_id, child_entity_id, role_id)
    } exists;
    return True
return self.comp_manager.add_composition(parent_entity_id, child_entity_id, role_id, link_field, 10)

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/models.py (относительно

```

# coding: utf-8
from dataclasses import dataclass, field
from enum import Enum

class SourceType(Enum):
    DATABASE = "database"
    URL = "url"

class FileFormat(Enum):
    JSON = "json"
    DBF = "dbf"
    CSV = "csv"
    EXCEL = "excel"

class FieldType(Enum):
    INTEGER = "integer"
    FLOAT = "float"
    BOOLEAN = "boolean"
    DATETIME = "datetime"
    BINARY = "binary"

# Вспомогательные классы для визуализации графа
class NodeType(Enum):
    MOZART_ENTITY = "mozart_entity"

class DataType(Enum):
    INTEGER = "integer"
    FLOAT = "float"
    BOOLEAN = "boolean"
    DATETIME = "datetime"

class LinkType(Enum):
    IMPORT_IMPORT = "source"
    COMPOSITION = "composition"

@dataclass
class FieldStr:
    data_type: DataType = DataType.STRING
    is_primary_key: bool = False
    references: Optional[str] = None

@dataclass
class Node:
    name: str
    fields: List[Field] = field(default_factory=list)
    x: float = 0
    y: float = 0
    table_name: Optional[str] = None # для таблиц источника
    class_alias: Optional[str] = None

@dataclass
class SourceField:
    name: str
    field_type: FieldType
    nullable: bool = True
    is_primary_key: bool = False
    reference: Optional[str] = None
    sample_values: List[Any] = field(default_factory=list)

@dataclass
class SourceTable:
    name: str
    fields: List[SourceField] = field(default_factory=list)
    foreign_keys: List[str] = field(default_factory=list)
    row_count: Optional[int] = None
    sample_rows: List[Dict[str, Any]] = field(default_factory=list)

@dataclass
class Schema:
    source_type: SourceType
    tables: List[SourceTable] = field(default_factory=list)

@dataclass
class MatchedField:
    source_field: str
    confidence: float = 1.0
    transformation: Optional[str] = None
    reference_id: Optional[int] = None
    field_type: str = "C55"
    is_source_id: bool = False

@dataclass
class MatchedEntity:
    source_name: str
    target_entity_id: Optional[int]
    target_entity_name: str
    confidence: MatchedField = field(default_factory=list)
    is_new: bool = False
    is_document: bool = False
    source_id_field: Optional[str] = None

@dataclass
class MappingProposal:
    source_schema: Schema
    entities: List[MatchedEntity] = field(default_factory=list)
    unresolved_tables: List[str] = field(default_factory=list)
    suggestions_for_new_entities: List[str] = field(default_factory=list)

@dataclass
class Profile:
    class Optional[int] = None
    name: str
    source_type: SourceType = SourceType.FILE
    format: Optional[FileFormat] = None
    connection_params: Dict[str, str] = field(default_factory=dict)
    mapping_proposal: Optional[MappingProposal] = None
    scheduled: bool = False

```



```

schedule_cron: Optional[str] = None
last_status: Optional[str] = None
scene_json: Optional[str] = None
# Нормализованные поля
source_tables: List[str] = field(default_factory=list)
target_entities: List[str] = field(default_factory=list)
source_fields: List[str] = field(default_factory=list)
# Новые реляции настраиваются вручную
source_references: List[Dict] = field(default_factory=list) # [{ 'source_table': str, 'source_field': str, 'ref_table': str, 'ref_field': str}]
# Параметры подключения
db_host: Optional[str] = None
db_port: Optional[int] = None
db_name: Optional[str] = None
db_pass: Optional[str] = None

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/mozart_matcher.py (относительно папки configure)

```

# mozart_import/core/mozart_matcher.py
from typing import Sequence, Mapping, Any, Tuple
from models import SourceSchema, SourceTable, SourceField, FieldType
from database import DatabaseService, MatchedField, MappingProposal

class MozartMatcher:
    """Сопоставляет исходную схему (SourceSchema) с метаданными Mozart.
    Возвращает MappingProposal с предложениями по сущностям и полям."""
    def __init__(self, db: DatabaseService):
        self.db = db
        # Загружаем метаданные Mozart при инициализации
        self.load_metadata() # {id: {name, alias, class_id, class_alias, fields}}
    def load_metadata(self):
        """Загружаем метаданные из БД"""
        rows = self.db.execute_query("""
            SELECT e.class_id, e.class_alias, e.class_id as class_id, c.class_id as class_alias
            FROM meta_entitytypes e
            ORDER BY e.class_id
        """)
        for row in rows:
            self.entitytypes.setdefault(row.class_id, {
                'name': row.class_alias,
                'alias': row.class_id,
                'class_id': row.class_id,
                'fields': {}
            })
        # Загружаем поля
        rows = self.db.execute_query("""
            SELECT ft.entitytypeid, ft.fieldname, ft.fieldtype,
            FROM meta_fields ft
            ORDER BY ft.entitytypeid, ft.isortorder
        """)
        for row in rows:
            eid, fieldname, alias, fieldtype, ref_eid, type_name = row
            if eid in self.entitytypes['fields']:
                self.entitytypes['fields'][fieldname] = {
                    'fieldtype': fieldtype,
                    'ref_entitytypeid': ref_eid,
                    'type_name': type_name
                }
    def similarity(self, a: str, b: str) -> float:
        return SequenceMatcher(None, a.lower(), b.lower()).ratio()
    def match_table_to_entity(self, table: SourceTable) -> Tuple[Optional[int], str, float]:
        """Пытается найти подходящую сущность в Mozart по имени таблицы.
        Возвращает (entity_id, entity_name, confidence)"""
        best_id = None
        best_name = None
        best_score = 0
        for eid, info in self.entitytypes.items():
            score = max(self.similarity(table.name, info['name']),
                        self.similarity(table.alias, info['alias']))
            if score > best_score:
                best_score = score
                best_id = eid
                best_name = info['name']
        return best_id, best_name, best_score
    def match_field_to_field(self, source_field: SourceField, target_fields: Dict) -> Tuple[Optional[str], float]:
        """Сопоставляет поле источника с полем сущности Mozart по имени и типу.
        Возвращает (target_field_name, confidence)"""
        best_field = None
        best_score = 0
        for name, info in target_fields.items():
            name_score = self.similarity(source_field.name, info['alias'])
            type_score = 0
            # Проверяем соответствие типов
            if info['fieldtype'] == 'C' and source_field.field_type == FieldType.STRING:
                type_score = 1
            elif info['fieldtype'] == 'I' and source_field.field_type == FieldType.INTEGER:
                type_score = 1
            elif info['fieldtype'] == 'N' and source_field.field_type == FieldType.FLOAT:
                type_score = 1
            elif info['fieldtype'] == 'D' and source_field.field_type == FieldType.DATE:
                type_score = 1
            elif info['fieldtype'] == 'L' and source_field.field_type == FieldType.BOOLEAN:
                type_score = 1
            else:
                type_score = 0.5
            total_score = name_score * 0.7 + type_score * 0.3
            if total_score > best_score:
                best_score = total_score
                best_field = name
        return best_field, best_score
    def propose(self, source_schema: SourceSchema) -> MappingProposal:
        """Собирает MappingProposal для всей исходной схемы.
        entities = []
        unresolved = []
        suggestions = []
        for table in source_schema.tables:
            entity_id, entity_name, conf = self.match_table_to_entity(table)
            if entity_id is not None:
                target_fields = self.entitytypes[entity_id]['fields']
                matched_fields = []
                for src_field in table.fields:
                    target_field, conf = self.match_field_to_field(src_field, target_fields)
                    if target_field is not None:
                        is_ref = target_fields[target_field]['fieldtype'] == 'R'
                        ref_entitytypeid = target_fields[target_field]['ref_entitytypeid']
                        matched_fields.append({
                            'source_field': src_field.name,
                            'target_field': target_field,
                            'confidence': conf,
                            'is_reference': is_ref
                        })

```

```

    )) ref_entitytype_id=ref_entity
    else:
        # Поле не найдено, предлагаем добавить
        matched_fields=src_field_name
        target_field=0 # будет новое
        confidence=0
        is_reference=False
    entities.append(MatchedEntity(
        source_table=table_name,
        target_entity_name=entity_name,
        confidence=conf,
        is_new=matched_fields,
    ))
    else:
        # Новая сущность
        unresolved.append(table.name)
    suggestions.append(table.name)
return MappingProposal(
    source_schema,
    entities=entities,
    unresolved_tables=unresolved,
    suggestions_for_new_entities=suggestions
)

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/profile_manager.py (относительно файла configure.py)

```

# mozart_import/core/profile_manager.py
# coding: utf-8
import json
from typing import Optional, List
from database import DatabaseService
from models import ImportProfile, SourceType

class ProfileManager:
    def __init__(self, db: DatabaseService):
        self.db = db

    def save_profile(self, profile: ImportProfile) -> int:
        source_type_code = profile.source_type.value if profile.source_type else None
        # 1. Сохраняем или обновляем основную профиль
        self.db.execute_query("""
            INSERT INTO meta.import_profiles
            (profile_name, source_type_code, source_path, file_format,
              db_name, db_port, db_user, db_pass,
              updated_at)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)
            ON DUPLICATE KEY UPDATE
            profile_name=profile_name, source_type_code=source_type_code, source_path=source_path,
            file_format=file_format, db_name=db_name, db_port=db_port, db_user=db_user,
            db_pass=db_pass, updated_at=NOW()
        """, (profile.name, source_type_code, profile.source_path,
              profile.file_format, profile.db_name, profile.db_port,
              profile.db_user, profile.db_pass, profile.updated_at))
        # Удаляем ссылки на источники, которые не являются новыми
        self.db.execute_query("DELETE FROM meta.import_source_links WHERE profile_id=%s", (profile.id,),
                              fetch=False)
        self.db.execute_query("DELETE FROM meta.import_mappings WHERE link_id IN (SELECT id FROM
                              meta.import_links WHERE profile_id=%s)", (profile.id,), fetch=False)
        self.db.execute_query("DELETE FROM meta.import_target_entities WHERE profile_id=%s", (profile.id,),
                              fetch=False)
        self.db.execute_query("DELETE FROM meta.import_source_references WHERE source_table_id IN (SELECT id
                              FROM meta.import_source_tables WHERE profile_id=%s)", (profile.id,),
                              fetch=False)
        self.db.execute_query("DELETE FROM meta.import_source_tables WHERE profile_id=%s", (profile.id,),
                              fetch=False)
        # 2. Сохраняем таблицы источника
        for table in profile.source_tables:
            self.db.execute_query("""
                INSERT INTO meta.import_source_tables (profile_id, table_name, row_count, sample_data)
                VALUES (%s, %s, %s, %s)
                ON DUPLICATE KEY UPDATE
                row_count=row_count, sample_data=sample_data
            """, (profile.id, table['name'], table.get('row_count'), json.dumps(table.get('sample_rows', []))))
        # 3. Сохраняем целевые сущности
        for entity in profile.target_entities:
            self.db.execute_query("""
                INSERT INTO meta.import_target_entities (profile_id, entity_id, entity_name)
                VALUES (%s, %s, %s)
                ON DUPLICATE KEY UPDATE
                entity_name=entity_name
            """, (profile.id, entity['id'], entity['name']))
        # 4. Сохраняем связи и маппинги (с обработкой дубликатов)
        for link in profile.links:
            src_id = source_table_ids.get(link['source_table'])
            tgt_id = target_entity_ids.get(link['target_entity'])
            self.db.execute_query("""
                INSERT INTO meta.import_links (profile_id, source_table_id, target_entity_id)
                VALUES (%s, %s, %s)
                ON DUPLICATE KEY UPDATE
                source_table_id=source_table_id, target_entity_id=target_entity_id
            """, (profile.id, src_id, tgt_id))
        # Сохраняем маппинги для этого link_id (на случай обновления)
        self.db.execute_query("""
            DELETE FROM meta.import_mappings WHERE link_id=%s", (link_id,), fetch=False)
        for mapping in link.mappings:
            self.db.execute_query("""
                INSERT INTO meta.import_mappings
                (link_id, source_table_id, source_field, target_entity_id, target_field,
                 is_source_key, is_target_key, is_excluded)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
                ON DUPLICATE KEY UPDATE
                source_table_id=source_table_id, source_field=source_field, target_entity_id=target_entity_id,
                target_field=target_field, is_source_key=is_source_key, is_target_key=is_target_key,
                is_excluded=is_excluded
            """, (link_id, src_id, mapping.get('source_field'), tgt_id, mapping.get('target_field'),
                  mapping.get('is_source_key'), mapping.get('is_target_key'), mapping.get('is_excluded')))
        # 5. Сохраняем первичные связи источника (FK)
        for fk in profile.source_references:
            src_id = source_table_ids.get(fk['source_table'])
            ref_id = source_table_ids.get(fk['target_table'])
            self.db.execute_query("""
                INSERT INTO meta.import_source_references (profile_id, source_table_id, source_field, ref_table_id, ref_field)
                VALUES (%s, %s, %s, %s, %s)
                ON DUPLICATE KEY UPDATE
                source_table_id=source_table_id, source_field=source_field, ref_table_id=ref_table_id,
                ref_field=ref_field
            """, (profile.id, src_id, fk['source_field'], ref_id, fk['target_field']), fetch=False)
        # 6. Сохраняем перекрестные ссылки источника (source_references)
        for ref in profile.source_references:
            src_id = source_table_ids.get(ref['source_table'])
            ref_id = source_table_ids.get(ref['target_table'])
            self.db.execute_query("""
                INSERT INTO meta.import_source_references (source_table_id, source_field, ref_table_id, ref_key_field)
                VALUES (%s, %s, %s, %s)
            """, (src_id, ref['source_field'], ref_id, ref['key_field']))

```

```

        ON CONFLICT (source_table_id, source_field)
        DO UPDATE SET ref_table_id = EXCLUDED.ref_table_id, ref_key_field = EXCLUDED.ref_key_field
    , (src_id, ref[source_field], dst_id, ref[ref_field]), fetch=False)
    return profile.id

def load_profile(self, profile_id: int) -> Optional[ImportProfile]:
    row = self.db.execute_query("""
    SELECT id, format, db_name, db_host, db_port, db_name, db_user, db_pass
    FROM meta.import_profiles
    WHERE profile_id = %s
    """, (profile_id,))
    if not row:
        return None
    row = row[0]
    profile = ImportProfile(
        name=row[1],
        source_path=SourceTypes(row[2]),
        format=row[3],
        db_host=row[4],
        db_port=row[5],
        db_name=row[6],
        db_user=row[7],
        db_pass=row[8]
    )
    source_tables = self.db.execute_query("""
    SELECT meta.table_name, source_table_id, sample_data
    FROM meta.import_profiles
    WHERE profile_id = %s
    """, (profile_id,))
    profile.source_tables = []
    for st in source_tables:
        tbl = st[0]
        name = st[1]
        row_count = st[2]
        sample_rows = st[3] if st[3] else []
        profile.source_tables.append(tbl)
        source_table = map(st[1]) == st[0]
        target_entities = self.db.execute_query("""
        SELECT meta.import_target_entities
        FROM meta.import_profiles
        WHERE profile_id = %s
        """, (profile_id,))
        profile.target_entities = []
        for te in target_entities:
            profile.target_entities.append(te)
        target_entity_mapper = {}
        links = self.db.execute_query("""
        SELECT meta.import_source_tables st ON f.source_table_id = st.id
        WHERE profile_id = %s
        """, (profile_id,))
        profile.links = []
        for mapping in links:
            self.db.execute_query("""
            SELECT source_field, target_field, field_type, field_length, is_source_id,
            FROM reference, ref_src_table, ref_src_key_field
            WHERE profile_id = %s
            """, (profile_id,))
            profile.links.append({
                'source_table': mapping[0], 'target_table': mapping[1],
                'source_field': mapping[2], 'target_field': mapping[3],
                'field_type': mapping[4], 'field_length': mapping[5],
                'is_source_id': mapping[6], 'is_source_ref': mapping[7]
            })
        source_fks = self.db.execute_query("""
        SELECT st1.table_name, is_source_field, st2.table_name, is.ref_field
        FROM meta.import_source_tables st1 ON is.source_table_id = st1.id
        JOIN meta.import_source_tables st2 ON is.ref_table_id = st2.id
        WHERE profile_id = %s
        """, (profile_id,))
        profile.source_fks = [(f[0], f[1], f[2], f[3]) for f in source_fks]
        source_refs = self.db.execute_query("""
        SELECT meta.import_source_references sr
        FROM meta.import_source_tables st1 ON sr.source_table_id = st1.id
        JOIN meta.import_source_tables st2 ON sr.ref_table_id = st2.id
        WHERE profile_id = %s
        """, (profile_id,))
        profile.source_references = [(r[0], r[1], r[2], r[3]) for r in source_refs]
    return profile

def list_profiles(self) -> List[ImportProfile]:
    row = self.db.execute_query("""
    SELECT id, format, db_name, db_host, db_port, db_name, db_user, db_pass
    FROM meta.import_profiles
    ORDER BY created_at DESC
    """)
    profiles = []
    for row in rows:
        profile = ImportProfile(
            name=row[1],
            source_type=SourceTypes(row[2]),
            source_path=row[3]
        )
    return profiles

def delete_profile(self, profile_id: int):
    self.db.execute_query("DELETE FROM meta.import_profiles WHERE id=%s", (profile_id,))

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/schema_analyzer.py (относительно папки /home/sergey/Documents/configurate/mozart_import/core/)

```

# mozart_import/core/schema_analyzer.py
from typing import Dict, List, Optional, Any
from .models import SourceSchema, SourceTable, SourceField, FieldType

class SchemaAnalyzer:
    """Анализирует исходную схему, добавляет метаданные:
    - определяет все первичные ключи (по имени или уникальности)
    - определяет все вторичные ключи (по уникальности)
    - кандидаты на иерархические связи (parent_id и т.д.)"""

    def __init__(self, schema: SourceSchema):
        self.schema = schema
        self.analyze()

    def analyze(self):
        for table in self.schema.tables:
            self.analyze_table(table)

    def analyze_table(self, table: SourceTable):
        # 1. Определяем вероятный первичный ключ
        pk_candidates = []
        for field in table.fields:
            if field.is_nullable:
                continue
            pk_candidates.append(field.name)
            elif field.name.lower() in ('id', 'code'):
                pk_candidates.append(field.name)

```

```

if pk_candidates:
    table.primary_key = pk_candidates
else:
    # если нет очевидного, предлагаем первый подходящий по типу целочисленный
    for i, field in enumerate(table.fields):
        if field.type in (Field.Type.INTEGER):
            table.primary_key = [int(fields[0])]
# 2. Определяем вероятные внешние ключи (по именам)
for field in table.fields:
    name_lower = field.name.lower()
    # предполагаем, что ссылка на таблицу без последнего суффикса
    target_table = name_lower.replace('_id', '').replace('id', '')
    if target_table == table.name: # ссылка на саму себя (иерархия)
        fk_candidates.append((field.name, target_table))
    table.foreign_keys = fk_candidates
# 3. Собираем статистику по полям (если есть sample_rows)
for field in table.fields:
    values = []
    for row in table.sample_rows:
        if row.get(field.name) is not None:
            values.append(row.get(field.name))
    # уникальность можно оценить на основе sample
    if len(values) > 0:
        unique_values = set(values)
        if len(unique_values) == len(values):
            # если в sample все значения уникальны, предполагаем, что поле уникально
            # предполагаем, что поле уникальный кандидат на ключ
            table.primary_key.append(field.name)

def get_primary_key_candidates(self, table_name: str) -> List[str]:
    for table in self.schema.tables:
        if table.name == table_name and table.primary_key:
            return table.primary_key
    return []

def get_foreign_key_candidates(self, table_name: str) -> Dict[str, str]:
    for table in self.schema.tables:
        if table.name == table_name:
            return {}
    return {}

def suggest_hierarchy_field(self, table_name: str) -> Optional[str]:
    """Возвращаем имя поля, вероятно, указывающего на родителя (для иерархий)."""
    for table in self.schema.tables:
        if table.name == table_name:
            for field in table.fields:
                if field.name.lower() in ('parentid', 'parent_id', 'pid', 'parent'):
                    return field.name
    return None

def suggest_reference(self, field_name: str) -> Optional[str]:
    """По имени поля предполагаем целевую сущность для внешней ссылки."""
    name_lower = field_name.lower()
    if name_lower.endswith('_id'):
        target = name_lower[:-3]
    elif name_lower.startswith('id_'):
        target = name_lower[3:]
    else:
        return None
    # ищем в схеме таблицу с таким именем
    for table in self.schema.tables:
        if table.name == target:
            return target
    # не найдено, но возвращаем предположение

```

Файл: /home/sergey/Documents/configurate/mozart_import/core/source_analyzer.py (относительный)

-*- coding: utf-8 -*-

Файл: /home/sergey/Documents/configurate/mozart_import/core/source_connector.py (относительный)

Файл: /home/sergey/Documents/configurate/mozart_import/designer/__init__.py (относительный)

```

# mozart_import/designer/__init__.py
# -*- coding: utf-8 -*-
from .designer_v2 import ImportDesignerV2
from .designer_trees import ImportTreeManager, MozartTreeManager
from .designer_widgets import ImportWidget
from .scene_profile_adapter import SceneProfileAdapter
from .entity_creator import EntityCreator
from .designer_utils import to_field_name

__all__ = [
    'ImportDesignerV2',
    'ImportTreeManager',
    'MozartTreeManager',
    'ImportWidget',
    'SceneProfileAdapter',
    'EntityCreator',
    'MassFieldDialog',
    'to_field_name'
]

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/designer_import.py (относительный)

```

# mozart_import/designer/designer_import.py
# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import QProgressDialog, QMessageBox
from PyQt5.QtCore import Qt
from .core_widgets import ImportProfile, SourceType, MappingProposal, MatchedEntity, MatchedField
from .core_import_engine import ImportEngine
from .designer_mass_field_dialog import MassFieldDialog

class ImportOperations:
    """Методы импорта и создания полей"""
    def __init__(self, parent, db, connector, translator, source_tables, mozart_entities, import_links):
        self.parent = parent
        self.db = db
        self.connector = connector
        self.source_tables = source_tables
        self.mozart_entities = mozart_entities
        self.import_links = import_links
    def set_source_tables(self, tables):
        self.source_tables = tables
    def set_mozart_entities(self, entities):
        self.mozart_entities = entities
    def set_import_links(self, links):
        self.import_links = links
    def create_fields_from_checked(self, self, target_entity_item, source_tree_manager):
        checked_fields = target_entity_item.tree_manager.get_checked_fields()
        if not checked_fields:
            QMessageBox.warning(self.parent, self.translator.tr('warning'), self.translator.tr('no_fields_selected'))
            dlg = MassFieldDialog(self.parent, checked_fields, self.translator)
            if not dlg.exec_():
                return
            fields_to_create = dlg.get_fields()
            if not fields_to_create:
                return

```



```

        return
    creator = FieldCreator(self.db)
    for link in fields_to_create:
        creator.create_fields(entity_id=[63],
                             cfieldtype='link', cfieldname='link',
                             alias='link', nlength=50)
    })

def perform_import(self):
    creator = FieldCreator(self.db)
    for link in self.import_links:
        if link['type'] == 'import' and link['target_field']:
            check = self.db.execute_query(
                (link['target_entity_id'], link['target_field'])
            )
            if not check:
                creator.create_fields(link['target_entity_id'], [{
                    cfieldtype='link', cfieldname='link',
                    alias='link', nlength=50,
                }])
            elif link['type'] == 'sourceid':
                check = self.db.execute_query(
                    (link['target_entity_id'],)
                )
                if not check:
                    creator.create_fields(link['target_entity_id'], [{
                        cfieldtype='sourceid',
                        alias='Source ID',
                        nlength=100
                    }])

# Группировка связей по целевой сущности
for link in self.import_links:
    if link['target_entity_id'] not in groups:
        groups[link['target_entity_id']].append(link)
    entities = []
    for entity in groups.items():
        for ent in self.mozart_entities:
            if ent['id'] == entity[0]:
                ent = entity[1]
                if not target_entity:
                    source_table_name = links[0]['source_table']
                    for link in links:
                        if link['type'] == 'import' and link['target_field']:
                            target_field = link['target_field']
                            entities.append(MatchedEntity(
                                source_table_name,
                                target_entity_id=ent['id'],
                                target_entity_name=target_entity['cname'],
                                fieldid=ent['fields'],
                                is_new=False
                            ))
    mapping_proposal = MappingProposal(
        source_schema=self.connector.get_schema(),
        entities=entities,
        suggestions_for_new_entities=[]
    )
    profile = ImportProfile(
        name=self.connector.source_path,
        source_path=self.connector.source_path,
        mapping_proposal=mapping_proposal
    )
    progress = OProgressDialog(self.translator.tr('import_progress'), self.translator.tr('cancel'), 0, 0, self.parent)
    progress.setWindowModality(Qt.WindowModal)
    try:
        engine = ImportEngine(self.db, self.connector, profile)
        progress.close()
        QMessageBox.information(self.parent, self.translator.tr('import_finished_title'),
                                self.translator.tr('import_finished_status').format(stats['inserted'], stats['updated'], stats['errors']))
    except Exception as e:
        QMessageBox.critical(self.parent, self.translator.tr('error'), str(e))

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/designer_links.py (открыть)

```

# mozart_import/designer/designer_links.py
from PyQt5.QtWidgets import QDialog, QFormLayout, QComboBox, QVBoxLayout, QDialogButtonBox,
from PyQt5.QtCore import Qt

class ImportLinksManager:
    def __init__(self, parent, links_table, translator, source_tables, mozart_entities):
        self.parent = parent
        self.links_table = links_table
        self.translator = translator
        self.source_tables = source_tables
        self.mozart_entities = mozart_entities
        self.import_links = []

    def set_source_tables(self, tables):
        self.source_tables = tables

    def set_mozart_entities(self, entities):
        self.mozart_entities = entities

    def add_import_link(self):
        dlg = QDialog(self.parent)
        dlg.setWindowTitle(self.translator.tr('add_import_link_title'))
        layout = QFormLayout(dlg)
        src_table_combo = QComboBox()
        for t in self.source_tables:
            src_table_combo.addItem(t.name, t.name)
        src_table_combo.currentIndexChanged.connect(lambda: self.update_src_fields(src_table_combo,
                                                                                       src_table_combo.currentText(),
                                                                                       src_table_combo.currentText()))
        target_entity_combo = QComboBox()
        for ent in self.mozart_entities:
            target_entity_combo.addItem(f"{ent['cname']} ({ent['alias']})", ent['id'])
        target_entity_combo.currentIndexChanged.connect(lambda: self.update_target_fields(target_entity_combo,
                                                                                           target_entity_combo.currentText(),
                                                                                           target_entity_combo.currentText()))
        type_combo = QComboBox()
        type_combo.addItem(self.translator.tr('import_type'), 'import')
        type_combo.addItem(self.translator.tr('sourceid_type'), 'sourceid')
        type_combo.currentIndexChanged.connect(lambda: self.update_type(type_combo, type_combo.currentText()))
        btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        btn_box.accepted.connect(lambda: self.accept())
        btn_box.rejected.connect(lambda: self.reject())
        layout.addWidget(btn_box)
        self.update_src_fields(src_table_combo, src_table_combo.currentText(), src_table_combo.currentText())
        self.update_target_fields(target_entity_combo, target_entity_combo.currentText(), target_entity_combo.currentText())
        self.update_type(type_combo, type_combo.currentText())
        self.add_import_link(src_table_combo.currentText(), target_entity_combo.currentText(), type_combo.currentText())

```



```

self.setup_context_menus()
self.links_table.setMinimumHeight(150)
self.links_table.setMaximumHeight(400)
def load_initial_data(self):
self.source_manager.load_source()
self.links_manager = ImportLinksManager(self, self.links_table, self.translator,
self.mozart_manager.mozart_entities)
self.import_ops = ImportOperations(self, self.connector, self.translator,
self.source_manager.source_tables,
self.links_manager.import_links)
def setup_context_menus(self):
self.mozart_tree.customContextMenuRequested.connect(self.source_context_menu)
def source_context_menu(self, pos):
if not item or item.parent() is not None:
return
menu = QMenu()
preview_action = QAction(self.translator.tr('preview table'), None)
menu.addAction(preview_action)
add_link_action = QAction(self.translator.tr('add source link'), None)
menu.addAction(add_link_action)
select_all_action = QAction(self.translator.tr('select all fields'), None)
menu.addAction(select_all_action)
deselect_all_action = QAction(self.translator.tr('deselect all fields'), None)
menu.addAction(deselect_all_action)
menu.exec_(self.source_tree.mapToGlobal(pos))
def mozart_tree_customContextMenuRequested(self, pos):
if not item or item.parent() is not None:
return
menu = QMenu()
create_fields_action = QAction(self.translator.tr('create fields from checked'), None)
self.source_manager.create_fields_from_checked.connect(lambda: self.import_ops.create_fields_from_checked(item,
self.source_manager))
menu.addAction(create_fields_action)
menu.exec_(self.mozart_tree.mapToGlobal(pos))
def start_splitter_drag(self, event):
self.dragging = True
self.start_y = event.globalY()
self.setCursor(Qt.SizeVerCursor)
def drag_splitter(self, event):
delta = event.globalY() - self.drag_start_y
new_height = self.start_height + delta
new_height = max(100, min(400, new_height))
self.links_table.setMaximumHeight(new_height)
def stop_splitter_drag(self, event):
self.setCursor(Qt.ArrowCursor)
def add_import_link(self):
self.import_ops.set_import_links(self.links_manager.import_links)
def save_profile(self):
source_tables = self.source_manager.source_tables
for table in source_tables:
if table.sample_rows else []
source_rows = table.sample_rows
name = table.name
row_count = self.connector.get_row_count(table.name),
sample_rows: sample_rows
# 2. Сбор целевых сущностей
target_entities = []
for link in self.links_manager.import_links:
if link.target_entity_id:
if link.target_entity_id not in self.mozart_manager.mozart_entities:
target_entities.append({'id': eid, 'name': ent['cname']})
break
# 3. Собираем связи импорта
import_links = []
for link in self.links_manager.import_links:
mappings = {}
source_field = link['source_field'],
target_field = link['target_field'],
field_type = link['type']
if field_type == 'source_id':
import_links.append({
'source_table': link['source_table'],
'target_entity_id': link['target_entity_id'],
'mappings': mappings
})
# 4. Сбор параметров подключения
db_type = None
db_host = None
db_port = None
db_name = None
db_user = None
db_pass = None
if hasattr(self.connector, 'file_format'):
file_format = self.connector.file_format.value if isinstance(self.connector.file_format, FileFormat) else
str(self.connector.file_format)
elif hasattr(self.connector, 'db_type'):
db_type = self.connector.db_type
db_host = self.connector.host
db_port = self.connector.port
db_name = self.connector.database
db_user = self.connector.user
db_pass = self.connector.password
# URLConnector не имеет дополнительных полей, кроме source_path
# Определяем тип источника
if hasattr(self.connector, 'file_format'):
source_type = SourceType.DATABASE
else:
source_type = SourceType.URL
profile = ImportProfile(
name = self.connector.source_path,
source_path = self.connector.source_path,
source_tables = source_tables,
target_entities = target_entities,
source_fk_links = self.source_manager.source_fk_links,
file_format = file_format,
db_type = db_type,
db_host = db_host,
db_port = db_port,
db_name = db_name,
db_user = db_user,
db_pass = db_pass
)
if self.editing_profile_id:
profile.id = self.editing_profile_id
prof_mgr = ProfileManager(self.db)

```

```

        prof_mgr.save_profile(profile)
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('profile_saved'))
def perform_import(self):
    self.links_manager.perform_import()
def create_entity(self):
    dlg = NewEntityDialog(self, self.db)
    if dlg.exec_() == QDialog.Accepted:
        data = dlg.get_data()
        """INSERT INTO meta_entity_types (name, alias, class_id, chaseclass, isversioned, issecuritystrategy, status_id)
        VALUES (%s, %s, %s, %s, %s, %s, %s)""" % (data['name'], data['alias'], data['class_id'],
        )
    if res:
        self.mozart_manager.load_mozart()
        self.links_manager.set_mozart_entities(self.mozart_manager.mozart_entities)
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('entity_created'))
def set_profile_for_edit(self, profile):
    self.restore_profile(profile)
def restore_profile(self, profile):
    self.links_manager.source_fk_links = profile.source_fk_links
    self.links_manager.update_source_links_table()
    self.links_manager.mapped_links = []
    for mapping in link_mappings:
        self.links_manager.mapped_links.append({
            'source_table': mapping['source_table'],
            'target_table': mapping['target_table'],
            'type': 'source' if mapping.get('is_source_id') else 'import'
        })
    self.links_manager.update_import_links_table()
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('profile_loaded'))

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/designer_mass_field_dialog.py

```

# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import QDialog, QVBoxLayout, QTableWidgetItem, QTableWidgetItem, QComboBox,
from PyQt5.QtCore import Qt
from .designer_utils import to_field_name

class MassFieldDialog(QDialog):
    def __init__(self, parent, source_fields, translator):
        super().__init__(parent)
        self.translator = translator
        self.setWindowTitle(self.translator.tr('mass_field_dialog_title'))
        layout = QVBoxLayout(self)
        self.table = QTableWidgetItem(len(source_fields), 4)
        self.translator.tr('source_field_name'),
        self.translator.tr('length')
        self.table.horizontalHeader().setStretchLastSection(True)
        for row, sf in enumerate(source_fields):
            self.table.setItem(row, 0, QTableWidgetItem(sf['source_field_name']))
            suggested = to_field_name(sf['source_field_name'])
            name_item = QTableWidgetItem(suggested)
            type_combo = QComboBox()
            type_combo.addItems(["I", "D", "T", "L", "M", "R"])
            self.table.setItem(row, 2, type_combo)
            length_spin = QSpinBox()
            length_spin.setRange(1, 10000)
            self.table.setItem(row, 3, length_spin)
        layout.addWidget(self.table)
        btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        btn_box.accepted.connect(self.accept)
        btn_box.rejected.connect(self.reject)
        layout.addWidget(btn_box)
    def get_fields(self):
        fields = []
        for row in range(self.table.rowCount()):
            source_field = self.table.item(row, 0).text()
            type_combo = self.table.item(row, 2)
            length_spin = self.table.item(row, 3)
            if type_combo.currentIndex() < 0:
                continue
            fields.append({
                'source_field': source_field,
                'field_type': type_combo.currentText(),
                'length': length_spin.value(),
                'alias': source_field.capitalize()[:50]
            })
        return fields

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/designer_models.py (от

```

# -*- coding: utf-8 -*-
import re
def to_field_name(source_name: str) -> str:
    """Преобразует имя поля PostgreSQL (латиница, подчёркивания)"""
    if name.isdigit():
        return name.lower()[:63]
    return name.lower()[:63]

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/designer_trees.py (от

```

# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import QTreeWidgetItem, QMenu, QAction, QMessageBox, QDialog, QFormLayout,
from PyQt5.QtCore import Qt
from .designer_utils import to_field_name

class SourceTreeManager:
    """Управление деревом таблиц источника"""
    def __init__(self, parent, source_tree, source_links_table, translator, connector):
        self.parent = parent
        self.source_tree = source_tree
        self.source_links_table = source_links_table
        self.translator = translator
        self.connector = connector
        self.source_fk_links = []
    def load_source(self):
        self.source_tree.clear()

```

```

schema = self.connector.get_schema()
self.source_tables = schema.tables
for table in self.source_tables:
    row_count = self.connector.get_row_count(table.name)
    item = QtWidgets.QTreeWidgetItem(table.name, count_str)
    for field in table.fields:
        item.addChild(QTreeWidgetItem(f"{field.name} ({field.field_type.name})", ""))
        item.setData(0, Qt.UserRole, field.name)
        item.setFlags(field.flags | Qt.ItemIsUserCheckable)
        item.setCheckState(0, Qt.Unchecked)
self.source_tree.addTopLevelItem(item)
self.update_source_links_table()

def detect_source_fk_links(self):
    for table in self.source_tables:
        for field in table.fields:
            if field.is_foreign_key and field.refers_to:
                self.source_fk_links.append(
                    {
                        'source_table': table.name,
                        'target_table': field.refers_to,
                        'target_field': id
                    })

def update_source_links_table(self):
    self.source_links_table.setRowCount(len(self.source_fk_links))
    for row, link in enumerate(self.source_fk_links):
        self.source_links_table.setItem(row, 0, QtWidgets.QTreeWidgetItem(link['source_table']))
        self.source_links_table.setItem(row, 1, QtWidgets.QTreeWidgetItem(link['target_table']))
        self.source_links_table.setItem(row, 2, QtWidgets.QTreeWidgetItem(link['target_field']))

def get_checked_fields(self):
    checked = self.source_tree.invisibleRootItem()
    for item in checked.children():
        table_name = item.child(0).text()
        for field in range(table_item.childCount()):
            if item.checkState(field) == Qt.Checked:
                checked.append(
                    {
                        'source_table': table_name,
                        'source_field': field_item.data(0, Qt.UserRole),
                    })
    return checked

def set_checks_for_table(self, table_item, state):
    for field in table_item.children():
        field_item.setCheckState(0, state)

def add_source_link(self, table_item):
    table_name = table_item.data(0, Qt.UserRole)
    dlg = QDialog(self.parent)
    dlg.setWindowTitle(self.translator.tr('add_source_link_title'))
    form = QFormLayout(dlg)
    source_field_combo = QComboBox()
    for table in self.source_tables:
        if table.name == table_name:
            source_field_combo.addItem(f.name, f.name)
    form.addRow(self.translator.tr('source_field'), source_field_combo)
    target_table_combo = QComboBox()
    for table in self.source_tables:
        if table.name != table_name:
            target_table_combo.addItem(table.name, table.name)
    form.addRow(self.translator.tr('target_table'), target_table_combo)
    target_field_combo = QComboBox()
    target_table_combo.currentIndexChanged.connect(lambda: self.update_target_fields(target_table_combo,
        self.update_target_fields(target_table_combo, target_field_combo))
    form.addRow(self.translator.tr('target_field'), target_field_combo)
    btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
    btn_box.accepted.connect(dlg.accept)
    btn_box.rejected.connect(dlg.reject)
    layout.addWidget(btn_box)
    self.source_fk_links.append(
        {
            'source_table': table_name,
            'source_field': source_field_combo.currentData(),
            'target_table': target_table_combo.currentData(),
            'target_field': target_field_combo.currentData(),
        })
    self.update_source_links_table()

def update_target_fields(self, target_table_combo, target_field_combo):
    target_table = target_table_combo.currentData()
    for table in self.source_tables:
        if table.name == target_table:
            for field in table.fields:
                target_field_combo.addItem(f.name, f.name)
            break

def preview_table(self, table_item):
    sample = self.connector.get_table_sample(table_name, limit=20)
    if not sample:
        QMessageBox.information(self.parent, self.translator.tr('info'), self.translator.tr('no_data'))
    dlg = QDialog(self.parent)
    dlg.setWindowTitle(self.translator.tr('preview_table_title').format(table_name))
    layout = QFormLayout(dlg)
    headers = list(sample[0].keys())
    table = QtWidgets.QTable(len(headers))
    table.setColumnCount(len(headers))
    table.setRowCount(len(sample))
    for row, col in enumerate(sample):
        table.setItem(row, col, QtWidgets.QTableWidgetItem(str(row.get(col, "")))
    layout.addWidget(table)
    dlg.exec_()

class MozartTreeManager:
    def __init__(self, parent, mozart_tree, translator, db):
        self.parent = parent
        self.mozart_tree = mozart_tree
        self.translator = translator
        self.mozart_entities = []

    def load_mozart(self):
        self.mozart_tree.clear()
        rows = self.db.execute_query("SELECT id, cname, alias, class_id FROM meta.entitytypes ORDER BY cname")
        for class_id, cname, alias, class_id in rows:
            class_name = self.db.execute_query("SELECT class_id FROM meta.entitytypes WHERE id=%s", (class_id,))
            fields = self.db.execute_query("SELECT fieldname, fieldtype FROM meta.fields WHERE entitytypeid=%s", (class_id,))
            item = QtWidgets.QTreeWidgetItem(f"{cname} ({alias} - {class_name})", "")
            for field in fields:
                item.addChild(QTreeWidgetItem(f"{field.fieldname} ({field.fieldtype})", ""))
                item.setData(0, Qt.UserRole, field.fieldname)
            self.mozart_tree.addTopLevelItem(item)
            self.mozart_entities.append(
                {
                    'id': class_id,
                    'cname': cname,
                    'alias': alias,
                    'class_id': class_id,
                    'fields': [f[0] for f in fields]
                })
        return self.mozart_entities

    def get_entity_by_id(self, entity_id):
        for ent in self.mozart_entities:
            if ent['id'] == entity_id:
                return ent
        return None

```



```

        if link.link_type == LinkType.IMPORT:
            self.scene.links.remove(link)
        rows = self.db.execute_query(
            "SELECT id, cname, alias, class_id FROM meta.entitytypes ORDER BY cname"
        )
        divider = self.scene.divider_x
        for eid, cname, alias, class_id in rows:
            class_res = self.db.execute_query(
                "SELECT class FROM meta.entity_classes WHERE id=%s", (class_id,)
            )
            class_alias = class_res[0][0] if class_res else "DICT"
            self.ext_helper.ensure_field(
                field_name=SOURCE_ID_FIELD,
                field_type=SOURCE_ID,
                alias=source_id,
                length=255
            )
            fields = []
            from self.db.execute_query(
                "SELECT cname, ctype FROM meta.fields WHERE entitytypeid=%s ORDER BY isortorder",
                (eid,)
            )
            for fname, ctype in rows:
                dt = DataTypeId(ctype)
                if dt == 'N': dt = DataType.INTEGER
                elif dt == 'Y': dt = DataType.BOOLEAN
                fields.append(Field(name=fname, data_type=dt))
            node = Node(id=f"moz_{eid}", name=f"{cname} ({alias})",
                       node_type=NodeType.MOZART_ENTITY, fields=fields)
            self.scene.add_node(node)
            if x > self.scene.height() - 200:
                x += 280
            for ld in existing_links:
                src_node, tgt_node, link_type = ld.get('mappings')
                self.scene.add_link(src_node, tgt_node, LinkType.IMPORT, mappings=ld.get('mappings'))

# Диалог настройки связи (импорта)
def edit_link_between(self, source_node_item, mozart_node_item, existing_mappings, checked_source_fields,
                      source_refs):
    checked_source_fields = [fi.field.name for fi in source_node_item.field_items]
    target_fields = [fi.field.name for fi in mozart_node_item.field_items]
    target_fields.append(SOURCE_ID_FIELD)
    dlg = LinkEditDialog(checked_source_fields, target_fields, existing_mappings, source_refs, self)
    if dlg.exec():
        new_mappings = dlg.get_mappings()
        if not new_mappings:
            old_link = self.scene.find_link_between(source_node_item, mozart_node_item)
            self.scene.remove_item(old_link)
            self.scene.add_link(source_node_item, mozart_node_item, LinkType.IMPORT, mappings=new_mappings)
            QMessageBox.information(self, "Связь обновлена", "Маппинг полей применён.")

# Диалог настройки ссылочных полей источника
def edit_source_refs(self, source_node_item):
    table_name = source_node_item.node.table_name
    all_tables = []
    for ni in self.scene.nodes.values():
        if ni.is_checked() and ni.node.type == SOURCE_TABLE:
            current_refs = getattr(source_node_item.node, 'refs', {})
            dlg = SourceRefDialog(table_name, fields, all_tables, current_refs, self)
            if dlg.exec():
                new_refs = dlg.get_refs()
                for ni in self.scene.nodes.values():
                    if ni.is_checked() and ni.node.type == SOURCE_TABLE and ni.node.table_name == table_name:
                        ni.node.refs = new_refs
                        break
            QMessageBox.information(self, "Связность", "Настройки ссылочных полей сохранены.")

# Превью, создание сущности, создание полей
def preview_table(self, node_item):
    table_name = node_item.node.table_name
    try:
        sample = self.connector.get_table_sample(table_name, limit=100)
    except:
        QMessageBox.warning(self, "Ошибка", "Не удалось получить данные: {e}")
    if not sample:
        QMessageBox.information(self, self.translator.tr('info'),
                                self.translator.tr('no_data'))
        return
    dlg = QDialog(self)
    dlg.setWindowTitle(table_name)
    layout = QVBoxLayout(dlg)
    headers = QTableWidget(101, keys())
    headers.setColumnHeader(0, headers.headers())
    table = QTableWidget(headers.headers())
    table.setColumnHeader(0, headers.headers())
    for rowid, row in enumerate(sample):
        for col, cell in enumerate(row):
            table.setItem(rowid, col, QTableWidgetItem(str(row.get(col, ""))))
    layout.addWidget(table)
    dlg.exec_()

def create_entity_from_source(self, source_node_item):
    from metamodelwizard.dialogs import NewEntityDialog
    dlg_name_edit = NewEntityDialog(self.db)
    dlg_name_edit.setText(source_node_item.node.name.capitalize())
    if dlg_name_edit.exec():
        sample = self.connector.get_table_sample(
            source_node_item.node.table_name, limit=1)
        except Exception:
            checked = source_node_item.get_checked_fields()
            entity_id = self.entity_creator.create_entity_from_source(
                entity_name=source_node_item.node.name,
                class_id=source_node_item.node.class_id,
                source_table_name=source_node_item.node.table_name,
                checked_fields=checked,
                source_sample=sample)
            if entity_id:
                self.read_mozart_entities()
                QMessageBox.information(self, self.translator.tr('info'),
                                        self.translator.tr('entity_created'))

def create_fields_for_mozart(self, mozart_node_item):
    for ni in self.scene.nodes.values():
        if ni.is_checked() and ni.node.type == NodeType.SOURCE_TABLE and ni.is_checked():
            break
    if not src_item:

```

```

        QMessageBox.warning(self, "Предупреждение", "Отметьте таблицу источника")
    return
sample = None
try:
    sample, rows = self.connector.get_table_sample(
        src_item.node.table_name, limit=1)
    if sample:
        sample_rows[0]
    except Exception:
        pass
fields_to_create = []
for src_field in src_item.get_checked_fields():
    from_designer_to_field_name = to_field_name(src_field)
    field_type = 755
    if sample and src_field in sample:
        val = sample[src_field]
        if isinstance(val, int):
            field_type = 7
            length = 0
        elif isinstance(val, float):
            field_type = 15
        elif isinstance(val, str):
            length = 0
        fields_to_create.append(
            (field_name, field_type,
             alias, src_field.capitalize()[:50],
             length, not True))
    else:
        field_type = 755
if fields_to_create:
    self.field_creator.create_fields(mozart_node_item.node.entity_id, fields_to_create)
    QMessageBox.information(self, "Поля созданы",
        "Создано {len(fields_to_create)} полей")
# Сохранение и импорт
def save_profile(self):
    profile = self.scene_adapter.scene_to_profile(self.scene, self.connector, self.db, self.editing_profile_id)
    self.connector.save_profile(profile)
    QMessageBox.information(self, self.translator.tr('info'),
        self.translator.tr('profile_saved'))
def perform_import(self):
    profile = self.scene_adapter.scene_to_profile(
        self.scene, self.connector, self.editing_profile_id)
    orchestrator = ImportOrchestrator(self.db, self.connector, profile)
    try:
        stats = orchestrator.run()
        msg = self.translator.import_finished_stats().format(
            stats)
        QMessageBox.information(self, self.translator.tr('import_finished_title'), msg)
        if orchestrator.errors:
            # выводим первые 20 ошибок
            if len(orchestrator.errors) > 20:
                QMessageBox.warning(self, self.translator.tr('orchestrator_errors_20'),
                    "Ошибок больше 20, выводим первые 20 ошибок")
            # выводим первые 10 ошибок в консоль
            for err in stats['details'][:10]:
                print("детали ошибок из движка:")
            except Exception as e:
                QMessageBox.critical(self, self.translator.tr('error'), str(e))
def load_profile_links(self, profile: ImportProfile):
    for link in self.scene.links:
        if link.link_type == LinkType.IMPORT:
            self.scene.remove_link(link)
    SceneProfileAdapter.restore_scene(self.scene, profile)
# Вспомогательные методы
def find_field_item(self, node_item, field_name):
    for field in node_item.fields:
        if field.name == field_name:
            return field
    return None
def find_node_by_table(self, table_name):
    for ni in self.scene.nodes.values():
        if ni.node_type == NodeType.SOURCE_TABLE and ni.node.table_name == table_name:
            return ni
    return None
def find_node_by_entity(self, entity_id):
    for ni in self.scene.nodes.values():
        if ni.node_type == NodeType.MOZART_ENTITY and ni.node.entity_id == entity_id:
            return ni
    return None

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/entity_creator.py (открыть)

```

# -*- coding: utf-8 -*-
from typing import Dict, Any, Optional
from PyQt5.QtCore import QObject
from PyQt5.QtDatabaseService import DatabaseService
from PyQt5.QtDesigner import Designer
from PyQt5.QtGui import QIcon
from PyQt5.QtWidgets import QMessageBox
from lang.local_translator import LocalTranslator

class EntityCreator:
    """
    Создает новую сущность Mozart на основе таблицы источника.
    копирует все отмеченные поля из источника в сущность.
    """
    def __init__(self, db: DatabaseService):
        self.db = db
        self.field_creator = FieldCreator(db)
        self.translator = LocalTranslator()

    def create_entity_from_source(self,
        entity_name: str,
        alias: str,
        source_table_name: str,
        checked_fields: list, # список имен полей источника
        source_sample: Optional[Dict[str, Any]] = None) -> Optional[int]:
        """
        Создает тип сущности ext-таблицы.
        Возвращает entity_id новой сущности.
        """
        # 1. Создаём запись в meta.entitytypes
        res = self.db.execute_query(
            """
            INSERT INTO meta.entitytypes
            (name, alias, class_id, baseclass, isversioned, issecuritystrategy, status_id)
            VALUES (%s, %s, %s, %s, %s, %s, %s)
            """
            % (entity_name, entity_alias, class_id, RETURNING id)
        )
        if not res:
            return None
        entity_id = res[0][0]

        # 2. Обеспечиваем существование ext-таблицы
        self.ext_helper.ensure_ext_table(entity_id)

        # 3. Создаём поля
        for field in checked_fields:
            # Генерируем имя поля Mozart (латиница, без спецсимволов)
            from_designer_to_field_name = to_field_name(field)
            # Генерируем тип поля Mozart (целое, вещное, строка и т.д.)
            field_type = 755 # строка по умолчанию
            if source_sample and src_field in source_sample:

```

```

        val = source_sample[src_field]
        instance(val, int):
            field_type = 0
        elif isinstance(val, float):
            field_type = 1
        elif isinstance(val, bool):
            field_type = 2
        else:
            field_type = 'C'
        fields_to_create.append(
            {'fieldname': field,
             'field_type': field_type,
             'length': field.capitalize():[:50],
             'indexed': True},
        )
    if fields_to_create:
        self.field_creator.create_fields(entity_id, fields_to_create)
    return entity_id

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/import_designer.py (0

```

# mozart_import/designer/designer_main.py
import sys
from PyQt5.QtWidgets import (QMainWindow, QSplitter, QVBoxLayout, QWidget, QTreeWidget,
                             QFormLayout, QLineEdit, QComboBox, QFileDialog,
                             QLabel, QCheckBox, QSpinBox, QFrame)
from PyQt5.QtCore import QObject, QTranslator, Qt
from PyQt5.QtGui import QIcon
from .core.models import SourceField, FieldType, ImportProfile
from .utils.field_source_type import MappingProposal, MatchedEntity, MatchedField
from .core import engine, import_profile_manager
from .designer_mass_field_dialog import MassFieldDialog

class ImportDesigner(QMainWindow):
    def __init__(self, on_connector, parent=None):
        super().__init__(parent)
        self.connector = connector
        self.translator = QTranslator()
        self.translator.load(self.translator.tr('import_designer_title'))
        self.editing_profile_id = None
        central = QWidget()
        main_layout = QVBoxLayout(central)
        top_widget = QWidget()
        top_layout = QHBoxLayout(top_widget)
        self.splitter = QSplitter(Qt.Horizontal)
        top_layout.addWidget(self.splitter)
        left_tabs = QTabWidget()
        self.source_tree = QTreeWidget()
        self.source_tree.setHeaderLabels([self.translator.tr('source_tables'), self.translator.tr('rows')])
        self.source_tree.setContextMenu(self.create_context_menu(self.source_tree.context_menu))
        left_tabs.addTab(self.source_tree, self.translator.tr('source_tables'))
        self.source_links_table = QTableWidget(0, 4)
        self.translator.tr('source_table'),
        self.translator.tr('source_field'),
        self.translator.tr('target_field'),
        self.source_links_table.horizontalHeader().setStretchLastSection(True)
        self.splitter.addWidget(left_tabs)
        self.mozart_tree = QTreeWidget()
        self.mozart_tree.setContextMenu(self.create_context_menu(self.mozart_tree.context_menu))
        self.splitter.addWidget(self.mozart_tree)
        main_layout.addWidget(top_widget, 2)
        splitter_handle = QFrame()
        splitter_handle.setFrameShape(QFrame.HLine)
        splitter_handle.setFrameSize(QFrame.Size)
        splitter_handle.setCursor(Qt.SizeVerCursor)
        splitter_handle.mousePressEvent = self.start_splitter_drag
        splitter_handle.mouseReleaseEvent = self.stop_splitter_drag
        main_layout.addWidget(splitter_handle)
        self.dragging = False
        self.start_height = 0
        self.links_table = QTableWidget(0, 6)
        self.translator.tr('source_table'),
        self.translator.tr('source_field'),
        self.translator.tr('target_field'),
        self.translator.tr('type'),
        self.links_table.horizontalHeader().setStretchLastSection(True)
        main_layout.addWidget(self.links_table, 1)
        btn_layout = QHBoxLayout()
        btn_add_link = QPushButton(self.translator.tr('add_import_link'))
        btn_save = QPushButton(self.translator.tr('save_profile'))
        btn_import = QPushButton(self.translator.tr('start_import'))
        btn_create_entity = QPushButton(self.translator.tr('create_entity'))
        btn_layout.addWidget(btn_add_link)
        btn_layout.addWidget(btn_save)
        btn_layout.addWidget(btn_import)
        btn_layout.addWidget(btn_create_entity)
        main_layout.addLayout(btn_layout)
        self.source_tables = []
        self.mozart_tables = []
        self.import_links = []
        self.load_source()
        self.translator.load()
        self.links_table.setMinimumHeight(150)
        self.links_table.setMaximumHeight(400)
    def set_profile_for_edit(self, profile):
        self.restore_profile(profile)
    def restore_profile(self, profile):
        self.source_tables = []
        self.mozart_tables = []
        self.import_links = []
        self.links_table.setRowCount(0)
        self.load_source()
        self.source_fk_links = profile.source_fk_links

```

```

self.update_source_links_table()
self.import_links = 1
for link in profile.import_links:
    for ent_id in self.mozart_entities:
        if ent_id == link['target_entity_id']:
            target_entity_id = ent_id
        if not target_entity_id:
            continue
        for mapping in link['mappings']:
            source_table = link['source_table']
            source_field = mapping['source_field']
            target_field = mapping['target_field']
            type = source_id_mappings.get('is_source_id') else 'import'
        self.update_import_links_table()
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('profile_loaded'))
def start_splitter_drag(self, event):
    self.drag_start_y = event.globalY()
    self.start_height = self.links_table.height()
    self.setCursor(Qt.SizeVerCursor)
def drag_splitter(self, event):
    self.is_dragging = True
    delta = event.globalY() - self.drag_start_y
    new_height = max(100, min(800, self.start_height + delta))
    self.links_table.setMaximumHeight(new_height)
def stop_splitter_drag(self, event):
    self.setCursor(Qt.ArrowCursor)
def retranslate_ui(self):
    self.setWindowTitle(self.translator.tr('import_designer_title'))
    self.mozart_tree.setHeaderLabels([self.translator.tr('source_tables'), self.translator.tr('rows')])
    self.source_links_table.setHeaderLabels(['mozart_entities', ''])
    self.translator.tr('source_table'),
    self.translator.tr('target_field'),
    self.translator.tr('type'),
    self.links_table.setHorizontalHeaderLabels([
        self.translator.tr('source_table'),
        self.translator.tr('target_field'),
        self.translator.tr('type'),
    ])
    self.btn_create_entity.setText(self.translator.tr('create_entity'))
    self.btn_save.setText(self.translator.tr('add_import_link'))
    self.btn_import.setText(self.translator.tr('start_import'))
def load_source(self):
    self.source_tree.clear()
    self.conn = self.connector.get_schema()
    for table in self.conn.tables:
        row_count = self.connector.get_row_count(table.name)
        count = self.connector.get_row_count(table.name) if row_count != '?' else '?'
        item = QTableWidgetItem(QStringRole, table.name, count_str)
        for field in table.fields:
            field_item = QTableWidgetItem(f'{field.name} ({field.field_type.name})', '')
            field_item.setFlags(field.flags | Qt.ItemIsUserCheckable)
            item.setData(Qt.CheckState(0), Qt.Unchecked)
        self.source_tree.addItem(item)
    self.detect_source_fk_links()
def detect_source_fk_links(self):
    self.source_fk_links = []
    for table in self.conn.tables:
        for field in table.fields:
            if field.is_foreign_key and field.refers_to:
                source_table = table.name
                target_table = field.refers_to
                self.source_fk_links.append((source_table, target_table, field.refers_to))
def update_source_links_table(self):
    self.source_links_table.setRowCount(len(self.source_fk_links))
    for i, link in enumerate(self.source_fk_links):
        self.source_links_table.setItem(row, 0, QTableWidgetItem(link['source_table']))
        self.source_links_table.setItem(row, 1, QTableWidgetItem(link['target_field']))
        self.source_links_table.setItem(row, 2, QTableWidgetItem(link['type']))
def load_mozart(self):
    self.mozart_tree.clear()
    rows = self.db.execute_query("SELECT id, cname, alias, class_id FROM meta.entitytypes ORDER BY cname")
    for row in rows:
        class_id = row[4]
        class_name = row[1]
        class_alias = row[2]
        class_id_in_rows = SELECT class_id FROM meta.entity_classes WHERE id=%s", (class_id,)
        fields = self.db.execute_query("SELECT f.cname, f.ftype FROM meta.fields WHERE entitytypeid=%s", (class_id,))
        item = QTableWidgetItem(f'{class_name} ({class_alias}) - {class_id}', '')
        for name, type in fields.items():
            field_item = QTableWidgetItem(f'{name} ({ftype})', '')
            item.addChild(field_item)
        self.mozart_tree.addItem(item)
    self.id, cname, alias, class_id, class_alias, fields = [f[0] for f in fields]
def source_context_menu(self, pos):
    if not item := self.source_tree.itemAt(pos):
        return
    menu = QMenu()
    menu.addAction(QAction(self.translator.tr('preview_table'), None))
    menu.addAction(QAction(self.translator.tr('add_source_link'), None))
    menu.addAction(QAction(self.translator.tr('select_all_fields'), None))
    menu.addAction(QAction(self.translator.tr('deselect_all_fields'), None))
    menu.exec_()
def set_checks_for_table(self, table, state):
    for field in table.fields:
        field.setChecked(state)
def add_source_link(self, table, item):
    dialog = QDialog(self)
    dialog.setWindowTitle(self.translator.tr('add_source_link_title'))
    form = QVBoxLayout()
    source_field_combo = QComboBox()
    for f in table.fields:
        source_field_combo.addItem(f.name, f.name)
    form.addWidget(source_field_combo)
    target_table_combo = QComboBox()

```

```

for t in self.source_tables:
    if t.name == table_name:
        target_table_combo.addItem(t.name, t.name)
        form.addRow(self.translator.tr('target table'), target_table_combo)
target_table_combo.currentIndexChanged.connect(lambda: self.update_target_fields(target_table_combo,
target_field_combo))
self.update_target_fields(target_table_combo, target_field_combo)
layout.addWidget(tranlator.tr('target field'), target_field_combo)
btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
btn_box.accepted.connect(lambda: self.accept())
layout.addWidget(btn_box)
if not self.source_fk_links:
    self.source_fk_links.append({
        'source_field': source_field_combo.currentData(),
        'target_table': target_table_combo.currentData(),
    })
self.update_source_links_table()
def update_target_fields(self, target_table_combo, target_field_combo):
    target_table_combo.clear()
    for f in self.source_tables:
        if f.name == target_table:
            target_field_combo.addItem(f.name, f.name)
            break
def moztart_context_menu(self, pos):
    item = self.mozart_tree.itemAt(pos)
    if not item:
        return QMenu()
    if item.parent() is None:
        create_action = QAction(self.translator.tr('create fields from checked'), None)
        create_action.triggered.connect(lambda: self.create_fields_from_checked(item))
    menu.addAction(self.mozart_tree.mapToGlobal(pos))
def create_fields_from_checked(self, target_entity_item):
    checked_fields = target_entity_item.data(0, Qt.UserRole)
    for i in range(target_entity_item.childCount()):
        for j in range(target_entity_item.childCount()):
            if checked_fields[i][j] == Qt.Checked:
                'source_table': target_entity_item.data(0, Qt.UserRole),
    if not checked_fields:
        QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('no_fields_selected'))
    dlg = MassFieldDialog(self, checked_fields, self.translator)
    if dlg.exec_() == QDialog.Accepted:
        fields_to_create = dlg.get_fields()
        if not fields_to_create:
            return
        creator = FieldCreator(self.db)
        for f in fields_to_create:
            creator.create_fields(entity_id, [f,
            'field_type': f['field_type'],
            'alias': f['alias'],
            'length': f['length'],
        ])
    self.load_mozart()
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('fields_created'))
def preview_table(self, table_name):
    table_name = table_name.lower()
    sample = self.connector.get_table_sample(table_name, limit=20)
    if not sample:
        QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('no_data'))
    dlg = QDialog(self)
    title = self.translator.tr('preview_table_title').format(table_name)
    layout = QVBoxLayout(dlg)
    headers = list(sample[0].keys())
    table = QtWidgets.QTable(headers)
    table.setColumnCount(len(headers))
    for row in enumerate(sample):
        for col in enumerate(headers):
            table.setItem(row.get(col, ''))
    layout.addWidget(table)
    dlg.exec_()
def create_entity(self):
    from NewEntityDialog import NewEntityDialog
    dlg = NewEntityDialog(self, self.db)
    if dlg.exec_() == QDialog.Accepted:
        data = dlg.get_data()
        """INSERT INTO meta.entitytypes (name, alias, class_id, superclass, isversioned, issecuritystrategy, status_id)
VALUES (%s, %s, %s, %s, %s, %s, %s) RETURNING id"""
        (data['name'], data['alias'], data['class_id'])
    if res:
        entity_id = res[0][0]
        QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('entity_created'))
def add_import_link(self):
    dlg = QDialog(self)
    title = self.translator.tr('add_import_link_title')
    layout = QVBoxLayout(dlg)
    form = QDialogForm()
    for src_table_combo in self.source_tables:
        form.addRow(self.translator.tr('source table'), src_table_combo)
    src_table_combo.currentIndexChanged.connect(lambda: self.update_src_fields(src_table_combo,
src_field_combo))
self.update_src_fields(src_table_combo, src_field_combo)
form.addRow(self.translator.tr('src field'), src_field_combo)
for ent in self.mozart_entities:
    target_entity_combo.addItem(f'{ent["name"]} ({ent["alias"]})', ent['id'])
target_entity_combo.currentIndexChanged.connect(lambda: self.update_target_fields(target_entity_combo,
target_field_combo))
self.update_target_fields(target_entity_combo, target_field_combo)
type_combo = QDialogForm()
form.addRow(self.translator.tr('import type'), type_combo)
form.addRow(self.translator.tr('source id'))
layout.addWidget(btn_box)
btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
btn_box.accepted.connect(lambda: self.accept())
layout.addWidget(btn_box)
dlg.exec_()
self.source_links.append({
    'source_table': src_table_combo.currentData(),
    'source_field': src_field_combo.currentData(),
    'target_table': target_entity_combo.currentData(),
    'type': type_combo.currentData()
})
self.update_import_links_table()
def update_src_fields(self, src_table_combo, src_field_combo):
    src_table_combo.clear()
    for f in self.source_tables:
        if f.name == table_name:
            src_field_combo.addItem(f.name, f.name)
            break

```



```

def update_target_fields_entity(self, target_entity_combo, target_field_combo):
    target_field_combo.clear()
    for ent in self.mozart_entities:
        if ent in self.entity_fields:
            for field in self.entity_fields:
                target_field_combo.addItem(fname, fname)
            break

def update_import_links_table(self):
    self.links_table.setRowCount(len(self.import_links))
    for row, link in enumerate(self.import_links):
        self.links_table.setItem(row, 0, QTableWidgetItem(link['source_table']))
        entity_name = self.mozart_entities[link['target_entity_id']]:
        self.links_table.setItem(row, 3, QTableWidgetItem(entity_name))
        self.links_table.setItem(row, 4, QTableWidgetItem(link['type'] or ''))
    btn = QPushButton(self.translator.tr('delete'))
    self.clicked.connect(lambda checked, r=row: self.remove_import_link(r))
    self.links_table.setCellWidget(row, 5, btn)

def remove_import_link(self, row):
    self.update_import_links_table()

def save_profile(self):
    source_tables = []
    for table, nrow in source_tables:
        nrow = self.connector.get_row_count(table.name)
        sample_rows = sample_rows[:5]
        source_tables.append((table, nrow))
    target_entities = []
    for link in self.import_links:
        if link['target_entity_id'] not in target_entities:
            if link['target_entity_id'] in self.mozart_entities:
                target_entities.append({'id': link['target_entity_id'], 'name': self.mozart_entities[link['target_entity_id']]['cname']})
            else:
                break
    import_links = []
    for mapping in self.mappings:
        source_table = mapping['source_table']
        target_entity_id = mapping['target_entity_id']
        mappings.append({'source_table': source_table, 'target_entity_id': target_entity_id})
    profile = ImportProfile(
        name=self.connector.source_path,
        source_path=self.connector.source_path,
        source_tables=source_tables,
        target_entities=target_entities,
        source_fk_links=self.source_fk_links
    )
    if self.editing_profile_id:
        profile.id = self.editing_profile_id
    profile.save(self.db)
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('profile_saved'))

def perform_import(self):
    for link in self.import_links:
        if link['type'] == 'import' and link['target_field']:
            check = self.db.execute_query(
                (link['target_entity_id'], link['target_field'])
            )
            if not check:
                create_fields(link['target_entity_id'], [{
                    'fieldname': link['target_field'],
                    'length': 255
                }])
            elif link['type'] == 'sourceid':
                check = self.db.execute_query(
                    (link['target_entity_id'],)
                )
                if not check:
                    create_fields(link['target_entity_id'], [{
                        'fieldname': 'sourceid',
                        'length': 100
                    }])
    groups = []
    for link in self.import_links:
        if link['target_entity_id'] not in groups:
            groups.append(link['target_entity_id'])
    entities = []
    for link in groups.items():
        target_entity_id = link[0]
        if target_entity_id in self.mozart_entities:
            target_entity = self.mozart_entities[target_entity_id]
            if not target_entity:
                continue
            source_table_name = links[0]['source_table']
            matched_fields = []
            for link in self.mappings:
                if link['type'] == 'import' and link['target_field']:
                    matched_fields.append({'source_table': link['source_table'], 'target_field': link['target_field']})
            entities.append(MatchedEntity(
                source_table=source_table_name,
                target_entity_name=target_entity['cname'],
                confidence=1.0,
                fields=matched_fields,
                is_new=False
            ))
    mapping_proposal = MappingProposal(
        entities=entities,
        suggested_tables=[],
        suggestions_for_new_entities=[]
    )
    profile = ImportProfile(
        name=self.connector.source_path,
        source_path=self.connector.source_path,
        mapping_proposal=mapping_proposal
    )
    progress = QProgressDialog(self.translator.tr('import_progress'), self.translator.tr('cancel'), 0, 0, self)
    progress.setWindowModality(Qt.WindowModal)
    try:
        engine = ImportEngine(self.db, self.connector, profile)
        progress.exec_()
    except:
        QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('import_failed'))

```

```
self.translator.tr('import_finished_stats').format(stats['inserted'], stats['updated'], stats['errors']))
except Exception as e:
    QMessageBox.critical(self, self.translator.tr('error'), str(e))
```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/link_edit_dialog.py (открыть)

```
# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import (QDialog, QVBoxLayout, QTableWidget, QTableWidgetItem,
                             QComboBox, QCheckBox, QDialogButtonBox, QHeaderView,
                             QMessageBox)
from PyQt5.QtCore import Qt
from lang.local_translator import LocalTranslator

class LinkEditDialog(QDialog):
    """Диалог настройки связи импорта.
    source_fields - список имен полей источника, уже отфильтрованных пользователем (чекбоксы).
    target_fields - полный список полей Mozart (включая source_id).
    source_refs - словарь ссылок {field name: {is_ref: bool, ref_table: str, ref_field: str}}
    """
    def __init__(self, source_fields: list, target_fields: list, current_mappings=None, source_refs=None,
                 parent=None):
        super().__init__(parent)
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('mapping_details_title'))
        self.resize(800, 400)
        self.source_fields = source_fields
        self.target_fields = target_fields
        self.current_mappings = current_mappings or []
        self.mappings = {}
        self.mappings = self.source_fields or {}
        layout = QVBoxLayout(self)
        self.table = QTableWidget(len(source_fields), 4)
        self.table.setHorizontalHeaderLabels([
            self.translator.tr('source_field'),
            self.translator.tr('source_id_type'),
            self.translator.tr('target_field'),
            self.translator.tr('ref_info')
        ])
        self.table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        self.table.verticalHeader().setVisible(False)
        for row, src_field in enumerate(source_fields):
            # Имя поля источника
            item = QTableWidgetItem(src_field)
            self.table.setItem(row, 0, item)
            # Выпадающий список полей Mozart
            combo = QComboBox()
            for tgt in target_fields:
                combo.addItem(self.translator.tr('ignore_field'), None) # ключ None = игнорировать
            self.table.setCellWidget(row, 1, combo)
            # Чекбокс ключевого поля (sourceid)
            chk_key = QCheckBox()
            self.table.setCellWidget(row, 2, chk_key)
            # Чекбокс "ссылка" (информативный)
            chk_ref = QCheckBox()
            if ref_info := self.source_refs.get(src_field, {}):
                if ref_info.get('is_ref', False):
                    chk_ref.setChecked(True)
            else:
                chk_ref.setChecked(False)
            self.table.setCellWidget(row, 3, chk_ref)
            # Восстанавливаем сохранённые маппинги (регистронезависимо)
            for mapping in self.current_mappings:
                if src_field == mapping['source_field']:
                    row = source_fields.index(src_field)
                    tgt = mapping['target_field']
                    # Регистронезависимый поиск
                    combo = self.table.cellWidget(row, 1)
                    for i in range(combo.count()):
                        if combo.itemData(i) and combo.itemData(i).lower() == tgt.lower():
                            idx = i
                            if idx < 0:
                                continue
                            if mapping.get('source_id', None) == src_field:
                                self.table.cellWidget(row, 2).setChecked(True)
            layout.addWidget(self.table)
            btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
            btn_box.rejected.connect(self.reject)
            layout.addWidget(btn_box)
        def on_source_id_changed(self, changed_row, state):
            state = Qt.Checked
            for row in range(self.table.rowCount()):
                if row == changed_row:
                    self.table.cellWidget(row, 2).setChecked(False)
        def on_accept(self):
            self.mappings = []
            for row in range(self.table.rowCount()):
                combo = self.table.cellWidget(row, 1)
                tgt = combo.currentData()
                if tgt:
                    if self.table.cellWidget(row, 2).isChecked():
                        src_field = self.source_fields[row]
                        mapping = {
                            'source_field': src_field,
                            'target_field': tgt,
                            'is_ref': self.table.cellWidget(row, 3).isChecked(),
                            'ref_table': self.translator.tr('ref_table'),
                            'ref_field': self.translator.tr('ref_field')
                        }
                        self.mappings.append(mapping)
            self.accept()
        def get_mappings(self):
            return self.mappings
```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/scene_profile_adapter.py (открыть)

```
# -*- coding: utf-8 -*-
```

```

from typing import Optional
from database import DatabaseService
from ..core.models import ImportProfile, SourceType, FileFormat, NodeType, LinkType

class SceneProfileAdapter:
    @staticmethod
    def get_entity_name(db: DatabaseService, entity_id: int) -> str:
        """Возвращает сцену в объект ImportProfile с гарантированным заполнением имени целевой сущности"""
        if not entity_id:
            return ""
        rows = db.execute_query("SELECT cname FROM meta.entitytypes WHERE id = %s", (entity_id,))
        return rows[0][0] if rows else ""

    @staticmethod
    def get_profile(scene, connector, db: DatabaseService, editing_profile_id: Optional[int] = None) -> ImportProfile:
        """Получает сцену в объект ImportProfile с гарантированным заполнением имени целевой сущности"""
        # Параметры подключения
        file_format = None
        db_type = None
        db_port = None
        db_name = None
        db_user = None
        db_pass = None

        if hasattr(connector, 'file_format'):
            src_type = SourceType.FILE
            file_format = connector.file_format.value if isinstance(connector.file_format, FileFormat) else None
        else:
            src_type = SourceType.DATABASE
            db_type = connector.db_type
            db_port = connector.port if connector.port else None
            db_name = connector.database
            db_pass = connector.password

        # Таблицы источника
        source_tables = []
        for node_item in scene.nodes.values():
            if node_item.node_type != NodeType.SOURCE_TABLE:
                row_count = None
                try:
                    row_count = connector.get_row_count(node_item.node.table_name)
                except Exception:
                    pass
                sample_rows = []
                try:
                    sample = connector.get_table_sample(node_item.node.table_name, limit=5)
                except Exception:
                    pass
                source_tables.append({
                    'name': node_item.node.table_name,
                    'row_count': row_count,
                    'sample_rows': sample_rows
                })

        # Группируем связи импорта по (source_table, target_entity_id)
        links_by_target_entity_id = {}
        for link in scene.links:
            if link.link_type != LinkType.IMPORT:
                continue
            start_node = link.start_item.node
            end_node = link.end_item.node
            if end_node.entity_id is None:
                continue

            key = (start_node.table_name, end_node.entity_id)
            if key not in links_by_target_entity_id:
                entity_name = end_node.name
                if not entity_name and end_node.entity_id:
                    entity_name = db.get_entity_name(end_node.entity_id)
                # Если в базе пусто, используем имя из узла или заглушку
                if not entity_name:
                    entity_name = end_node.name or f"Entity_{end_node.entity_id}"
                links_by_target_entity_id[key] = {
                    'source_table': start_node.table_name,
                    'target_entity_id': end_node.entity_id,
                    'mappings': []
                }

        # Обрабатываем маппинги линии (принудительно берём из link.mappings)
        for link in scene.links:
            src_field = link.get('source_field')
            enriched = {}
            if src_field:
                src_info = link.get('src_info')
                target_field = src_info.get('target_field')
                is_reference = src_info.get('is_reference', False)
                ref_source_table = src_info.get('ref_source_table')
                ref_source_key_field = src_info.get('ref_source_key_field')
                ref_info = {}
                if ref_source_table:
                    ref_info['ref_source_table'] = ref_source_table
                if ref_source_key_field:
                    ref_info['ref_source_key_field'] = ref_source_key_field
                enriched['source_field'] = src_field
                enriched['target_field'] = target_field
                enriched['is_reference'] = is_reference
                enriched['ref_source_table'] = ref_info.get('ref_source_table')
                enriched['ref_source_key_field'] = ref_info.get('ref_source_key_field')

            # Защита от дубликатов
            if not any(ex['source_field'] == enriched['source_field'] and ex['target_field'] == enriched['target_field'] for ex in links_by_target_entity_id[key]['mappings']):
                links_by_target_entity_id[key]['mappings'].append(enriched)

        # Одиночный маппинг
        src_field = link.get('source_field')
        ref = link.get('ref')
        mapping = {}
        if src_field:
            src_info = link.get('src_info')
            target_field = src_info.get('target_field')
            is_reference = src_info.get('is_reference', False)
            ref_source_table = src_info.get('ref_source_table')
            ref_source_key_field = src_info.get('ref_source_key_field')
            mapping['source_field'] = src_field
            mapping['target_field'] = target_field
            mapping['is_reference'] = is_reference
            mapping['ref_source_table'] = ref_source_table
            mapping['ref_source_key_field'] = ref_source_key_field

            # Автоматическое добавление ключевой пары (sourceid) если отсутствует
            for key, links in links_by_target_entity_id.items():
                for m in links:
                    if m.get('is_source_id') is None:
                        source_table_name = entry['source_table']
                        for n in scene.nodes.values():
                            if (n.node_type == NodeType.SOURCE_TABLE and
                                n.name == source_table_name):
                                break
                        key_field = None
                        for n in scene.nodes.values():
                            if n.name == source_table_name:
                                key_field = n.fields[0].name if source_node.node.fields else None
                                break
                        if key_field:
                            entry['mappings'].append({
                                'source_field': key_field,
                                'target_field': 'sourceid',
                                'is_reference': False,
                                'ref_source_table': ""
                            })

```

```

target_entities = []
for target_entity_id in links by target items():
    target_entity_id = entry['target_entity_id'], 'name': entry['target_entity_name']]
    source_table = entry['source_table'],
    target_entity_id = entry['target_entity_id'],
    mappings = entry['mappings'] -
}
# FK связи источника
for link in scene.links:
    if link.link_type == LinkType.FK_SOURCE:
        source_fk_links.append({'start_item_node.table_name',
                                'source_field': link.start_item_node.field_name,
                                'target_table': link.end_item_node.table_name,
                                'target_field': link.end_item_node.field_name
                                })
# Сбор ручных настроек ссылок источника (source_references)
for node_item in scene.nodes.values():
    if node_item.node_type != NodeType.SOURCE_TABLE:
        continue
    ref_attr = getattr(node_item, 'refs', {})
    for ref_attr in ref_attr.items():
        if ref_attr.get('ref') in refs.items():
            source_references.append({
                'source_table': node_item.table_name,
                'ref_table': ref_attr['ref_table'],
                'ref_field': ref_attr['ref_field']
            })
return ImportProfile(
    id=editing_profile_id,
    name=connector_source_path,
    source_path=connector_source_path,
    source_tables=source_tables,
    target_entities=target_entities,
    source_fk_links=source_fk_links,
    source_references=source_references,
    file_format=file_format,
    db_host=db_host,
    db_port=db_port,
    db_name=db_name,
    db_pass=db_pass
)
@staticmethod
def restore_scene(scene_profile: ImportProfile, mozart_entities=None):
    """Восстанавливаем связи импорта и вывоза ссылок на сцене"""
    # Восстанавливаем связи между всеми узлами источника из profile.source_references
    for nitem in scene.nodes.values():
        if (nitem.node_type == NodeType.SOURCE_TABLE and
            nitem.get_attr('table_name') == source_table):
            refs = source_references.get('refs', {})
            if 'ref_table' in refs and 'ref_field' in refs:
                nitem.node.refs = refs
            break
    # Восстанавливаем связи импорта
    for link_data in profile.import_links:
        source_entity_id = link_data['source_entity_id']
        source_node = None
        target_node = None
        for nitem in scene.nodes.values():
            if (source_node == nitem and nitem.node_type == NodeType.SOURCE_TABLE and nitem.node.table_name == source_table:
                source_node = nitem
            elif nitem.node_type == NodeType.MOZART_ENTITY and nitem.node.entity_id == target_entity_id:
                target_node = nitem
            if not source_node or not target_node:
                continue
        mappings = link_data.get('mappings', [])
        # Удаляем старые линии между этими узлами
        for link in scene.links:
            if link.link_type == LinkType.IMPORT and link.start_item == source_node and link.end_item == target_node:
                scene.links.remove(link)
        # Создаём новую линию и принудительно записываем маппинги
        link = scene.add_link(source_node, target_node, LinkType.IMPORT, mappings=mappings)
        link.mappings = mappings
    else:
        link = scene.add_link(source_node, target_node, LinkType.IMPORT)
    scene.update_all_links()

```

Файл: /home/sergey/Documents/configurate/mozart_import/designer/source_ref_dialog.py (

```

# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import (QDialog, QVBoxLayout, QTableWidget, QTableWidgetItem,
                             QMessageBox, QComboBox, QDialogButtonBox, QHeaderView, QMessageBox)
from PyQt5.QtCore import Qt

class SourceRefDialog(QDialog):
    """Диалог настройки ссылок полей внутри таблицы источника,
    выбора целевой таблицы и поля, которое служит ключом в этой таблице.
    """
    def __init__(self, source_table_name: str, source_fields: list, all_source_table_names: list,
                 current_refs: dict = None, parent=None):
        self.source_table_name = source_table_name
        self.source_fields = source_fields
        self.all_source_table_names = all_source_table_names
        self.current_refs = current_refs or {}
        super().__init__(parent)
        self.setWindowTitle("Ссылочные поля таблицы «{source_table_name}»")
        self.resize(800, 400)
        self.source_fields = source_fields
        self.all_source_table_names = all_source_table_names
        self.result_refs = {}
        layout = QVBoxLayout(self)
        self.table = QTableWidget(len(source_fields), 3)
        self.table.setHorizontalHeaderLabels(["Ссылка?", "Целевая таблица", "поле"])
        self.table.verticalHeader().setVisible(False)
        for row, field_name in enumerate(source_fields):
            item = QTableWidgetItem(field_name)
            self.table.setItem(row, 0, item)
            # Чекбокс "Ссылка"
            chk = QtCheckBox()
            ref_info = self.current_refs.get(field_name, {})
            self.table.setCellWidget(row, 1, chk)
            # Комбобокс с целевой таблицей и полем
            for tbl in all_source_table_names:

```

```

# Добавляем поля этой таблицы (упрощённо: предполагаем, что ключевое поле называется 'id' или
# в реальном приложении нужно знать схему всех таблиц, но здесь используем заглушку: поле 'id'
# combo.addItems(['id', 'url']) # Пока предполагаем, что ключевое поле всегда 'id'
# get_selected_ref: функция уже установлена, выбираем соответствующий пункт
target = ref.info.get('ref_table')
field = ref.info.get('ref_field')
idx = 0
while 0 <= 10.findData((target, field)):
    combo.setCurrentIndex(idx)
self.table.setCellWidget(row, 2, combo)
layout.addWidget(self.table)
btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
btn_box.accepted.connect(self.save)
btn_box.rejected.connect(self.reject)
layout.addWidget(btn_box)
def save(self):
    self.result_refs = {}
    for row in range(self.table.rowCount()):
        field_name = self.table.cellWidget(row, 1).text()
        checked = self.table.cellWidget(row, 2).isChecked()
        target_table, target_field = combo.currentData()
        self.result_refs[field_name] = {
            'ref_table': target_table,
            'ref_field': target_field,
        }
    self.accept()
def get_refs(self):
    return self.result_refs

```

Файл: /home/sergey/Documents/configurate/mozart_import/dialogs/profile_manager_dialog

```

# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import (QDialog, QVBoxLayout, QHBoxLayout, QTableWidget,
                              QComboBox, QGroupBox, QLabel, QLineEdit, QFileDialog,
                              QDialogButtonBox)
from PyQt5.QtCore import Qt
from database import DatabaseService
from lang.local_translator import LocalTranslator
from core.profile_manager import ProfileManager
from designer.scene_profile_adapter import ProfileAdapter
from connectors.file_connector import FileConnector
from connectors.url_connector import UrlConnector
from core.models import SourceType, FileFormat

class ProfileManagerDialog(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.db = DatabaseService()
        self.translator = LocalTranslator()
        self.profmgr = ProfileManager(self.db)
        self.setWindowTitle(self.translator.tr('profile_manager_title'))
        self.resize(800, 500)
        layout = QVBoxLayout(self)
        self.table = QTableWidget(4)
        self.table.setHorizontalHeaderLabels([
            self.translator.tr('profile_id'),
            self.translator.tr('source_name'),
            self.translator.tr('created_at')
        ])
        self.table.horizontalHeader().setStretchLastSection(True)
        self.table.setSelectionMode(QTableWidget.SingleSelection)
        layout.addWidget(self.table)
        btn_layout = QHBoxLayout()
        btn_add = QPushButton(self.translator.tr('profile_add'))
        btn_edit = QPushButton(self.translator.tr('profile_edit'))
        btn_delete = QPushButton(self.translator.tr('profile_delete'))
        btn_close = QPushButton(self.translator.tr('close'))
        self.btn_add.clicked.connect(self.add_profile)
        self.btn_edit.clicked.connect(self.edit_profile)
        self.btn_delete.clicked.connect(self.delete_profile)
        self.btn_close.clicked.connect(self.accept)
        btn_layout.addWidget(self.btn_add)
        btn_layout.addWidget(self.btn_edit)
        btn_layout.addWidget(self.btn_delete)
        btn_layout.addWidget(self.btn_close)
        layout.addLayout(btn_layout)
        self.load_profiles()
    def load_profiles(self):
        self.table.setRowCount(len(self.profmgr.list_profiles()))
        for row, prof in enumerate(self.profmgr.list_profiles()):
            self.table.setItem(row, 0, QTableWidgetItem(str(prof.id)))
            self.table.setItem(row, 1, QTableWidgetItem(str(prof.name)))
            self.table.setItem(row, 2, QTableWidgetItem(str(prof.source_type.value if prof.source_type else '')))
            self.table.setItem(row, 3, QTableWidgetItem(str(prof.created_at) if hasattr(prof, 'created_at') else ''))
    def get_selected_profile_id(self):
        current = self.table.currentRow()
        if current < self.table.item(current, 0).data(Qt.UserRole):
            return None
        return self.table.item(current, 0).data(Qt.UserRole)
    def add_profile(self):
        self.run_import_designer()
    def edit_profile(self):
        profile_id = self.get_selected_profile_id()
        if not profile_id:
            QMessageBox.warning(self, self.translator.tr('warning'),
                                self.translator.tr('select_profile_first'))
            return
        profile = self.profmgr.load_profile(profile_id)
        if not profile:
            QMessageBox.critical(self, self.translator.tr('error'),
                                 self.translator.tr('profile_load_error'))
            return
        self.run_import_designer(existing_profile=profile)
    def delete_profile(self):
        profile_id = self.get_selected_profile_id()
        if not profile_id:
            QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('select_profile_first'))
        if QMessageBox.question(self, self.translator.tr('confirm_delete'),
                                self.translator.tr('delete_profile_confirm')) == QMessageBox.Yes:
            self.profmgr.delete_profile(profile_id)
            self.load_profiles()
    def run_import_designer(self, existing_profile=None):
        dlg = QDialog(self, self.translator.tr('select_source_dialog_title'))
        layout = QVBoxLayout(dlg)
        type_combo = QComboBox()
        type_combo.addItem(self.translator.tr('source_type_file'), SourceType.FILE)
        type_combo.addItem(self.translator.tr('source_type_url'), SourceType.URL)
        layout.addWidget(type_combo)
        # Группы для файлов
        file_layout = QVBoxLayout(self.translator.tr('file_group'))

```



```

format_combo = QComboBox()
for fmt in FileFormat:
    format_combo.addItem(fmt.value, fmt)
file_layout.addWidget(QLabel(self.translator.tr('file_format_label')))
path_edit = QLineEdit(format_combo)
browse_btn = QPushButton(self.translator.tr('browse_button'))
browse_btn.clicked.connect(lambda: browse_file())
path_layout.addWidget(path_edit)
path_layout.addWidget(browse_btn)
layout.addWidget(path_group)

# Группа для DB
db_layout = QVBoxLayout(self.translator.tr('db_group'))
db_type_combo = QComboBox()
db_type_combo.addItems(["postgres", "mysql", "mssql", "oracle", "sqlite"])
db_layout.addWidget(db_type_combo)
db_host_label = QLabel(self.translator.tr('db_host_label'))
db_port_label = QLabel(self.translator.tr('db_port_label'))
db_name_label = QLabel(self.translator.tr('db_name_label'))
db_user_label = QLabel(self.translator.tr('db_user_label'))
db_pass_label = QLabel(self.translator.tr('db_pass_label'))
db_pass.setEchoMode(QLineEdit.Password)
db_layout.addWidget(db_host_label)
db_layout.addWidget(db_port_label)
db_layout.addWidget(db_name_label)
db_layout.addWidget(db_user_label)
db_layout.addWidget(db_pass_label)
layout.addWidget(db_group)

# Группа для URL
url_layout = QVBoxLayout(self.translator.tr('url_group'))
url_edit = QLineEdit()
method_combo = QComboBox()
method_combo.addItems(["GET", "POST"])
url_layout.addWidget(QLabel(self.translator.tr('url_label')))
url_layout.addWidget(url_edit)
url_layout.addWidget(QLabel(self.translator.tr('http_method_label')))
layout.addWidget(url_group)

def toggle_groups():
    if type_combo.currentData() == SourceType.FILE:
        db_group.setVisible(False)
        url_group.setVisible(True)
    elif type_combo.currentData() == SourceType.DATABASE:
        db_group.setVisible(True)
        url_group.setVisible(False)

def toggle_db_fields():
    if type_combo.currentText() == 'sqlite':
        db_host_label.setVisible(False)
        db_port_label.setVisible(False)
        db_name_label.setVisible(True)
        db_user_label.setVisible(True)
        db_pass_label.setVisible(True)
    else:
        db_host_label.setVisible(True)
        db_port_label.setVisible(True)
        db_name_label.setVisible(False)
        db_user_label.setVisible(False)
        db_pass_label.setVisible(False)

def browse_db_file():
    path = QFileDialog.getOpenFileName(dlg, "Выберите файл SQLite")
    if path:
        self.db_name.setText(path)

def browse_file():
    path = QFileDialog.getOpenFileName(dlg, self.translator.tr('select_file_dialog'))
    if path:
        path_edit.setText(path)

type_combo.currentIndexChanged.connect(lambda: toggle_groups())
toggle_groups()

# Заполнение полей из профиля
profile = existing_profile
if type_combo.findData(profile.source_type):
    idx = type_combo.setCurrentIndex(idx)
    if profile.source_type == SourceType.FILE:
        if profile.source_path:
            fmt_idx = format_combo.findData(FileFormat(profile.file_format))
            if fmt_idx:
                format_combo.setCurrentIndex(fmt_idx)
        elif profile.source_type == SourceType.DATABASE:
            if profile.db_type:
                db_idx = db_type_combo.findText(profile.db_type)
                if db_idx:
                    db_type_combo.setCurrentIndex(db_idx)
            if profile.db_host:
                self.db_host.setText(profile.db_host)
            if profile.db_port:
                self.db_port.setText(profile.db_port)
            if profile.db_name:
                self.db_name.setText(profile.db_name)
            if profile.db_user:
                self.db_user.setText(profile.db_user)
            if profile.db_pass:
                self.db_pass.setText(profile.db_pass)
        elif profile.source_path:
            url_edit.setText(profile.source_path)
btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
btn_box.clicked.connect(dlg.reject)
layout.addWidget(btn_box)
if dlg.exec_() != QDialog.Accepted:
    return

try:
    if str == type_combo.currentData():
        connector = FileConnector(path_edit.text(), format_combo.currentData())
    elif str == SourceType.DATABASE:
        params = {
            'db_type': db_type_combo.currentText(),
            'host': self.db_host.text() or None,
            'port': self.db_port.text() or None,
            'db_name': self.db_name.text(),
            'user': self.db_user.text(),
            'password': self.db_pass.text()
        }
        connector = DBConnector("", **params)
    else:
        connector = URLConnector(url_edit.text())
except ImportError:
    QMessageBox.critical(self, self.translator.tr('error'), str(e))
return
designer = ImportDesignerV2(self.db, connector, self)
if existing_profile:

```

```

        designer.editing_profile_id = existing_profile_id
        # Вот так устанавливаем связи после загрузки сцены
        designer.load_profile_links(existing_profile)
        self.load_profiles()

```

Файл: /home/sergey/Documents/configurate/mozart_import/graph/__init__.py (относительный)

```

# -*- coding: utf-8 -*-

```

Файл: /home/sergey/Documents/configurate/mozart_import/graph/graphics_items.py (относительный)

```

# mozart_import/graph/graphics_items.py
from PyQt5.QtWidgets import (QGraphicsRectItem, QGraphicsTextItem,
from PyQt5.QtCore import QPointF, QRect, QCheckBox, QGraphicsItem,
from PyQt5.QtGui import QBrush, QColor, QPen, QFont
from ..core.models import Field, NodeType
SOURCE_ID_FIELD = 'sourceid'

class FieldItem(QGraphicsRectItem):
    """Поле, вступит ли в реакцию, отенника таблицы/сущности. Может иметь чекбокс (для источника)"""
    def __init__(self, field: Field, show_checkbox: bool = False, is_sourceid: bool = False, parent=None):
        super().__init__(parent)
        self.field = field
        self.show_checkbox = show_checkbox
        self.is_sourceid = is_sourceid
        self.setRect(0, 0, width, 22)
        if is_sourceid:
            self.setPen(QPen(QBrush(QColor(200, 200, 200))))
        else:
            self.setPen(QPen(QBrush(QColor(240, 240, 240))))
        self.setFlag(QGraphicsItem.ItemIsSelectable)
        self.setAcceptHoverEvents(True)
        # offset = 2
        if show_checkbox and not is_sourceid:
            self.checkbox = QCheckBox(True)
            self.proxy = QGraphicsProxyWidget(self)
            self.proxy.setWidget(self.checkbox)
            text_x = 25
        else:
            self.checkbox = None
            text_x = 5 if not is_sourceid else 25
        label = f"□ {field.name}" if is_sourceid else field.name
        self.text = QGraphicsTextItem(label, self)
        self.text.setDefaultTextColor(Qt.black)
        self.text.setPos(text_x, y_offset)

    def hoverEnterEvent(self, event):
        if not self.is_sourceid:
            super().hoverEnterEvent(event)
        self.setBrush(QBrush(QColor(200, 230, 255)))

    def hoverLeaveEvent(self, event):
        if not self.is_sourceid:
            super().hoverLeaveEvent(event)
        self.setBrush(QBrush(QColor(240, 240, 240)))

    def is_checked(self) -> bool:
        if self.checkbox:
            return self.checkbox.isChecked()
        return False

    def set_checked(self, state: bool):
        if self.checkbox:
            self.checkbox.setChecked(state)

    def boundingRect(self):
        return self.rect()

class NodeItem(QGraphicsRectItem):
    """Прямоугольник, таблицы/сущности с чекбоксом в заголовке (для выбора таблицы) и чекбоксами полей (для источника)"""
    def __init__(self, node, parent=None):
        super().__init__(parent)
        self.node = node
        self.field_items = []
        self.setFlag(QGraphicsItem.ItemIsMovable)
        self.setFlag(QGraphicsItem.ItemIsSelectable)
        self.setFlag(QGraphicsItem.ItemSendsGeometryChanges)
        width = 240
        height = 10 + len(node.fields) * 20
        self.setRect(0, 0, width, height)
        if node.node_type == NodeType.SOURCE_TABLE:
            self.color = QColor(220, 255, 220) # светло-зелёный
        else:
            self.color = QColor(255, 255, 200) # светло-жёлтый
        self.setPen(QPen(Qt.black, 2))
        # Чекбокс заголовка (выбор всей таблицы)
        self.checkbox = QCheckBox(False)
        self.proxy = QGraphicsProxyWidget(self)
        self.proxy.setWidget(self.checkbox)
        self.proxy.setPos(5, 5)
        self.title = QGraphicsTextItem(node.name, self)
        self.title.setDefaultColor(Qt.darkblue)
        self.title.setPos(25, 5)
        # Поля
        self.show_checkbox = (node.node_type == NodeType.SOURCE_TABLE)
        for field in node.fields:
            if field.name == SOURCE_ID_FIELD and (node.node_type == NodeType.MOZART_ENTITY):
                fi = FieldItem(field, show_checkbox=show_checkbox, is_sourceid=is_sid, parent=self)
                self.field_items.append(fi)
                y += 20
            else:
                fi = FieldItem(field, show_checkbox=show_checkbox, is_sourceid=False, parent=self)
                self.field_items.append(fi)
                y += 20
        self.setPos(node.x, node.y)

    def itemChange(self, change, value):
        if change == QGraphicsItem.ItemPositionChange:
            new_pos = self.max(0, new_pos.y())
            if self.scene() and hasattr(self.scene(), 'divider_x'):
                divider_x = self.scene().divider_x
                if self.node.node_type == NodeType.SOURCE_TABLE:
                    new_pos.setX(max(10, min(new_pos.x(), max_x)))
                else:
                    new_pos.setX(divider_x + 10)
            return new_pos
        return super().itemChange(change, value)

    def mouseReleaseEvent(self, event):
        super().mouseReleaseEvent(event)
        # Обновляем все линии сцены
        self.scene().update_all_links()

```



```

def show_link_context_menu(self, link_line, screen_pos):
    menu = QMenu()
    act_edit = QAction("Настроить связь", None)
    act_add_triggered.connect(lambda: self._emit_edit_link(link_line))
    act_del = QAction("Удалить связь", None)
    act_del_triggered.connect(lambda: self.remove_link(link_line))
    menu.addAction(act_edit)
    menu.addAction(act_del)
    menu.exec_(screen_pos)

def show_scene_context_menu(self, screen_pos):
    selected_mozart = self._get_single_checked(NodeType.MOZART_ENTITY)
    menu = QMenu()
    if selected_source and selected_mozart:
        act_link_triggered.connect(lambda: self._link_selected(selected_source, selected_mozart))
        menu.addAction(act_link)
    else:
        hint = "Отметьте чекбоксами по одной таблице источника и сущности Mozart"
        menu.addAction(hint)
    menu.addSeparator()
    act_refresh_triggered.connect(lambda: self.update_all_links())
    menu.addAction(act_refresh)
    menu.exec_(screen_pos)

def _get_single_checked(self, node_type):
    checked = self.nodes.values()
    for ni, node in checked:
        if ni.node_type == node_type and ni.is_checked():
            return checked[0] if len(checked) == 1 else None
    return checked[0] if len(checked) == 1 else None

def _link_selected(self, source_node_item, mozart_node_item):
    checked_source_fields = self._get_checked_fields(source_node_item)
    checked_source_fields = [fi.field.name for fi in source_node_item.field_items]
    target_fields = [fi.field.name for fi in mozart_node_item.field_items]
    target_fields.append(SOURCE_ID_FIELD)
    existing_mappings = []
    for link in self.links:
        if link.start_item == source_node_item and link.end_item == mozart_node_item:
            existing_mappings = link.mappings
            break
    if not existing_mappings:
        for d in self.get_import_links_data():
            if d['target_entity_id'] == mozart_node_item.node.entity_id and d['source_entity_id'] == source_node_item.node.entity_id:
                existing_mappings = d.get('mappings', [])
                break
    if self.on_edit_link:
        source_refs = getattr(source_node_item.node, 'refs', {})
        self.on_edit_link(checked_source_fields, mozart_node_item, existing_mappings)
    else:
        QMessageBox.information(None, "Информация", "Обработчик настройки связи не назначен")

def _show_import_links_for(self, mozart_node_item):
    entity_name = mozart_node_item.node.name
    relevant_links = []
    for link in self.links:
        if link.start_item == mozart_node_item and link.end_item == source_node_item:
            relevant_links.append(link)
    if not relevant_links:
        QMessageBox.information(None, f"Связи для {entity_name}", "Нет назначенных импортируемых связей.")
    return relevant_links

def _show_import_dialog(self, entity_name):
    dlg = QDialog()
    title = f"Связи импорта: {entity_name}"
    table = QTableWidget(0, 3)
    table.setHorizontalHeaderLabels(["Источник (таблица.поле)", "Поле Mozart", "Ключевое", "Ссылка"])
    table.horizontalHeader().setStretchLastSection(True)
    for link in relevant_links:
        for table_row in link.mappings:
            row = table.rowCount()
            table.insertRow(row)
            table.setItem(row, 0, QTableWidgetItem(f"{src_table} {m['source_field']}"))
            table.setItem(row, 1, QTableWidgetItem(f"{target_table} {m['target_field']}"))
            table.setItem(row, 2, QTableWidgetItem(f"{m['key']}"))
            table.setItem(row, 3, QTableWidgetItem(f"{m['ref']}"))
            if m.get('ref') == 'reference':
                ref_info = f"{m.get('ref_source_table', '')} {m.get('ref_source_key_field', '')}"
            else:
                ref_info = ""
            table.setItem(row, 4, QTableWidgetItem(ref_info))
    layout.addWidget(table)
    dlg.exec_()

def update_all_links(self):
    for link in self.links:
        link.update_position()

def get_import_links_data(self):
    for link in self.links:
        if link.start_item == LinkType.IMPORT:
            data.append({
                'source_table': link.start_item.node.table_name,
                'target_table': link.end_item.node.table_name,
                'mappings': link.mappings
            })
    return data

# эмиттеры
def _emit_preview(self, node_item):
    self.on_preview_table(node_item)

def _emit_create_entity(self, node_item):
    self.on_create_entity(node_item)

def _emit_create_fields(self, node_item):
    self.on_create_fields_from_source(node_item)

def _emit_edit_link(self, link_line):
    checked_node_item = link_line.start_item
    checked_fields = checked_node_item.get_checked_fields()
    if not checked_fields:
        checked_fields = [fi.field.name for fi in source_node_item.field_items]
    self.on_edit_link(link_line.start_item, link_line.end_item, link_line.mappings, checked_fields, source_refs)

def _emit_set_source_refs(self, node_item):
    self.on_set_source_refs(node_item)

```

Файл: /home/sergey/Documents/configurate/mozart_import/matchers/__init__.py (относите

```

# mozart_import/matchers/__init__.py
from typing import Match
from .constraint_matcher import ConstraintMatcher

__all__ = (
    'NameMatcher',
    'ConstraintMatcher'
)

```


1

Файл: /home/sergey/Documents/configurate/mozart_import/matchers/constraint_matcher.py

```
# mozart_import/matchers/constraint_matcher.py
from typing import List, Dict, Any

class ConstraintMatcher:
    """Сопоставление по ограничениям (первичные ключи, внешние ключи, уникальность)."""
    @staticmethod
    def is_primary_key_candidate(field_name: str) -> bool:
        """Проверяет, похоже ли имя поля на первичный ключ."""
        return name_lower in ('id', 'code', 'num', 'key', 'pk')
    @staticmethod
    def is_foreign_key_candidate(field_name: str) -> bool:
        """Проверяет, похоже ли имя поля на внешний ключ."""
        name_lower = field_name.lower()
        return name_lower.endswith('_id') or name_lower.startswith('id_') or name_lower in ('parentid', 'parent_id')
    @staticmethod
    def extract_referenced_table(field_name: str, current_table: str) -> str:
        """По имени внешнего ключа предполагает имя целевой таблицы."""
        name_lower = field_name.lower()
        if name_lower.endswith('_id'):
            target = name_lower[:-3]
        elif name_lower.startswith('id_'):
            target = name_lower[3:]
        else:
            target = name_lower
        if target == 'parent':
            return target
        return target
```

Файл: /home/sergey/Documents/configurate/mozart_import/matchers/name_matcher.py (от

```
# mozart_import/matchers/name_matcher.py
from typing import Tuple, Optional

class NameMatcher:
    """Сопоставление по имени (таблиц, полей)."""
    @staticmethod
    def match(a: str, b: str) -> float:
        return SequenceMatcher(None, a.lower(), b.lower()).ratio()
    @staticmethod
    def best_match(name: str, candidates: list, threshold: float = 0.6) -> Tuple[Optional[str], float]:
        best_score = 0.0
        for cand in candidates:
            score = NameMatcher.similarity(name, cand)
            if score > best_score:
                best_score = score
        if best_score > threshold:
            return name, best_score
        return None, 0.0
```

Файл: /home/sergey/Documents/configurate/mozart_import/matchers/type_matcher.py (от

```
# mozart_import/matchers/type_matcher.py
from ..core.models import FieldType

class TypeMatcher:
    """Сопоставление типов данных источника и Mozart."""
    # Мappings типов источника -> коды типов Mozart (C, I, N, D, T, L, M, R)
    type_mapping = {
        'id': 'C',
        'code': 'I',
        'num': 'N',
        'key': 'D',
        'parentid': 'T',
        'parent_id': 'L',
        'date': 'M',
        'time': 'R',
        'binary': 'F',
    }
    @classmethod
    def get_mozart_type(cls, source_type: FieldType) -> str:
        return cls.type_mapping.get(source_type, 'C')
    @classmethod
    def is_compatible(cls, source_type: FieldType, mozart_type: str) -> bool:
        target_type = cls.get_mozart_type(source_type)
        if target_type == mozart_type:
            return True
        # source_type == FieldType.INTEGER and mozart_type == 'N':
        if source_type == FieldType.FLOAT and mozart_type == 'N':
            return True
        if source_type == FieldType.STRING and mozart_type == 'C':
            return True
        return False
```

Файл: /home/sergey/Documents/configurate/mozart_import/utils/_init_.py (относительно

```
# mozart_import/utils/_init_.py
from ..ext_helper import ExtHelper
from ..log_manager import LogManager
__all__ = ['ExtHelper', 'LogManager']
```

Файл: /home/sergey/Documents/configurate/mozart_import/utils/ext_helper.py (относительн

```
# mozart_import/utils/ext_helper.py
from database import DatabaseService

class ExtHelper:
    """Вспомогательный класс для работы с ext-таблицами и полями."""
    def __init__(self, db: DatabaseService):
        self.db = db
    def get_ext_table_name(self, entity_alias: str) -> str:
        return f'ext_{entity_alias.lower()}'
    def ensure_ext_table(self, entity_id: int) -> bool:
        """Проверяет, существует ли таблица ext. Если нет, создаёт при необходимости."""
        alias = self.db.execute_query("SELECT alias FROM meta.entitytypes WHERE id=%s", (entity_id,))
        if not alias:
            return False
        ext_table = self.get_ext_table_name(alias)
        check = self.db.execute_query("SELECT to_regclass(%s)", (ext_table,))
        if check and check[0][0]:
            return True
        sql = f"CREATE TABLE IF NOT EXISTS {ext_table} ("
        sql += f"    versionid INTEGER NOT NULL REFERENCES data.entity_versions(id) ON DELETE CASCADE"
        sql += ")"
        self.db.execute_query(sql, fetch=False)
        return True
```

```

def ensure_field(self, entity_id: int, field_name: str, field_type: str = 'C') -> int:
    """
    calias: str = None, ref_entity_id: int = None, length: int = 255) -> int:
    Создаёт поле в meta.fields (если его нет) и вызывает p_field_sync.
    Возвращает id поля.
    if calias is None:
        # Проверка, существует ли поле
        existing = self.db.execute_query(
            'SELECT id FROM meta.fields WHERE entitytypeid=%s AND cfieldname=%s',
            (entity_id, field_name))
        if existing:
            return existing[0][0]
        res = self.db.execute_query(
            'INSERT INTO meta.fields
            (entitytypeid, cfieldname, cfieldtype, calias, nlength, lisindexed, ref_entitytypeid, status_id)
            VALUES (%s, %s, %s, %s, %s, %s, %s, %s, 1)
            (entity_id, field_name, field_type, calias, length, True, ref_entity_id)
        )
        field_id = res[0][0]
        return field_id
    return self._procedure("meta.p_field_sync", (field_id,))

```

Файл: /home/sergey/Documents/configurate/mozart_import/utils/field_creator.py (относительно mozart_import/)

```

# mozart_import/utils/field_creator.py
# coding: utf-8
from database import DatabaseService

class FieldCreator:
    def __init__(self, db: DatabaseService):
        self.db = db
    def create_fields(self, entity_id: int, fields: list) -> list:
        created_ids = []
        for f in fields:
            exists = self.db.execute_query(
                'SELECT id FROM meta.fields WHERE entitytypeid=%s AND cfieldname=%s',
                (entity_id, f['cfieldname']))
            if exists:
                created_ids.append(exists[0][0])
                continue
            cfieldname = f['cfieldname'][:63]
            cfieldtype = f.get('cfieldtype', 'C')
            nlength = f.get('nlength', 0)
            lisindexed = f.get('lisindexed', True)
            ref_entitytypeid = f.get('ref_entitytypeid', None)
            res = self.db.execute_query(
                'INSERT INTO meta.fields
                (entitytypeid, cfieldname, cfieldtype, calias, nlength, lisindexed, ref_entitytypeid, status_id)
                VALUES (%s, %s, %s, %s, %s, %s, %s, %s, 1)
                (entity_id, cfieldname, cfieldtype, calias,
                nlength, nlength, lisindexed, ref_entitytypeid)
            )
            if res:
                field_id = res[0][0]
                self._procedure("meta.p_field_sync", (field_id,))
            else:
                created_ids.append(field_id)
        return created_ids

```

Файл: /home/sergey/Documents/configurate/mozart_import/utils/log_manager.py (относительно mozart_import/)

```

# mozart_import/utils/log_manager.py
import logging
from datetime import datetime

class LogManager:
    """
    Простой логгер для импорта.
    """
    def __init__(self, name="mozart_import"):
        self.logger = logging.getLogger(name)
        if not logger.handlers:
            handler = logging.StreamHandler()
            formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')
            handler.setFormatter(formatter)
            self.logger.addHandler(handler)
    def info(self, msg):
        self.logger.info(msg)
    def error(self, msg):
        self.logger.error(msg)
    def warning(self, msg):
        self.logger.warning(msg)
    def debug(self, msg):
        self.logger.debug(msg)

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/_init_.py (относительно mozart_import/wizards/)

```

# mozart_import/wizards/_init_.py
# coding: utf-8
from .source_page import SourcePage
from .schema_page import SchemaPage
from .relation_page import RelationPage
from .execute_page import ExecutePage
all = [
    'SourcePage',
    'SchemaPage',
    'RelationPage',
    'ExecutePage',
]

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/execute_page.py (относительно mozart_import/wizards/)

```

# mozart_import/wizards/execute_page.py
# coding: utf-8
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QProgressBar, QTextEdit, QPushButton,
from PyQt5.QtCore import Qt, QThread, pyqtSignal
from ..core.translator import LocalTranslator
from ..core.profile_manager import ProfileType, MappingProposal, MatchedEntity, MatchedField
from ..core.field_creator import FieldCreator

class ImportThread(QThread):
    progress = pyqtSignal(int)
    error = pyqtSignal(str)
    def __init__(self, db, connector, profile):

```

```

super().__init__()
self.db = db
self.connector = connector
self.profile = profile
def run(self):
    try:
        engine = ImportEngine(self.db, self.connector, self.profile)
        self.finished.emit(stats)
        self.progress_callback = self.progress.emit()
    except Exception as e:
        self.error.emit(str(e))
class ExecutePage(QWizardPage):
    def __init__(self, wizard):
        super().__init__(wizard)
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('execute_page_title'))
        self.setSubTitle(self.translator.tr('execute_page_subtitle'))
        layout = QVBoxLayout(self)
        self.progress = QProgressBar()
        layout.addWidget(self.progress)
        self.log = QTextEdit()
        layout.addWidget(self.log)
        btn_layout = QHBoxLayout()
        self.btn_start.clicked.connect(self.start_import)
        self.btn_start.setEnabled(False)
        layout.addWidget(self.btn_start)
        layout.addLayout(btn_layout)
        self.import_thread = None
    def retranslate_ui(self):
        self.setWindowTitle(self.translator.tr('execute_page_title'))
        self.setSubTitle(self.translator.tr('execute_page_subtitle'))
        self.btn_start.setText(self.translator.tr('start_import_btn'))
    def initializePage(self):
        self.prepare_profile()
        self.btn_start.setEnabled(True)
    def prepare_profile(self):
        # Создаем недостающие сущности и поля
        # for node in self.wizard.pending_entities and self.wizard.pending_entities:
        #     for node in self.wizard.pending_entities:
        """INSERT INTO meta.entity_types (name, alias, class_id, superclass, isversioned, issecuritystrategy, status_id)
        (entity_data['name'], entity_data['alias'], entity_data['class_id'], None, None, None, None)
        """
        if res:
            entity_id = res[0][0]
            node_item.node.entity_id = entity_id
        # Строим MappingProposal
        source_node = self.id: n for n in getattr(self.wizard, 'source_nodes', [])}
        if hasattr(self.wizard, 'mozart_nodes'):
            for target_node in self.wizard.mozart_nodes:
                if target_node in self.wizard.pending_entities:
                    for node_item in self.wizard.pending_entities:
                        target_node.node_item.node_id = node_item.node_id
        links = getattr(self.wizard, 'links', [])
        for link in links:
            if target_id not in target_groups:
                target_groups[target_id].append(link)
        entities = []
        for target_id, link_list in target_groups.items():
            if not target_node:
                continue
            source_table_name = link_list[0]['source_node_id'].replace('src_', '')
            for link in link_list:
                if link.get('target_field'):
                    if link.get('target_field') == 'id':
                        matched_fields.append(MatchedField(
                            source_field=link['source_field'],
                            target_field=link['target_field'],
                            reference=1,
                            ref_entity_type_id=ref_entity_id
                        ))
                    entities.append(MatchedEntity(
                        source_table_name=source_table_name,
                        target_entity_id=target_node.entity_id,
                        field_id=link['target_field'],
                        is_new=(target_node.entity_id is None)
                    ))
        self.wizard.mapping_proposal = MappingProposal(
            source_schema=self.wizard.schema,
            entities=entities,
            suggestions_for_new_entities=[]
        )
        profile = ImportProfile(
            name=self.wizard.connector.source_path,
            source_type=self.wizard.connector.source_type,
            source_path=self.wizard.connector.source_path,
            scene_json=getattr(self.wizard, 'mapping_scene_json', None)
        )
        self.wizard.profile = profile
        prof_mgr.save_profile(profile)
    def start_import(self):
        if QMessageBox.Warning(self, self.translator.tr('warning'), self.translator.tr('profile_not_ready')):
            return
        self.btn_start.setEnabled(False)
        self.import_thread = ImportThread(self.wizard.db, self.wizard.connector, self.wizard.profile)
        self.import_thread.finished.connect(self.on_import_finished)
        self.import_thread.start()
    def update_progress(self, current, total):
        self.progress.setMaximum(total)
        self.log.append(f'{self.translator.tr('processed_records')}: {current} / {total}'))
    def on_finished(self, stats):
        self.translator.tr('import_finished'))
        self.translator.tr('inserted')): {stats['inserted']}
        QMessageBox.Information(self, self.translator.tr('import_finished_title'), self.translator.tr('import_finished'))
        self.btn_start.setEnabled(False)
    def on_error(self, msg):
        self.log.append(f'{self.translator.tr('error')}: {msg}')
        QMessageBox.Critical(self, self.translator.tr('error'), msg)
        self.btn_start.setEnabled(True)

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/import_wizard.py (открыто)

mozart_import/wizards/import_wizard.py

coding: utf-8

```

from PyQt5.QtWidgets import QWizard
from PyQt5.QtCore import Qt, QtEvent
and local_translator import LocalTranslator
database import DatabaseService
schema_page import SchemaPage
relation_page import RelationPage
mapping_detail_page import MappingDetailPage
from .execute_page import ExecutePage

class ImportWizard(QWizard):
    def __init__(self, parent=None):
        self.db = db = DatabaseService
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('wizard_title'))
        self.setWizardStyle(QWizard.ModernStyle)
        # Данные, передаваемые между страницами
        self.schema = None
        self.source_nodes = []
        self.pending_entities = []
        self.import_links = []
        # Страницы
        self.source_page = SourcePage(self)
        self.schema_page = SchemaPage(self)
        self.relation_page = MappingDetailPage(self)
        self.execute_page = ExecutePage(self)
        self.addWidget(self.source_page)
        self.addWidget(self.schema_page)
        self.addWidget(self.relation_page)
        self.addWidget(self.execute_page)
        self.setButtonText(QWizard.FinishButton, self.translator.tr('wizard_finish'))
        self.setButtonText(QWizard.CancelButton, self.translator.tr('btn_cancel'))
    def retranslate_ui(self):
        self.setWindowTitle(self.translator.tr('wizard_title'))
        self.setButtonText(QWizard.FinishButton, self.translator.tr('wizard_finish'))
        self.source_page.retranslate_ui()
        self.schema_page.retranslate_ui()
        self.relation_page.retranslate_ui()
        self.execute_page.retranslate_ui()
    def changeEvent(self, event):
        if event.type() == QtEvent.LanguageChange:
            super().changeEvent(event)

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/mapping_detail_page.py

```

# mozart_import/wizards/mapping_detail_page.py
# coding: utf-8
from PyQt5.QtWidgets import (QWizardPage, QVBoxLayout, QHBoxLayout, QTableWidget,
                             QDialogButtonBox, QPushButton, QMessageBox,
                             QComboBox, QCheckBox, QSpinBox)
from PyQt5.QtCore import Qt
from .local_translator import LocalTranslator

class MappingDetailPage(QWizardPage):
    def __init__(self, wizard):
        self.wizard = wizard
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('detail_page_title'))
        self.setSubTitle(self.translator.tr('detail_page_subtitle'))
        layout = QVBoxLayout(self)
        self.table = QTableWidget(0, 8)
        self.table.setHorizontalHeaderLabels([
            self.translator.tr('source_table'),
            self.translator.tr('target_table'),
            self.translator.tr('target_field'),
            self.translator.tr('mozart_type'),
            self.translator.tr('source_id'),
            self.translator.tr('action')
        ])
        self.table.horizontalHeader().setStretchLastSection(True)
        layout.addWidget(self.table)
        btn_layout = QHBoxLayout()
        self.btn_apply = QPushButton(self.translator.tr('apply_mapping_btn'))
        self.btn_back = QPushButton(self.translator.tr('back_btn'))
        self.btn_back.clicked.connect(lambda: self.wizard.back())
        btn_layout.addWidget(self.btn_apply)
        btn_layout.addWidget(self.btn_back)
        layout.addLayout(btn_layout)
    def retranslate_ui(self):
        self.setWindowTitle(self.translator.tr('detail_page_title'))
        self.setSubTitle(self.translator.tr('detail_page_subtitle'))
        self.table.setHorizontalHeaderLabels([
            self.translator.tr('source_table'),
            self.translator.tr('target_table'),
            self.translator.tr('target_field'),
            self.translator.tr('mozart_type'),
            self.translator.tr('source_id'),
            self.translator.tr('action')
        ])
        self.btn_apply.setText(self.translator.tr('apply_mapping_btn'))
        self.btn_back.setText(self.translator.tr('back_btn'))
    def initializePage(self):
        self.load_mappings()
    def load_mappings(self):
        source_nodes = self.wizard.getattr('links', [1])
        target_nodes = [node.id for node in getattr(self.wizard, 'source_nodes', [])}
        if hasattr(self.wizard, 'pending_entities'):
            for target_node in self.wizard.pending_entities:
                target_nodes[target_node.id] = target_node
        self.table.setRowCount(len(links))
        for source_node in source_nodes:
            target_node = target_nodes.get(link['source_node_id'])
            self.table.setItem(row, 0, QTableWidgetItem(source_node.name if source_node else ''))
            self.table.setItem(row, 2, QTableWidgetItem(target_node.name if target_node else ''))
            combo_target = QComboBox()
            self.translator.tr('ignore_field') + " --", ""))
            if target_node and target_node.entity_type:
                fields = self.wizard.execute_query(
                    (target_node.entity_id,
                     for name, alias in fields:
                         for name, alias in fields:
                             f" {alias} ({fname})" fname)
                )
            combo_target.addItem(f" {self.translator.tr('create_new_field')} + " ++", "_NEW_")
            idx = combo_target.findData(link['target_field'])
            if idx >= 0:
                combo_target.setCurrentIndex(idx)
        combo_target.connect(lambda idx, r=row, cb=combo_target:
            self.table.setCellWidget(row, 3, combo_target)

```



```

        QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('connector_not_ready'))
    return self.clear_scene()
    if not self.wizard.connector.get_schema():
        QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('no_tables_found'))
    return
    x = 50
    for table in schema.tables:
        fields = table.fields:
        for f in table.fields:
            if f.field_type.name == 'STRING':
                dt = DataType.STRING
            elif f.field_type.name == 'INTEGER':
                dt = DataType.INTEGER
            elif f.field_type.name == 'FLOAT':
                dt = DataType.FLOAT
            elif f.field_type.name == 'BOOLEAN':
                dt = DataType.BOOLEAN
            elif f.field_type.name == 'DATE':
                dt = DataType.DATE
            elif f.field_type.name == 'DATETIME':
                dt = DataType.DATETIME
            fields.append(f)
            data_type = dt
            is_primary_key = f.is_primary_key
            references = f.references
        node = Node(
            name=table.name,
            node_type=NodeType.SOURCE_TABLE,
            fields=fields,
        )
        table_name = table.name
        self.scene.add_node(node)
        self.mozart.nodes.append(node)
        if x > 500:
            x = 50
        y = 250
    # Автоматические связи FK
    for node in self.source_nodes:
        for field in node.fields:
            if field.is_primary_key and field.references:
                target_table_name = field.references
                if target_table_name == target_table_name:
                    for f in target_table.fields:
                        if f.is_primary_key:
                            target_field = f
                            if target_field:
                                start_fi = self.find_field_item(node, field.name)
                                end_fi = self.find_field_item(target_node, target_field.name)
                                self.scene.add_link(start_fi, end_fi, LinkType.FK_SOURCE)
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('source_loaded').format(len(schema.tables)))
    self.translator.tr('source_loaded').format(len(schema.tables)))
    def find_field_item(self, node, field_name):
        for f in node.fields:
            if f.name == field_name:
                return f
        return None
    def load_existing_entities(self):
        QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('db_not_ready'))
        return
        rows = self.wizard.db.execute_query("SELECT id, cname, alias, class_id FROM meta.entitytypes ORDER BY
        cname")
        y = 600
        for id, cname, alias, class_id in rows:
            class_res = self.wizard.db.execute_query("SELECT alias FROM meta.entity_classes WHERE id=%s", (class_id,))
            class_alias = class_res[0][0] if class_res else None
            frows = self.wizard.db.execute_query("SELECT cfieldname, cfieldtype FROM meta.fields WHERE
            entitytype=%s", (id,))
            for f in frows:
                dt = DataType.STRING
                if f.field_type.name == 'INTEGER':
                    dt = DataType.INTEGER
                elif f.field_type.name == 'FLOAT':
                    dt = DataType.FLOAT
                elif f.field_type.name == 'BOOLEAN':
                    dt = DataType.BOOLEAN
                elif f.field_type.name == 'DATE':
                    dt = DataType.DATE
                elif f.field_type.name == 'DATETIME':
                    dt = DataType.DATETIME
                fields.append(f)
                data_type = dt
            node = Node(
                name=id,
                fields=fields,
                node_type=NodeType.MOZART_ENTITY,
            )
            entity_id = id
            class_alias = class_alias
            self.scene.add_node(node)
            self.mozart.nodes.append(node)
            if x > 1200:
                x = 250
            y = 250
    # Связи композиции
    comp = self.wizard.db.execute_query("SELECT parent_entityid, child_entityid, c_link_fieldname FROM
    meta.composition")
    for parent_id, child_id, link_field in comp:
        parent_node = None
        child_node = None
        for n in self.mozart.nodes:
            if n.entity_id == parent_id:
                parent_node = n
            if n.entity_id == child_id:
                child_node = n
        if parent_node and child_node:
            # Имя поля в дочерней сущности с именем link_field
            for f in child_node.fields:
                if f.name == link_field:
                    child_fi = f
                    if child_fi:
                        # Для родителя используем первое поле (или можно создать линию без привязки к полю)
                        parent_fi = self.find_field_item(parent_node, parent_node.fields[0].name)
                        and_fi = self.find_field_item(child_node, child_fi.name)
                        self.scene.add_link(parent_fi, and_fi, LinkType.COMPOSITION)
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('mozart_loaded').format(len(rows)))
    def create_entity(self):
        if not self.wizard.db:
            QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('db_not_ready'))
            return
        data = self.wizard.db.get_data()
        node = Node(
            name=data['alias'],
            fields=[],
            node_type=NodeType.MOZART_ENTITY,
        )
        class_alias = "DICT"
        self.scene.add_node(node)
        self.mozart.nodes.append(node)
        self.wizard.pending_entities.append((node, data))
        QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('entity_added_to_scene'))
    def save_mapping(self):

```

```

nodes_pos = {}
for node_item in self.scene.nodes.values():
    nodes_pos[node_item.node_id] = {'x': node_item.node.x, 'y': node_item.node.y}
links_data = {}
for link_line in self.scene.links:
    source_node_id = link_line.start_item.node_id
    target_node_id = link_line.end_item.node_id
    link_type = link_line.link_type.value
    is_source_id = getattr(link_line, 'is_source_id', False)
    links_data.append(link_line)
scene_data = {'nodes': nodes_pos,
              'links': links_data}
self.wizard.import_scene_json = json.dumps(scene_data, ensure_ascii=False)
self.wizard.mozart_nodes = self.mozart_nodes
self.wizard.links_data = links_data
self.wizard.next()

def preview_table(self, node_item):
    QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('connector_not_ready'))
    table_name = node_item.node.table_name
    sample = self.wizard.connector.get_table_sample(table_name, limit=20)
    QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('no_data'))
    dlg = QDialog(self)
    dlg.setWindowTitle(self.translator.tr('preview_table_title').format(table_name))
    layout = QVBoxLayout(dlg)
    table = QTableWidget()
    headers = list(sample[0].keys())
    table.setHorizontalHeaderLabels(headers)
    table.setRowCount(len(sample))
    for row in enumerate(sample):
        for col in enumerate(headers):
            layout.addWidget(table.QTableWidgetItem(str(row.get(col, ""))))
    dlg.exec_()

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/schema_page.py (относительно файла configure.py)

```

# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import QWizardPage, QVBoxLayout, QLabel, QComboBox, QTableWidget,
from lang.local_translator import LocalTranslator

class SchemaPage(QWizardPage):
    def __init__(self, wizard):
        super().__init__(wizard)
        self.wizard = wizard
        self.setTranslator(LocalTranslator())
        self.setWindowTitle(self.translator.tr('schema_page_title'))
        self.setSubTitle(self.translator.tr('schema_page_subtitle'))
        layout = QVBoxLayout(self)
        self.table_combo = QComboBox()
        self.table_combo.currentIndexChanged.connect(self.table_selected)
        layout.addWidget(self.table_combo)
        self.info_label = QLabel("")
        layout.addWidget(self.info_label)
        self.fields_table = QTableWidget(0, 2)
        self.setFieldsTableHeaderLabels(self.translator.tr('field_type_header'))
        layout.addWidget(self.fields_table)
        self.loading = QLabel(self.translator.tr('loading_message'))
        layout.addWidget(self.loading)

    def retranslateUi(self):
        self.setWindowTitle(self.translator.tr('schema_page_title'))
        self.setSubTitle(self.translator.tr('schema_page_subtitle'))
        self.setFieldsTableHeaderLabels(self.translator.tr('field_type_header'))

    def initializePage(self):
        self.loading.show()
        self.wizard.schema = self.wizard.connector.get_schema()
        for table in self.wizard.schema.tables:
            self.table_combo.addItem(table.name, table.name)
        if self.wizard.schema.tables:
            self.loading.hide()

    def table_selected(self, idx):
        if not table_name = self.table_combo.currentData():
            return
        for t in self.wizard.schema.tables:
            if t.name == table_name:
                self.info_label.setText(
                    f'{self.translator.tr('rows_count_label')}: {t.row_count or self.translator.tr('unknown')}',
                    f'{self.translator.tr('fields_count_label')}: {len(t.fields) or self.translator.tr('unknown')}'
                )
                for f in enumerate(t.fields):
                    self.fields_table.setItem(f[0], QTableWidget(f[1].field_type.name))
                break

    def validatePage(self):
        self.wizard.selected_table_name = self.table_combo.currentData()
        # -*- coding: utf-8 -*-

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/source_page.py (относительно файла configure.py)

```

# -*- coding: utf-8 -*-
# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import QWizardPage, QVBoxLayout, QHBoxLayout, QLabel, QComboBox, QLineEdit,
from lang.local_translator import LocalTranslator
from core.models import SourceType, FileFormat
from connectors.file_connector import FileConnector
from connectors.url_connector import URLConnector

class SourcePage(QWizardPage):
    def __init__(self, wizard):
        super().__init__(wizard)
        self.wizard = wizard
        self.setTranslator(LocalTranslator())
        self.setWindowTitle(self.translator.tr('source_page_title'))
        self.setSubTitle(self.translator.tr('source_page_subtitle'))
        layout = QVBoxLayout(self)
        self.type_combo = QComboBox()
        self.type_combo.addItems(self.translator.tr('source_type_file'), SourceType.FILE)
        self.type_combo.addItems(self.translator.tr('source_type_db'), SourceType.DATABASE)
        self.type_combo.addItems(self.translator.tr('source_type_url'), SourceType.URL)
        layout.addWidget(QLabel(self.translator.tr('source_type_label')))
        layout.addWidget(self.type_combo)
        # Файл
        self.file_group = QGroupBox(self.translator.tr('file_group'))

```

```

file_layout = QVBoxLayout(self.file_group)
self.format_combo = QComboBox()
for format in [FileFormat.SOURCE, FileFormat.XML, FileFormat.DBF, FileFormat.CSV, FileFormat.EXCEL]:
    self.format_combo.addItem(format.value)
file_layout.addWidget(self.format_combo)
self.path_edit = QLineEdit()
self.browse_btn = QPushButton(self.translator.tr('browse_button'))
self.path_layout = QHBoxLayout()
self.path_layout.addWidget(self.path_edit)
self.path_layout.addWidget(self.browse_btn)
file_layout.addWidget(self.path_layout)

# База данных
db_group = QGroupBox(self.translator.tr('db_group'))
db_layout = QVBoxLayout(db_group)
db_type_combo = QComboBox()
db_type_combo.addItems(["mysql", "mssql", "oracle"])
db_layout.addWidget(db_type_combo)
db_host_edit = QLineEdit(self.translator.tr('db_host_label'))
db_port_edit = QLineEdit(self.translator.tr('db_port_label'))
db_name_edit = QLineEdit(self.translator.tr('db_name_label'))
db_user_edit = QLineEdit(self.translator.tr('db_user_label'))
db_pass_edit = QLineEdit(self.translator.tr('db_pass_label'))
db_layout.addWidget(db_host_edit)
db_layout.addWidget(db_port_edit)
db_layout.addWidget(db_name_edit)
db_layout.addWidget(db_user_edit)
db_layout.addWidget(db_pass_edit)
db_group.setLayout(db_layout)

# URL
url_group = QGroupBox(self.translator.tr('url_group'))
url_layout = QVBoxLayout(url_group)
url_edit = QLineEdit(self.translator.tr('url_label'))
url_layout.addWidget(url_edit)
url_method_combo = QComboBox()
url_method_combo.addItems(["GET", "POST"])
url_layout.addWidget(url_method_combo)
url_group.setLayout(url_layout)

self.toggle_groups()

def retranslate_ui(self):
    self.setWindowTitle(self.translator.tr('source_page_title'))
    self.subtitle.setText(self.translator.tr('source_page_subtitle'))
    self.type_combo.setCurrentText(self.translator.tr('source_type_file'))
    self.type_combo.addItems([self.translator.tr('source_type_db'), self.translator.tr('source_type_url')])
    self.db_group.setTitle(self.translator.tr('db_group'))
    self.url_group.setTitle(self.translator.tr('url_group'))
    self.browse_btn.setText(self.translator.tr('browse_button'))
    self.path_edit.setText(self.translator.tr('path_label'))

def toggle_groups(self):
    if self.type_combo.currentData() == SourceType.FILE:
        self.db_group.setVisible(False)
        self.url_group.setVisible(True)
    elif self.type_combo.currentData() == SourceType.DATABASE:
        self.db_group.setVisible(True)
        self.url_group.setVisible(False)
    elif self.type_combo.currentData() == SourceType.URL:
        self.db_group.setVisible(False)
        self.url_group.setVisible(True)

def browse_file(self):
    path = QFileDialog.getOpenFileName(self, self.translator.tr('select_file_dialog'))
    if path:
        self.path_edit.setText(path)

def validate_page(self):
    st = self.type_combo.currentData()
    if st == SourceType.FILE:
        self.wizard.connector = FileConnector(self.path_edit.text(), self.format_combo.currentData())
    elif st == SourceType.DATABASE:
        params = {}
        params['host'] = self.db_host_edit.text()
        params['port'] = self.db_port_edit.text()
        params['db'] = self.db_name_edit.text()
        params['user'] = self.db_user_edit.text()
        params['password'] = self.db_pass_edit.text()
        self.wizard.connector = DBConnector("", **params)
    else:
        self.wizard.connector = URLConnector(self.url_edit.text())
    self.wizard.connector.connect()
    except Exception as e:
        QMessageBox.critical(self, self.translator.tr('error'), str(e))
    return False

```

Файл: /home/sergey/Documents/configurate/mozart_import/wizards/wizard_dialogs.py (открыть)

```

# -*- coding: utf-8 -*-
from PyQt5.QtWidgets import QDialog, QVBoxLayout, QFormLayout, QLineEdit, QComboBox, QSpinBox,
from lang.local_translator import LocalTranslator

class NewFieldDialog(QDialog):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('new_field_dialog_title'))
        layout = QVBoxLayout(self)
        form = QFormLayout()
        self.name_edit = QLineEdit()
        form.addRow(self.translator.tr('field_name_latin'), self.name_edit)
        self.alias_edit = QLineEdit()
        form.addRow(self.translator.tr('field_alias'), self.alias_edit)
        self.type_combo = QComboBox()
        self.type_combo.addItems(["C", "N", "I", "D", "T", "L", "M", "R"])
        form.addRow(self.translator.tr('mozart_type'), self.type_combo)
        self.length_edit = QSpinBox()
        self.length_edit.setValue(255)
        form.addRow(self.translator.tr('length_for_string'), self.length_edit)
        layout.addLayout(form)
        btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        btn_box.accepted.connect(self.accept)
        btn_box.rejected.connect(self.reject)
        self.name_edit.textChanged.connect(lambda: self.name_edit.setText(self.name_edit.text().lstrip()))
        layout.addWidget(btn_box)

    def get_data(self):
        return {
            'name': self.name_edit.text(),
            'alias': self.alias_edit.text() or self.name_edit.text(),
            'type': self.type_combo.currentText(),
            'length': self.length_edit.value()
        }

class NewEntityDialog(QDialog):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.translator = LocalTranslator()

```



```

    self.wizard.connector = DBConnector("", **params)
    else:
        self.wizard.connector = URLConnector(self.url_edit.text())
    self.wizard.connector.connect()
except Exception as e:
    QMessageBox.critical(self, self.translator.tr('error'), str(e))
    return False

class SchemaPage(QWizardPage):
    def __init__(self, wizard):
        self.wizard = wizard
        self.translator = WizardTranslator()
        self.translator.tr('schema_page_title')
        self.translator.tr('schema_page_subtitle')
        layout = QVBoxLayout(self)
        self.table_combo = QComboBox()
        layout.addWidget(self.table_combo)
        self.info_label = QLabel("")
        layout.addWidget(self.info_label)
        self.fields_table = QTableWidget(0, 2)
        layout.addWidget(self.fields_table)
        self.loading = QLabel(self.translator.tr('loading_message'))
        layout.addWidget(self.loading)

    def retranslate_ui(self):
        self.translator.tr('schema_page_title')
        self.translator.tr('schema_page_subtitle')
        self.translator.tr('field_name_header')
        self.translator.tr('field_type_header')

    def initializePage(self):
        self.loading.show()
        self.wizard.schema = self.wizard.connector.get_schema()
        for t in self.wizard.schema.tables:
            self.table_combo.addItem(t.name, t.name)
        if self.wizard.schema.tables:
            self.table_combo.setCurrentIndex(0)
        self.loading.hide()

    def table_selected(self, idx):
        table_name = self.table_combo.currentData()
        for t in self.wizard.schema.tables:
            if t.name == table_name:
                self.info_label.setText(self.translator.tr('rows_count_label') + str(t.row_count or self.translator.tr('unknown')) + ", " + self.translator.tr('samples_label') + str(len(t.samples) or 1))
                self.fields_table.setRowCount(len(t.fields))
                for f in t.fields:
                    self.fields_table.setItem(0, f.name)
                break

    def validatePage(self):
        self.wizard.selected_table_name = self.table_combo.currentData()
        return True

class MappingPage(QWizardPage):
    def __init__(self, wizard):
        self.wizard = wizard
        self.translator = WizardTranslator()
        self.translator.tr('mapping_page_title')
        self.translator.tr('mapping_page_subtitle')
        self.current_entity_id = None
        self.pending_fields = {} # {row: {name, type, length, alias, ref}}
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)
        # Верхняя панель: выбор/создание сущности
        entity_combo = QComboBox()
        layout.addWidget(entity_combo)
        entity_combo.setCurrentIndex(self.translator.tr('create_new_entity').None)
        layout.addWidget(QLabel(self.translator.tr('target_entity_label')))
        btn_create_entity = QPushButton(self.translator.tr('create_new_entity') + self.translator.tr('entity_button'))
        layout.addWidget(btn_create_entity)
        # Таблица маппинга: исходные поля -> целевые поля
        self.mapping_table = QTableWidget(1, 2)
        self.translator.tr('source_field_header')
        self.translator.tr('target_field_header')
        self.translator.tr('ref_header')
        self.mapping_table.horizontalHeader().setStretchLastSection(True)
        layout.addWidget(self.mapping_table)
        # Кнопки действий
        btn_apply_structure = QPushButton(self.translator.tr('apply_structure_button'))
        btn_import_data = QPushButton(self.translator.tr('import_data_button'))
        layout.addWidget(btn_apply_structure)
        layout.addWidget(btn_import_data)
        # Вкладка предпросмотра данных
        preview_table = QTableWidget()
        layout.addWidget(preview_table)
        self.loading = QLabel(self.translator.tr('loading_message'))
        layout.addWidget(self.loading)
        self.refresh_entity_list()

    def retranslate_ui(self):
        self.translator.tr('mapping_page_title')
        self.translator.tr('mapping_page_subtitle')
        self.translator.tr('create_new_entity')
        self.translator.tr('entity_button')
        self.translator.tr('apply_structure_button')
        self.translator.tr('import_data_button')
        self.translator.tr('data_preview_tab')
        self.translator.tr('source_field_header')
        self.translator.tr('target_field_header')
        self.translator.tr('ref_header')

    def refresh_entity_list(self):
        self.entity_combo.clear()
        self.translator.tr('create_new_entity').None)
        for eid, cname, alias, query in self.wizard.connector.query("SELECT id, cname, alias FROM meta.entitytypes ORDER BY cname"):
            self.entity_combo.addItem(f"{cname} ({alias})", eid)

    def on_entity_selected(self):
        self.current_entity_id = self.entity_combo.currentData()
        self.load_mapping_table()

```



```

def load_mapping_table(self):
    if not self.wizard.schema:
        return
    source_table = None
    for i, name in enumerate(self.wizard.selected_table_name):
        source_table = self.wizard.selected_table_name
        break
    if not source_table:
        self.mapping_table.setRowCount(0)
        return
    self.mapping_table.setRowCount(len(source_table.fields))
    for row in range(self.mapping_table.setRowCount()):
        self.mapping_table.setItem(row, 0, QTableWidgetItem(sf.name))
        # combo для выбора существующих полей + опция создания нового
        combo = QComboBox()
        if self.current_entity_id:
            self.translator.tr('ignore_field') + " --", ""))
        fields = self.wizard.execute_query(
            (self.current_entity_id,
             (self.current_entity_id,
              for name, alias in fields:
                  self.translator.tr('create_new_field') + " ++", "NEW")
            self.mapping_table.setCurrentIndex(lambda idx, r=row: self.on_new_field_selected(r, combo))
        self.mapping_table.setItem(row, 2, QTableWidgetItem(sf.field_type.name))
        self.mapping_table.setItem(row, 3, QTableWidgetItem(sf.field_type.name))
        # row, в которой pending поля для этой строки, восстановить
        self.show_pending_editor(row, self.pending_fields[row])
    # Загрузить предпросмотр данных
    self.load_preview(source_table)

def on_new_field_selected(self, row, combo):
    if combo.currentIndex() == NEW:
        combo.setCurrentIndex(0)
        combo.blockSignals(False)
        self.show_pending_editor(row)
    else:
        self.show_pending_editor(self, row, pending=None):
        # в режиме создания в выпадающем списке нового поля
        # для простоты использования ODBC в ячейках колонок 2-4
        name_edit = QLineEdit()
        if pending and pending.get('name'):
            self.name_edit.setText(pending['name'])
        else:
            source_field = self.mapping_table.item(row, 0).text()
            self.name_edit.setText(source_field)
        self.mapping_table.setItem(row, 2, name_edit)
        type_combo = QComboBox()
        pending_and_pending.get('type')
        type_combo.setCurrentIndex(idx)
        if idx == type_combo.currentIndex():
            self.type_combo.setText(pending['type'])
        else:
            type_combo.setCurrentIndex(idx)
        type_combo.setCurrentIndex(idx)
        self.mapping_table.setItem(row, 3, type_combo)
        length_spin = QSpinBox()
        length_spin.setValue(pending.get('length', 255) if pending else 255)
        self.mapping_table.setItem(row, 4, length_spin)
        self.update_pending(r, 'name', t)
        self.update_pending(r, 'type', t)
        self.update_pending(r, 'length', v)
    # Создать pending
    self.pending_fields[row] = {
        'name': name_edit.text(),
        'type': type_combo.currentText(),
        'length': length_spin.value(),
        'alias': name_edit.text()
    }

def update_pending(self, row, key, value):
    if row in self.pending_fields:
        self.pending_fields[row][key] = value

def to_field_name(self, source_field):
    # преобразует имя исходного поля в допустимое имя поля MySQL/CamelCase
    import re
    name = re.sub(r'[^a-zA-Z0-9_]', '_', source_field)
    return name.lower()

def apply_structure(self):
    if self.current_entity_id:
        QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('select_entity_first'))
    # собираем все новые поля из pending_fields
    fields_to_create = []
    for row, pending in self.pending_fields.items():
        continue
        fields_to_create.append({
            'field_name': pending['name'],
            'data_type': pending['type'],
            'length': pending.get('length', 255),
            'indexed': True
        })
    if not fields_to_create:
        QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('no_fields_to_create'))
    # создаем поля через FieldCreator
    creator = FieldCreator(self.wizard.db)
    try:
        created_ids = creator.create_fields(self.current_entity_id, fields_to_create)
    except Exception as e:
        QMessageBox.critical(self, self.translator.tr('error'), str(e))
    # выводим таблицу mappings: убрать редакторы, загрузить существующие поля
    self.load_mapping_table()

def import_data(self):
    # импорт данных для импорта
    for row in range(self.mapping_table.setRowCount()):
        source_field = self.mapping_table.item(row, 0).text()
        target_field = self.mapping_table.item(row, 1).text()
        if target_field and target_field != NEW:
            matched_fields.append({
                'source_field': source_field,
                'target_field': target_field,
                'confidence': 1.0,
                'is_reference': False
            })
    # Создать MatchedEntity
    matched_entity = MatchedEntity(
        source_table=self.wizard.selected_table_name,
        target_entity_name=self.current_entity_id,
        confidence=1.0,
        fields=matched_fields,
        is_new=False
    )
    self.wizard.matching_proposal = MappingProposal(
        source_schema=self.wizard.schema,
        involved_names=entity,
        suggestions_for_new_entities=[]
    )
    # Переходим на страницу импорта (ExecutePage)
    self.wizard.next()

def load_preview(self, source_table):
    if not source_table:
        self.preview_table.setRowCount(0)

```



```

        match = True
        for col, pattern in self.filters.items():
            value = str(record[self.headers[col]]) if record[self.headers[col]] is not None else ""
            # Попробуем match с pattern, но не выражение
            if not re.match(regex, value):
                match = False
                break
        if match:
            self.filtered_rows.append(row)
        self.endResetModel()

class DBFViewer(QMainWindow):
    def __init__(self):
        self.setWindowTitle("Просмотрщик DBF")
        self.resize(900, 700)
        central = QWidget()
        main_layout = QVBoxLayout(central)
        # Верхняя панель
        top_layout = QHBoxLayout()
        self.btn_open = QPushButton("Открыть DBF файл")
        self.btn_open.clicked.connect(self.open_file)
        top_layout.addWidget(self.btn_open)
        self.btn_apply = QPushButton("Применить фильтр")
        top_layout.addWidget(self.btn_apply)
        self.btn_clear = QPushButton("Сбросить фильтры")
        top_layout.addWidget(self.btn_clear)
        self.info_label = QLabel("Файл не открыт")
        top_layout.addWidget(self.info_label)
        main_layout.addLayout(top_layout)
        # Панель фильтров (горизонтальные QLineEdit)
        self.filter_layout = QHBoxLayout()
        main_layout.addLayout(self.filter_layout)
        # Таблица
        self.table_view = QTableView()
        font = self.table_view.font()
        font.setPointSize(12)
        self.table_view.setFont(font)
        self.table_view.horizontalHeader().setStretchLastSection(True)
        main_layout.addWidget(self.table_view)
        self.model = None
        self.filter_edits = []

    def open_file(self):
        file_path = QFileDialog.getOpenFileName(
            self, "Выберите DBF файл", "", "DBF files (*.DBF);;All files (*)"
        )
        if not file_path:
            return
        try:
            # Определяем кодировку
            encodings = ['cp866', 'utf-8', 'windows-1251']
            for enc in encodings:
                try:
                    dbf = DBF(file_path, encoding=enc)
                    # Проверим, что читается первая запись
                    next(iter(dbf))
                    except UnicodeDecodeError, StopIteration):
                        continue
                    if dbf is None:
                        dbf = DBF(file_path)
                    # Читаем все записи (для больших файлов осторожно, но КЛАДР влезет)
                    data = list(dbf)
                    headers = [field.name for field in dbf.fields]
                    # Создаём модель
                    self.model = DBFTableModel(data, headers)
                    self.table_view.setModel(self.model)
                    # Создаём поля ввода для фильтров
                    self.create_filter_inputs(headers)
                    self.info_label.setText(f"Файл: {file_path.split('/')[-1]} | Всего записей: {len(data)}")
                    except Exception as e:
                        QMessageBox.critical(self, "Ошибка", f"Не удалось прочитать файл: \n{str(e)}")

    def create_filter_inputs(self, headers):
        # Создаём панель для фильтров
        self.filter_layout.addWidget(QLabel("Фильтры:"))
        for col, header in enumerate(headers):
            line_edit = QLineEdit()
            line_edit.setText(header)
            line_edit.setPlaceholderText(header)
            self.filter_layout.addWidget(line_edit)
            self.filter_edits.append((col, line_edit))

    def clear_filter_layout(self):
        for i in reversed(range(self.filter_layout.count())):
            widget = self.filter_layout.itemAt(i).widget()
            if widget:
                widget.deleteLater()

    def apply_filters(self):
        if self.model is None:
            return
        for col, line_edit in self.filter_edits:
            text = line_edit.text().strip()
            self.model.setFilter(col, text if text else None)

    def clear_filters(self):
        if self.model is None:
            return
        self.model.clearFilters()
        for line_edit in self.filter_edits:
            line_edit.clear()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    viewer = DBFViewer()
    viewer.show()
    sys.exit(app.exec_())

```

Файл: /home/sergey/Documents/configurate/sqlite/data_browser_dialog.py (относительно

```

from PyQt5.QtWidgets import (QVBoxLayout, QTableWidget, QTableWidgetItem,
                             QPushButton, QMessageBox, QHeaderView, QAbstractItemView)
from database import DatabaseManager
class DataBrowserDialog(QDialog):
    def __init__(self, db: DatabaseManager, table_name: str, parent=None):
        super().__init__(parent)
        self.table_name = table_name
        self.setWindowTitle(f"Данные: {table_name}")
        self.setGeometry(900, 600)
        self.load_data()

    def init_ui(self):
        layout = QVBoxLayout(self)
        self.data_table = QTableWidget()
        self.data_table.setHeader(QAbstractItemView.AllEditTriggers)
        self.data_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)

```

```

layout.addWidget(self.data_table)
btn_layout = QHBoxLayout()
btn_add = QPushButton("Добавить строку")
btn_del = QPushButton("Удалить строку")
btn_save = QPushButton("Сохранить изменения")
btn_layout.addWidget(btn_add)
btn_layout.addWidget(btn_del)
btn_layout.addWidget(btn_save)
layout.addLayout(btn_layout)

def load_data(self):
    cursor = self.cursor.execute(f"SELECT * FROM {self.table_name}")
    self.data_table.setColumnCount(len(cursor.description))
    self.data_table.setRowCount(len(cursor.description))
    for row_idx, row_data in enumerate(cursor.fetchall()):
        self.data_table.setRow(row_idx)
        for col_idx, (value, _type) in enumerate(row_data):
            self.data_table.setItem(row_idx, col_idx, QTableWidgetItem(txt))

def add_row(self):
    self.data_table.insertRow(self.data_table.rowCount())

def delete_row(self):
    row = self.data_table.currentRow()
    self.data_table.removeRow(row)

def save_changes(self):
    reply = QMessageBox.question(self, "Подтверждение",
    "Перезаписать все данные таблицы?",
    QMessageBox.Yes | QMessageBox.No)
    if reply != QMessageBox.Yes:
        return
    try:
        self.perform_save()
    except Exception as e:
        if "FOREIGN KEY constraint failed" in err_msg:
            reply = QMessageBox.warning(self, "Ошибка сохранения",
            "Операция нарушает целостность внешних ключей:\n{err_msg}\n\n"
            "Одобрить временно отключить внешние ключи?",
            QMessageBox.Yes | QMessageBox.No)
            if reply == QMessageBox.Yes:
                try:
                    self.db.execute("PRAGMA foreign_keys = OFF")
                    self.db.execute("PRAGMA foreign_keys = ON")
                    QMessageBox.information(self, "Готово", "Изменения сохранены с отключенными FK.")
                except:
                    self.db.rollback()
                    QMessageBox.critical(self, "Ошибка", str(e2))
            else:
                self.db.rollback()
                QMessageBox.critical(self, "Ошибка", err_msg)
                self.db.rollback()

def perform_save(self):
    self.db.execute(f"DELETE FROM {self.table_name}")
    cols = [self.data_table.horizontalHeaderItem(i).text()
    for i in range(self.data_table.columnCount())]
    sql = f"INSERT INTO {self.table_name} ({','.join(cols)}) VALUES ({placeholders})"
    for row in range(self.data_table.rowCount()):
        for col in range(self.data_table.columnCount()):
            item = self.data_table.item(row, col)
            val = item.text() if item else ""
            # попытка преобразования в число
            try:
                if '.' in val:
                    val = float(val)
                else:
                    val = int(val)
            except ValueError:
                pass
            values.append(val)
        self.db.execute(sql, values)
    self.db.commit()
    QMessageBox.information(self, "Готово", "Данные успешно сохранены.")

```

Файл: /home/sergey/Documents/configurate/sqlite/database.py (относительно: sqlite/data

```

import sqlite3
class DatabaseManager:
    """Управление соединением с SQLite и выполнение операций."""
    def __init__(self):
        self.path = None
    def open(self, db_path: str):
        if self.connection:
            self.connection.close()
        self.path = db_path
        self.connection = sqlite3.connect(db_path)
        self.connection.execute("PRAGMA foreign_keys = ON")
        self.connection.row_factory = sqlite3.Row # для доступа по именам столбцов
    def close(self):
        if self.connection:
            self.connection.close()
            self.connection = None
    def execute(self, sql: str, params=None):
        """Выполнить запрос, возвращает курсор."""
        cursor = self.connection.cursor()
        if params:
            cursor.execute(sql, params)
        else:
            cursor.execute(sql)
        return cursor
    def commit(self):
        self.connection.commit()
    def rollback(self):
        self.connection.rollback()
    def get_tables(self):
        """Список пользовательских таблиц"""
        cursor = self.execute("SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE 'sqlite_%'")
        return [row['name'] for row in cursor.fetchall()]
    def get_views(self):
        """Список представлений и их определений"""
        cursor = self.execute("SELECT name, sql FROM sqlite_master WHERE type='view'")
        return [(row['name'], row['sql']) for row in cursor.fetchall()]
    def get_table_schema(self, table_name: str):
        """Вся информация о таблице"""
        cursor = self.execute(f"PRAGMA table_info('{table_name}'))")
        columns = [dict(row) for row in cursor.fetchall()]
        # Получаем внешние ключи
        foreign_keys = [dict(row) for row in cursor.fetchall()]
        return {
            'name': table_name,

```

```

        'columns': columns,
        'foreign_keys': foreign_keys
    }
    def table_exists(self, table_name: str) -> bool:
        cursor = self.execute(f"SELECT name FROM sqlite_master WHERE type='table' AND name=?", (table_name,))
        return bool(cursor.fetchone())

```

Файл: /home/sergey/Documents/configurate/sqlite/main.py (относительно: sqlite/main.py)

```

import sys
from PyQt5.QtWidgets import QApplication
from main_window import MainWindow
DEFAULT_DIR = "/home/sergey/Documents/configurate/sqlite"
def main():
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec_())
if __name__ == "__main__":
    main()

```

Файл: /home/sergey/Documents/configurate/sqlite/main_window.py (относительно: sqlite

```

import os
from PyQt5.QtWidgets import QMainWindow, QTabWidget, QMenuBar, QAction, QFileDialog, QStatusBar
from database import DatabaseManager, TableListWidget
from widgets.view_list_widget import ViewListWidget
DEFAULT_DIR = "/home/sergey/Documents/configurate/sqlite"
class MainWindow(QMainWindow):
    """
    Главное окно приложения.
    """
    def __init__(self):
        self.db = DatabaseManager()
        self.setWindowTitle("SQLite Manager")
        self.resize(1100, 750)
        self.setup_menu()
        self.statusBar().showMessage("Готов. Откройте или создайте базу данных.")
    def setup_menu(self):
        file_menu = self.menuBar().addMenu("Файл")
        open_act = QAction("Открыть", self)
        file_menu.addAction(open_act).triggered.connect(self.open_db)
        create_act = QAction("Создать", self)
        file_menu.addAction(create_act).triggered.connect(self.create_db)
        file_menu.addSeparator()
        exit_act = QAction("Выход", self)
        file_menu.addAction(exit_act).triggered.connect(self.close)
    def setup_ui(self):
        self.table = TableListWidget(self.db)
        self.view_widget = ViewListWidget(self.db)
        self.tabs.addTab(self.table_widget, "Таблицы")
        self.tabs.addTab(self.view_widget, "Представления")
        self.setCentralWidget(self.tabs)
    def open_db(self):
        path = QFileDialog.getOpenFileName(self,
            "Открыть SQLite файлы (*.db *.sqlite *.sqlite3)",
            DEFAULT_DIR,
            "SQLite файлы (*.db *.sqlite *.sqlite3); Все файлы (*)")
        if path:
            self.db.open(path)
            self.after_connect()
    def create_db(self):
        path = QFileDialog.getSaveFileName(self,
            "Создать базу данных",
            DEFAULT_DIR,
            "SQLite файлы (*.db *.sqlite *.sqlite3)")
        if path:
            if not os.path.exists(path):
                open(path).close()
            self.db.open(path)
            self.after_connect()
    def after_connect(self):
        self.table_widget.refresh()
        self.statusBar().showMessage(f"База данных: {self.db.path}")
    def closeEvent(self, event):
        event.accept()

```

Файл: /home/sergey/Documents/configurate/sqlite/models.py (относительно: sqlite/mode

```

from dataclasses import dataclass, field
from typing import List, Optional
@dataclass
class ColumnDef:
    """
    Описание столбца таблицы.
    """
    col_type: str = "TEXT" # тип как строка (INTEGER, TEXT, ...)
    pk: bool = False # часть первичного ключа
    default: Optional[str] = None
@dataclass
class ForeignKey:
    """
    Описание ключа.
    """
    to_column: str
@dataclass
class Schema:
    """
    Описание таблицы.
    """
    name: str
    columns: List[ColumnDef] = field(default_factory=list)
    foreign_keys: List[ForeignKey] = field(default_factory=list)
    def add_column(self, col: ColumnDef):
        self.columns.append(col)
    def add_foreign_key(self, fk: ForeignKey):
        self.foreign_keys.append(fk)
    def generate_create_sql(self) -> str:
        """
        Сгенерировать SQL CREATE TABLE.
        """
        cols_sql = ""
        for col in self.columns:
            part = f"{col.name} {col.col_type}"
            if col.pk:
                part += " PRIMARY KEY"
            if col.default:
                part += f" DEFAULT {col.default}"
            if col.col_type == "TEXT":
                part += " NOT NULL"
            cols_sql += f"{part}, "
        cols_sql = cols_sql.rstrip(", ")
        fks_sql = ""
        for fk in self.foreign_keys:
            part = f"CONSTRAINT {fk.name} FOREIGN KEY ({fk.to_column})"
            fks_sql += f"{part}, "
        fks_sql = fks_sql.rstrip(", ")
        return f"CREATE TABLE {self.name} ({cols_sql}{fks_sql})"

```



```

        pk_cols.append(col.name)
    cols_sql.append(part)
# FK
for fk in self.foreign_keys:
    cols_sql.append(
        f"FOREIGN KEY ({fk.from_column}) REFERENCES {fk.to_table}({fk.to_column})"
    )
if pk_cols:
    cols_sql.append(f"PRIMARY KEY ({', '.join(pk_cols)})")
sql = f"CREATE TABLE {self.name} (\n " + "\n ".join(cols_sql) + "\n)"
return sql

```

Файл: /home/sergey/Documents/configurate/sqlite/table_editor_dialog.py (относительно: s

```

from PyQt5.QtWidgets import (
    QApplication, QDialog, QVBoxLayout, QLabel, QLineEdit, QTableWidget,
    QGroupBox, QDialogButtonBox, QMessageBox, QHeaderView, QWidget,
)
from PyQt5.QtCore import Qt
from models import ColumnDef, ForeignKey, TableSchema
from database import DatabaseManager
SQLITE_TYPES = [
    "INTEGER", "TEXT", "REAL", "BLOB", "NUMERIC",
    "DATE", "DATETIME", "TIMESTAMP", "BOOLEAN",
    "VARCHAR(255)", "CHAR(50)", "DECIMAL(16,2)",
]

class TableEditorDialog(QDialog):
    """
    Диалог создания и редактирования структуры таблицы
    """
    def __init__(self, db: DatabaseManager, table_name: str = None, parent=None):
        super().__init__(parent)
        self.db = db
        self.original_name = table_name
        self.setWindowTitle(f"Редактировать таблицу '{table_name}'" if table_name else "Создать таблицу")
        self.resize(800, 550)
        if self.load_existing_schema():
            self._load_existing_schema()

    def _init_ui(self):
        layout = QVBoxLayout(self)

        # Название
        name_layout = QHBoxLayout()
        name_label = QLabel("Название таблицы:")
        name_edit = QLineEdit()
        name_layout.addWidget(name_label)
        name_layout.addWidget(name_edit)
        layout.addLayout(name_layout)

        # Таблица столбцов
        self.col_table = QTableWidget(0, 5)
        self.col_table.setHorizontalHeaderLabels(["Имя", "Тип", "PK", "Not Null", "Unique"])
        layout.addWidget(self.col_table)
        self.col_table.setSectionResizeMode(QHeaderView.Stretch)

        # Кнопки столбцов
        col_btn_layout = QHBoxLayout()
        btn_add_col = QPushButton("Добавить столбец")
        btn_remove_col = QPushButton("Удалить столбец")
        col_btn_layout.addWidget(btn_add_col)
        col_btn_layout.addWidget(btn_remove_col)
        layout.addLayout(col_btn_layout)

        # Внешние ключи
        fk_group = QGroupBox("Внешние ключи")
        self.fk_table = QTableWidget(0, 2)
        self.fk_table.setHorizontalHeaderLabels(["Столбец", "Ссылка (таблица.столбец)"])
        fk_layout = QVBoxLayout(fk_group)
        fk_layout.addWidget(self.fk_table)
        fk_btn_layout = QHBoxLayout()
        btn_add_fk = QPushButton("Добавить FK")
        btn_remove_fk = QPushButton("Удалить FK")
        fk_btn_layout.addWidget(btn_add_fk)
        fk_btn_layout.addWidget(btn_remove_fk)
        fk_group.addLayout(fk_btn_layout)
        layout.addWidget(fk_group)

        # OK/Cancel
        btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        btn_box.accepted.connect(self._on_save)
        btn_box.rejected.connect(self._reject)
        layout.addWidget(btn_box)

        self.name_edit.editingFinished.connect(self._update_schema_name)

    def _add_column_row(self):
        row = self.col_table.rowCount()
        self.col_table.setRowCount(row + 1)
        self.col_table.setItem(row, 0, QTableWidgetItem(""))
        type_combo = QComboBox()
        type_combo.addItems(SQLITE_TYPES)
        self.col_table.setCellWidget(row, 1, type_combo)
        for col_idx in [2, 3, 4]:
            w = QWidget()
            lay = QVBoxLayout(w)
            lay.setAlignment(Qt.AlignCenter)
            cb = QCheckBox()
            lay.addWidget(cb)
            self.col_table.setCellWidget(row, col_idx, w)

    def _remove_column_row(self):
        row = self.col_table.currentRow()
        self.col_table.removeRow(row)

    def _add_fk_row(self):
        row = self.fk_table.rowCount()
        self.fk_table.setRowCount(row + 1)
        self.fk_table.setItem(row, 0, QTableWidgetItem(""))
        ref_combo = QComboBox()
        ref_combo.setTableTitle("")
        tables = self.db.get_tables()
        for schema in self.db.get_schemas():
            for col in schema.columns:
                ref_combo.addItem(f"{col['name']}")
        self.fk_table.setCellWidget(row, 1, ref_combo)

    def _remove_fk_row(self):
        row = self.fk_table.currentRow()
        self.fk_table.removeRow(row)

    def _load_existing_schema(self):
        self._init_ui()
        self.name_edit.setText(self.original_name)
        for col in self.db.columns:
            row = self.col_table.rowCount() - 1
            self.col_table.setItem(row, 0, col['name'])
            type_combo = self.col_table.cellWidget(row, 1)
            type_combo.setCurrentText(col['type'].upper())
            pk_cb = self.col_table.cellWidget(row, 2).findChild(QCheckBox)
            pk_cb.setChecked(col['pk'])
            nn_cb = self.col_table.cellWidget(row, 3).findChild(QCheckBox)
            nn_cb.setChecked(col['notnull'])
            un_cb = self.col_table.cellWidget(row, 4).findChild(QCheckBox)

```

```

un_cb.setChecked(False) # уникальность не загружаем, можно доработать
for fk in info['foreign_keys']:
    self.add_fk_table_row_count()
    self.fk_table.item(row, 0).setText(fk['from'])
    ref_combo = self.fk_table.cellWidget(row, 1)
    ref_combo.setCurrentText(ref_str)
def build_schema(self) -> TableSchema:
    schema = TableSchema(name=self.schema_name_edit.text().strip())
    for name, item in self.col_table.item(row, 0):
        if not name: item or not name: item.text().strip():
            name = item.text().strip()
            type_combo = self.col_table.cellWidget(row, 1)
            col_type = type_combo.currentText()
            pk = self.col_table.cellWidget(row, 3).findChild(QCheckBox).isChecked()
            un = self.col_table.cellWidget(row, 4).findChild(QCheckBox).isChecked()
            schema.add_column(Column(name, col_type, pk, un, un))
    for row in self.fk_table.item(row, 0):
        ref_widget = self.fk_table.cellWidget(row, 1)
        if not item and ref_widget():
            from_col = ref_widget().strip()
            to_table = ref_widget().strip()
            to_col = ref_widget().strip()
            schema.add_foreign_key(ForeignKey(from_col, to_table, to_col))
    return schema
def on_save(self):
    schema = self.build_schema()
    if not schema.name:
        QMessageBox.warning(self, "Ошибка", "Имя таблицы не может быть пустым")
    if not schema.columns:
        QMessageBox.warning(self, "Ошибка", "Добавьте хотя бы один столбец")
    return
    # Новая таблица
    try:
        self.db.execute(schema.generate_create_sql())
        self.db.commit()
    except Exception as e:
        QMessageBox.critical(self, "Ошибка", str(e))
        self.db.rollback()
    # Редактирование существующей
    old_name = self.schema_name
    try:
        # Создаём таблицу с новой структурой
        new_cols = schema.generate_create_sql()
        old_cols = self.db.execute('PRAGMA table_info({})'.format(schema(old_name)['columns']))
        new_cols = [c for c in old_cols if c in new_cols]
        if not new_cols:
            self.db.execute('DROP TABLE IF EXISTS {}'.format(schema(old_name)))
            self.db.execute('CREATE TABLE {}'.format(schema(new_name)))
        else:
            self.db.execute('INSERT INTO {}'.format(schema(new_name)) + ' SELECT {cols} FROM {}'.format(schema(old_name)))
        self.db.execute('PRAGMA foreign_keys = OFF')
        self.db.execute('PRAGMA foreign_keys = ON')
        self.db.commit()
    except Exception as e:
        QMessageBox.critical(self, "Ошибка", str(e))
    try:
        self.db.execute('PRAGMA foreign_keys = OFF')
        if self.db.execute('PRAGMA table_info({})'.format(schema(new_name))) != 0:
            self.db.execute('PRAGMA foreign_keys = ON')
        self.db.commit()
    except:
        self.db.rollback()
def update_schema_name(self):
    self.schema_name = self.name_edit.text().strip()

```

Файл: /home/sergey/Documents/configurate/sqlite/view_dialog.py (относительно: sqlite/v

```

from PyQt5.QtWidgets import (
    QDialog, QVBoxLayout, QLabel, QLineEdit, QDialogButtonBox, QMessageBox
)
from database import DatabaseManager
class ViewDialog(QDialog):
    """Создание или редактирование представления."""
    def __init__(self, db: DatabaseManager, view_name=None, view_sql=None, parent=None):
        self.db = db
        self.original_name = view_name
        self.schema_name = "Редактировать представление" if view_name else "Создать представление"
        layout = QVBoxLayout(self)
        layout.addWidget(QLabel("Имя представления:"))
        layout.addWidget(self.name_edit)
        layout.addWidget(QLabel("SQL-запрос:"))
        layout.addWidget(self.sql_edit)
        self.name_edit.setText(view_name)
        self.sql_edit.setText(view_sql)
        btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        btn_box.accepted.connect(self.save)
        btn_box.rejected.connect(self.reject)
        layout.addWidget(btn_box)
    def save(self):
        name = self.name_edit.text().strip()
        if not name:
            QMessageBox.warning(self, "Ошибка", "Заполните имя и запрос")
            return
        try:
            if self.original_name:
                self.db.execute('DROP VIEW IF EXISTS {}'.format(self.original_name))
            self.db.execute('CREATE VIEW {}'.format(name) + ' AS {}'.format(self.sql))
            self.db.commit()
        except Exception as e:
            QMessageBox.critical(self, "Ошибка", str(e))
            self.db.rollback()

```

Файл: /home/sergey/Documents/configurate/sqlite/widgets/__init__.py (относительно: sqlite/w

```

# widgets/ __init__.py
from .table_list_widget import TableListWidget

```

Файл: /home/sergey/Documents/configurate/sqlite/widgets/table_list_widget.py (относитель

```

from PyQt5.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QListWidget, QMessageBox
from database import DatabaseManager
from data_browser_dialog import DataBrowserDialog
class TableListWidget(QWidget):
    """Вкладка со списком таблиц и кнопками управления."""
    def __init__(self, db: DatabaseManager, parent=None):

```

```

super().__init__(parent)
self.db = db
self.init_ui()
def init_ui(self):
    layout = QVBoxLayout(self)
    self.list_widget = QListWidget()
    layout.addWidget(self.list_widget)
    btn_layout = QHBoxLayout()
    self.add_btn = QPushButton("Добавить")
    self.del_btn = QPushButton("Удалить")
    self.browse_btn = QPushButton("Browse")
    btn_layout.addWidget(self.add_btn)
    btn_layout.addWidget(self.del_btn)
    btn_layout.addWidget(self.browse_btn)
    layout.addLayout(btn_layout)
    self.add_btn.clicked.connect(self.add_table)
    self.del_btn.clicked.connect(self.delete_table)
    self.browse_btn.clicked.connect(self._browse_table)
def refresh(self):
    self.list_widget.clear()
    tables = self.db.get_tables()
    self.list_widget.addItems(tables)
def add_table(self):
    dlg = TableEditorDialog(self.db, parent=self)
    if dlg.exec_():
        self.refresh()
def edit_table(self):
    item = self.list_widget.currentItem()
    if not item: return
    name = item.text()
    dlg = TableEditorDialog(self.db, name, parent=self)
    if dlg.exec_():
        self.refresh()
def delete_table(self):
    item = self.list_widget.currentItem()
    if not item: return
    name = item.text()
    if QMessageBox.question(self, "Удалить", f"Удалить таблицу {name}?") == QMessageBox.Yes:
        try:
            self.db.execute(f"DROP TABLE IF EXISTS {name}")
            self.db.commit()
        except Exception as e:
            QMessageBox.critical(self, "Ошибка", str(e))
def browse_table(self):
    item = self.list_widget.currentItem()
    if not item: return
    name = item.text()
    dlg = DataBrowserDialog(self.db, name, parent=self)
    dlg.exec_()

```

Файл: /home/sergey/Documents/configurate/sqlite/widgets/view_list_widget.py (относительн

```

from PyQt5.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QPushButton, QListWidget, QMessageBox
from PyQt5.QtCore import Qt
from PyQt5.QtSql import QSqlDatabase, QSqlQuery
from PyQt5.QtGui import QIcon
from PyQt5.QtCore import Qt
from PyQt5.QtSql import QSqlDatabase, QSqlQuery
from PyQt5.QtGui import QIcon
class ViewListWidget(QWidget):
    """Вкладка со списком представлений и кнопками."""
    def __init__(self, db, DatabaseManager, parent=None):
        super().__init__(parent)
        self.init_ui()
    def init_ui(self):
        layout = QVBoxLayout(self)
        self.list_widget = QListWidget()
        layout.addWidget(self.list_widget)
        btn_layout = QHBoxLayout()
        self.add_btn = QPushButton("Добавить")
        self.del_btn = QPushButton("Удалить")
        self.browse_btn = QPushButton("Browse")
        btn_layout.addWidget(self.add_btn)
        btn_layout.addWidget(self.del_btn)
        btn_layout.addWidget(self.browse_btn)
        layout.addLayout(btn_layout)
        self.add_btn.clicked.connect(self.add_view)
        self.del_btn.clicked.connect(self.delete_view)
        self.browse_btn.clicked.connect(self._browse_view)
    def refresh(self):
        self.list_widget.clear()
        views = self.db.get_views()
        for view in views:
            self.list_widget.addItem(v['name'])
    def add_view(self):
        dlg = ViewEditorDialog(self.db, parent=self)
        if dlg.exec_():
            self.refresh()
    def delete_view(self):
        item = self.list_widget.currentItem()
        if not item: return
        name = item.text()
        if QMessageBox.question(self, "Удалить", f"Удалить представление {name}?") == QMessageBox.Yes:
            try:
                self.db.execute(f"DROP VIEW IF EXISTS {name}")
                self.db.commit()
            except Exception as e:
                QMessageBox.critical(self, "Ошибка", str(e))
    def browse_view(self):
        item = self.list_widget.currentItem()
        if not item: return
        name = item.text()
        dlg = DataBrowserDialog(self.db, name, parent=self)
        dlg.exec_()
        dlg.data_table.setEditableTriggers(QAbstractItemView.NoEditTriggers)
        dlg.add_btn.setVisible(False)
        dlg.save_btn.setVisible(False)
        self.refresh()

```

Файл: /home/sergey/Documents/configurate/tabs/__init__.py (относительно: tabs/__init__.p

```

from .balance_units_tab import BalanceUnitsTab
from .balances_tab import BalancesTab
from .documents_tab import DocumentsTab
from .data_view_tab import DataViewTab
from .relations_tab import RelationsTab
__all__ = [
    'BalanceUnitsTab',
    'BalancesTab',
    'DocumentsTab',
    'DataViewTab',
    'RelationsTab'
]

```



```

    dlg = BalanceUnitEditDialog(self, data, db=self.db)
    if dlg.exec_() == dlg.get_data():
        sql = """UPDATE auth.balanceunits SET cname=%s, calias=%s, parentid=%s, lisbalanceunit=%s WHERE
        self.db.execute_query(sql, [new_data["cname"], new_data["calias"], new_data["parentid"],
        new_data["lisbalanceunit"], item_id], fetch=False)
    def delete_item(self):
        item_id = self.get_selected_id()
        self.show_warning("Выберите запись для удаления")
        if QMessageBox.question(self, "Подтверждение", "Удалить выбранную запись?") == QMessageBox.Yes:
            self.db.execute_query(sql, (item_id,), fetch=False)
            self.load_data()

```

Файл: /home/sergey/Documents/configurate/tabs/base_tab.py (относительно: tabs/base_tab.py)

```

# tabs/base_tab.py
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QMessageBox
from lang.local_translator import LocalTranslator

class BaseTab(QWidget):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.translate()
        self.layout.setContentsMargins(5, 5, 5, 5)
    def show_error(self, message):
        QMessageBox.critical(self, self.translator.tr('error'), message)
    def show_info(self, message):
        QMessageBox.information(self, self.translator.tr('info'), message)
    def show_warning(self, message):
        QMessageBox.warning(self, self.translator.tr('warning'), message)
    def refresh(self):
        pass
    def retranslate_ui(self):
        pass

```

Файл: /home/sergey/Documents/configurate/tabs/data_view_tab.py (относительно: tabs/data_view_tab.py)

```

from PyQt5.QtWidgets import QLabel, QComboBox, QPushButton, QTableWidgetItem, QTableWidgetItem,
QVBoxLayout, QHBoxLayout, QHeaderView
from .base_tab import BaseTab

class DataViewTab(BaseTab):
    def __init__(self, parent=None, db=None):
        super().__init__(parent, db)
        h_layout = QHBoxLayout()
        self.combo_entity = QComboBox()
        self.combo_entity.currentIndexChanged.connect(self.on_entity_changed)
        self.btn_refresh = QPushButton("Обновить")
        self.btn_refresh.clicked.connect(self.load_data)
        h_layout.addWidget(self.btn_refresh)
        self.layout.addLayout(h_layout)
        self.table = QTableWidgetItem()
        self.table.setSelectionBehavior(QTableWidgetItem.SelectRows)
        self.table.setSelectionMode(QTableWidgetItem.NoSelection)
        self.layout.addWidget(self.table)
        self.load_entity_types()
    def load_entity_types(self):
        rows = self.db.execute_query(sql)
        self.combo_entity.clear()
        for self.combo_entity.addItem(f"{{name}} ({{alias}})", eid)
    def on_entity_changed(self, index):
        self.load_data()
    def load_data(self):
        if not self.combo_entity.currentData():
            return
        sql = """SELECT calias FROM meta.entitytypes WHERE id=%s"""
        if not res:
            return
        alias = res[0][0]
        ext_table = f"ext.ext_{{alias.lower()}}"
        sql = """SELECT to_regclass(%s)"""
        if not res:
            return
        self.table.setRowCount(0)
        self.table.setColumnCount(0)
        return show_info(f"Таблица {ext_table} не найдена")
        sql fields = """SELECT c.fieldname, calias FROM meta.fields
        fields = self.db.execute_query(sql_fields, (entity_id,))
        self.table.setRowCount(0)
        self.table.setColumnCount(0)
        return show_info(f"У сущности нет полей")
        field_names = [f[0] for f in fields]
        select_cols = ["v.id as version_id", "v.cname as entity_name"] + [f"e.{{fn}}"] for fn in field_names
        sql = """SELECT {{f}} FROM {{v}} JOIN {{e}} ON v.id = e.instanceid AND v.cversionstatus = 'Active'
        LIMIT 1000"""
        rows = self.db.execute_query(sql, (entity_id,))
        headers = ["ID версии", "Наименование"] + field_aliases
        self.table.setColumnCount(len(headers))
        self.table.setRowCount(len(headers))
        for i in range(1, len(headers)):
            pos = self.table.rowCount()
            self.table.setRow(pos, headers[i])
            self.table.setItem(pos, i, item)

```



```

self.translator = LocalTranslator()
self.composition_manager = CompositionManager(db=self.db, parent_widget=self)
self.tabs = QTabWidget()
self.layout.addWidget(self.tabs)
self.doc_types_tab = self.create_doc_types_tab()
self.tabs.addTab(self.doc_types_tab, self.translator.tr('tab_document_types'))
self.roles_tab = self.create_roles_tab()
self.tabs.addTab(self.roles_tab, self.translator.tr('tab_composition_roles'))
self.composition_tab = self.create_composition_tab()
self.tabs.addTab(self.composition_tab, self.translator.tr('tab_master_detail'))
self.retranslate_ui()
def retranslate_ui(self):
    self.tabs.setTabText(0, self.translator.tr('tab_document_types'))
    self.tabs.setTabText(1, self.translator.tr('tab_composition_roles'))
    if hasattr(self.doc_crud, 'retranslate_ui'):
        self.doc_crud.retranslate_ui()
    if hasattr(self.comp_crud, 'retranslate_ui'):
        self.comp_crud.retranslate_ui()
    self.update_role_table_headers()
    self.update_comp_table_headers()
    self.load_doc_types()
    self.load_compositions()
def update_doc_tree_headers(self):
    self.doc_tree.setHeaderLabels([
        self.translator.tr('field_name'),
        self.translator.tr('field_alias'),
        self.translator.tr('label_hierarchical'),
        self.translator.tr('label_versioned')
    ])
def update_role_table_headers(self):
    self.role_table.setHorizontalHeaderLabels([
        self.translator.tr('field_id'),
        self.translator.tr('field_name')
    ])
def update_comp_table_headers(self):
    self.comp_table.setHorizontalHeaderLabels([
        self.translator.tr('field_id'),
        self.translator.tr('field_master'),
        self.translator.tr('field_role'),
        self.translator.tr('field_link_field')
    ])
def create_doc_types_tab(self):
    tab = QWidget()
    layout = QVBoxLayout(tab)
    layout.addWidget(self.doc_crud)
    self.doc_tree.setHeaderLabels([self.translator.tr('field_name'), self.translator.tr('field_alias'),
        self.translator.tr('label_hierarchical'), self.translator.tr('label_versioned')])
    self.doc_tree.itemDoubleClicked.connect(self.edit_doc_type)
    layout.addWidget(self.doc_crud)
    self.doc_crud.add_clicked.connect(self.add_doc_type)
    self.doc_crud.delete_clicked.connect(self.delete_doc_type)
    return tab
def create_roles_tab(self):
    tab = QWidget()
    layout = QVBoxLayout(tab)
    layout.addWidget(self.role_crud)
    self.role_table.setHorizontalHeaderLabels([self.translator.tr('field_id'),
        self.translator.tr('field_name')])
    self.role_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.role_table.setSelectionMode(QTableWidget.SelectRows)
    layout.addWidget(self.role_crud)
    self.role_crud.add_clicked.connect(self.add_role)
    self.role_crud.delete_clicked.connect(self.delete_role)
    return tab
def create_composition_tab(self):
    tab = QWidget()
    layout = QVBoxLayout(tab)
    layout.addWidget(self.comp_crud)
    self.comp_table.setHorizontalHeaderLabels([self.translator.tr('field_id'),
        self.translator.tr('field_master'),
        self.translator.tr('field_role')])
    self.comp_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.comp_table.setSelectionMode(QTableWidget.SelectRows)
    layout.addWidget(self.comp_crud)
    self.comp_crud.add_clicked.connect(self.add_composition)
    self.comp_crud.delete_clicked.connect(self.delete_composition)
    return tab
def load_doc_types(self):
    sql = FROM meta.entity_types
    WHERE class_id = %s
    ORDER BY class_id
    rows = self.db.execute_query(sql)
    for row in rows:
        item = QListWidgetItem(self.doc_tree)
        item.setText(0, row[0])
        item.setText(1, row[1])
        item.setText(2, self.translator.tr('yes') if row[2] else self.translator.tr('no'))
        item.setText(3, self.translator.tr('yes') if row[3] else self.translator.tr('no'))
        item.setData(0, Qt.UserRole, row[0])
def add_doc_type(self):
    dialog = DocumentTypeEditDialog(self, db=self.db)
    if dialog.exec_():
        sql = FROM meta.entity_types
        WHERE class_id = %s
        ORDER BY class_id
        self.db.execute_query(sql, data=[dialog.class_id, dialog.class_name])
        self.load_doc_types()
def edit_doc_type(self):
    item = self.doc_tree.currentItem()
    if not item:
        self.show_warning(self.translator.tr('warning_select_document_type'))
    eid = item.data(0, Qt.UserRole)
    sql = FROM meta.entity_types WHERE class_id = %s
    row = self.db.execute_query(sql, (eid,))
    if not row:
        return
    data = "name:" + row[0][0],
        "alias:" + row[0][1],

```

```

        "lshierarchy": row[0][2],
        "lisversioned": row[0][4]
    }
    dlg = DocumentTypeEditDialog(self, data, db=self.db)
    if dlg.exec_():
        new_data = dlg.get_data()
        sql = "UPDATE meta.entitytypes SET calias=%s, lisdependent=%s, lshierarchy=%s, lisversioned=%s WHERE id=%s" % (
            new_data["calias"], new_data["lisdependent"], new_data["lshierarchy"],
            new_data["lisversioned"], eid), fetch=False)
        self.load_doc_types()
    def delete_doc_type(self):
        if not self.doc_tree.currentItem():
            self.show_warning(self.translator.tr('warning_select_document_type'))
            return
        eid = self.doc_tree.currentItem().data(0, Qt.UserRole)
        if QMessageBox.question(self, self.translator.tr('confirm_delete'),
            self.translator.tr('delete document type'), QMessageBox.Yes) == QMessageBox.Yes:
            sql = "DELETE FROM meta.entitytypes WHERE id=%s" % eid
            self.db.execute_query(sql, (eid,), fetch=False)
            self.load_doc_types()
    def load_roles(self):
        sql = "SELECT calias, cname FROM meta.composition_roles ORDER BY id"
        rows = self.db.execute_query(sql)
        self.role_table.setRowCount(0)
        for row in rows:
            pos = self.role_table.rowCount()
            self.role_table.insertRow(pos)
            for col, val in enumerate(row):
                item = QTableWidgetItem(str(val))
                if col == 0:
                    self.role_table.setItem(pos, col, item)
                else:
                    self.role_table.setItem(pos, col, item)
    def add_role(self):
        dlg = CompositionRoleEditDialog(self)
        if dlg.exec_():
            data = dlg.get_data()
            self.db.execute_query("INSERT INTO meta.composition_roles (calias, cname, mcomment) VALUES (%s, %s, %s)" % (
                data["calias"], data["cname"], data["mcomment"]), fetch=False)
            self.load_roles()
    def edit_role(self):
        row = self.role_table.currentRow()
        if row < 0:
            self.show_warning(self.translator.tr('warning_select_role'))
            return
        item = self.role_table.item(row, 0)
        rid = item.data(Qt.UserRole)
        row_data = self.db.execute_query("SELECT calias, cname, mcomment FROM meta.composition_roles WHERE id=%s" % rid)
        if not row_data:
            return
        data = {
            "calias": row_data[0][0],
            "cname": row_data[0][1],
            "mcomment": row_data[0][2]
        }
        dlg = CompositionRoleEditDialog(self, data)
        if dlg.exec_():
            new_data = dlg.get_data()
            self.db.execute_query("UPDATE meta.composition_roles SET calias=%s, cname=%s, mcomment=%s WHERE id=%s" % (
                new_data["calias"], new_data["cname"], new_data["mcomment"], rid), fetch=False)
            self.load_roles()
    def delete_role(self):
        row = self.role_table.currentRow()
        if row < 0:
            self.show_warning(self.translator.tr('warning_select_role'))
            return
        item = self.role_table.item(row, 0)
        rid = item.data(Qt.UserRole)
        if QMessageBox.question(self, self.translator.tr('confirm_delete'),
            self.translator.tr('delete role'), QMessageBox.Yes) == QMessageBox.Yes:
            sql = "DELETE FROM meta.composition_roles WHERE id=%s" % rid
            self.db.execute_query(sql, (rid,), fetch=False)
            self.load_roles()
    def load_compositions(self):
        sql = "SELECT f.id, p.cname, ch.cname, r.cname, c.c_link_fieldname FROM meta.entitytypes f JOIN meta.entitytypes p ON f.parent_entityid = p.id JOIN meta.composition_roles r ON r.role_id = f.id JOIN meta.composition_roles c ON c.role_id = r.id"
        rows = self.db.execute_query(sql)
        self.comp_table.setRowCount(0)
        for row in rows:
            pos = self.comp_table.rowCount()
            self.comp_table.insertRow(pos)
            for col, val in enumerate(row):
                item = QTableWidgetItem(str(val))
                if col == 0:
                    self.comp_table.setItem(pos, col, item)
                else:
                    self.comp_table.setItem(pos, col, item)
    def get_doc_types_list(self):
        sql = "SELECT id, cname FROM meta.entitytypes WHERE class = 'DOC'"
        return self.db.execute_query(sql)
    def get_child_types_list(self):
        sql = "SELECT id, cname FROM meta.entitytypes ORDER BY cname"
        return self.db.execute_query(sql)
    def get_roles_list(self):
        sql = "SELECT id, cname FROM meta.composition_roles ORDER BY cname"
        return self.db.execute_query(sql)
    def add_composition(self):
        doc_types = self.get_doc_types_list()
        child_types = self.get_child_types_list()
        roles = self.get_roles_list()
        dlg = CompositionEditDialog(self, doc_types=doc_types, child_types=child_types, roles=roles, db=self.db)
        if dlg.exec_():
            data = dlg.get_data()
            success = self.composition_manager.add_composition(
                parent_entityid=data["parent_entityid"],
                child_entityid=data["child_entityid"],
                role_id=data["role_id"],
                c_link_fieldname=data["c_link_fieldname"],
                isortorder=data["isortorder"])
            if success:
                self.load_compositions()
    def edit_composition(self):
        row = self.comp_table.currentRow()
        if row < 0:
            self.show_warning(self.translator.tr('warning_select_composition'))
            return
        item = self.comp_table.item(row, 0)
        comp_id = item.data(Qt.UserRole)
        sql = "SELECT parent_entityid, child_entityid, role_id, c_link_fieldname, isortorder FROM meta.entitytypes WHERE id=%s" % comp_id
        row_data = self.db.execute_query(sql, (comp_id,))
        if not row_data:
            return
        data = {
            "parent_entityid": row_data[0][0],
            "child_entityid": row_data[0][1],
            "role_id": row_data[0][2],
            "c_link_fieldname": row_data[0][3],
            "isortorder": row_data[0][4]
        }
        doc_types = self.get_doc_types_list()
        child_types = self.get_child_types_list()
        roles = self.get_roles_list()
        dlg = CompositionEditDialog(self, data, doc_types, child_types, roles, db=self.db)
        if dlg.exec_():
            new_data = dlg.get_data()
            success = self.composition_manager.update_composition(
                parent_entityid=new_data["parent_entityid"],

```



```

left_w = QWidget()
self.left_panel = QLabel(left_w)
self.v.addWidget(self.left_panel) + self.translator.tr('entity_types') + "</b>")
self.v.addWidget(self.left_panel)
self.tree_entities.setHeaderLabels(self.translator.tr('field_system_name'), self.translator.tr('field_alias'))
self.tree_entities.itemClicked.connect(self.on_entity_selected)
self.selection_model(QAbstractItemView.ExtendedSelection)
splitter.addWidget(left_w)
# Правая панель: закладки
right_w = QWidget()
right_layout.setContentsMargins(0, 0, 0, 0)
self.tabs = QTabWidget()
right_layout.addWidget(self.tabs)
self.fields_tab = FieldsTab(self, db=self.db)
self.tabs.addTab(self.fields_tab, self.translator.tr('tab_fields'))
self.bu_tab = BUVisibilityTab(self, db=self.db)
self.tabs.addTab(self.bu_tab, self.translator.tr('tab_bu_visibility'))
self.composition_tab = CompositionTab(self, db=self.db)
self.tabs.addTab(self.composition_tab, self.translator.tr('tab_composition'))
self.routes_tab = RoutesTab(self, db=self.db)
self.tabs.addTab(self.routes_tab, self.translator.tr('tab_routes'))
self.import_tab = ImportTab(self, db=self.db)
self.tabs.addTab(self.import_tab, self.translator.tr('tab_import'))
splitter.setSizeHint(right_w)
self.ent_crud.add_clicked.connect(self.add_entity)
self.ent_crud.delete_clicked.connect(self.delete_selected_entities)
self.current_entity_id = None
self.refresh()
def retranslate_ui(self):
    # Обновление заголовков левой панели
    self.header.setText("<b>" + self.translator.tr('entity_types') + "</b>")
    self.tree_entities.setHeaderLabels(["", self.translator.tr('field_system_name'), self.translator.tr('field_alias')])
    # Заголовок правой панели
    self.tabs.setTabText(0, self.translator.tr('tab_fields'))
    self.tabs.setTabText(1, self.translator.tr('tab_bu_visibility'))
    self.tabs.setTabText(2, self.translator.tr('tab_composition'))
    self.tabs.setTabText(3, self.translator.tr('tab_routes'))
    self.tabs.setTabText(4, self.translator.tr('tab_import'))
    # Перевод закладок
    self.fields_tab.retranslate_ui()
    self.bu_tab.retranslate_ui()
    self.composition_tab.retranslate_ui()
    self.routes_tab.retranslate_ui()
    self.import_tab.retranslate_ui()
    self.tabs.setTabText(4, self.translator.tr('tab_import'))
def refresh(self):
    self._update_right_panels(None)
def load_entities(self):
    rows = self.db.execute_query("SELECT cname, alias, status_id FROM meta.entitytypes ORDER BY cname")
    self.tree_entities.clear()
    for row in rows:
        item = QtGui.QTreeWidgetItem([row[0], row[1], row[2]])
        item.setStatus(Qt.Checked if row[2] == 1 else Qt.Unchecked)
        item.setData(0, Qt.UserRole, row[0])
        item.setData(1, Qt.UserRole, row[1])
        item.setData(2, Qt.UserRole, row[2])
        self.tree_entities.addItem(item)
def on_entity_selected(self, item):
    self.current_entity_id = item.data(0, Qt.UserRole)
    self._update_right_panels(self.current_entity_id)
def _update_right_panels(self, entity_id):
    self.fields_tab.set_entity_id(entity_id)
    self.bu_tab.set_entity_id(entity_id)
    self.composition_tab.set_entity_id(entity_id)
    self.routes_tab.set_entity_id(entity_id)
    self.import_tab.set_entity_id(entity_id)
def add_entity(self):
    dialog = QDialogBox(self, db=self.db)
    dialog.exec_()
    data = dialog.get_data()
    (cname, alias, mcomment, cbaclass, lisdependent, lishierarchy, lisversioned, issecuritystrategy, class_id,
     status_id) = data
    self.db.execute_query("""INSERT INTO meta.entitytypes (cname, alias, mcomment, cbaclass, lisdependent, lishierarchy, lisversioned, issecuritystrategy, class_id, status_id) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s)""")
    self.load_entities()
if data['class_id'] is not None:
    class_alias = self.db.execute_query("""SELECT alias FROM meta.entity_classes WHERE id = %s""", (data['class_id'],))
    if class_alias and class_alias[0][0] == 'ADDRESS':
        self.add_address_fields(data['alias'])
def add_address_fields(self, entity_alias):
    self.db.execute_query("""SELECT id FROM meta.entitytypes WHERE alias = %s""", (entity_alias,))
    if not res:
        return
    for frame, ftype, alias, length, indexed in ADDRESS_FIELDS:
        self.db.execute_query("""SELECT id FROM meta.fields WHERE entitytypeid=%s AND cfieldname=%s""", (entity_id, frame))
        if not exists:
            self.db.execute_query("""INSERT INTO meta.fields (entitytypeid, cfieldname, ctype, alias, length, indexed, status_id) VALUES (%s, %s, %s, %s, %s, %s, %s)""")
            self.db.execute_procedure("meta.p_field_sync", (frame,))
            self.show_warning(self.translator.tr('warning_select_entity'))
            self.current_entity_id = entity_id
            self._update_right_panels(entity_id)
def edit_entity(self):
    if not self.tree_entities.currentItem():
        self.show_warning(self.translator.tr('warning_select_entity'))
    eid = self.tree_entities.currentItem().data(0, Qt.UserRole)
    status_id = item.data(0, Qt.UserRole + 1)
    sql = """SELECT cname, alias, mcomment, cbaclass, lisdependent, lishierarchy, lisversioned, issecuritystrategy, class_id FROM meta.entitytypes WHERE id=%s"""
    [pw, old_data] = self.db.execute_query(sql, (eid,))
    if not pw:
        old_data = {}

```



```

        "calias": row[0][1],
        "mcomment": row[0][2],
        "baseclass": row[0][4],
        "isdependent": row[0][4],
        "isversioned": row[0][6],
        "securitystrategy": row[0][7],
        "status_id": row[0][9]
    }
    read_only = (status_id == 2)
    if not old_exec_ok:
        return
    new_data = dlg.get_data()
    new_cname = new_data["cname"].strip().lower()
    if old_cname != new_cname and status_id != 2:
        reply = QMessageBox.question(
            self, self.translator.tr('confirm_rename_title'),
            QMessageBox.Yes | QMessageBox.No
        )
        if reply == QMessageBox.No:
            return
        check_sql = """
        WHERE table_schema=ext AND table_name=%s
        old_table = f"ext.{old_cname}"
        if table_exists:
            self.db.execute_query(check_sql, (old_table,))
            rename_sql = f"ALTER TABLE ext.{old_table} RENAME TO ext.{new_cname}"
            try:
                self.db.execute_query(rename_sql, fetch=False)
                self.show_info(self.translator.tr('info_table_renamed').format(old_table, new_cname))
            except Exception as e:
                return show_error(self.translator.tr('error_rename_table').format(e))
        else:
            self.show_warning(self.translator.tr('warning_table_not_found').format(old_table))
    elif old_cname == new_cname and status_id != 2:
        return
    sql = """UPDATE meta.entitytypes SET
    lishierarchy=%s, isversioned=%s, securitystrategy=%s, class_id=%s, status_id=%s
    WHERE id=%s""" % (new_data["lshierarchy"], new_data["calias"], new_data["mcomment"],
    new_data["baseclass"], new_data["isdependent"], new_data["lshierarchy"],
    new_data["class_id"], new_data["status_id"], eid), fetch=False)
    self.load_entities()
    if self.current_entity_id == eid:
        self.update_right_panels(eid)
    def delete_selected_entities(self):
        items = self.tree.entities.selectedItems()
        if self.show_warning(self.translator.tr('warning_select_entity')):
            return
        dev_ids = []
        for item in items:
            eid = item.data(0, Qt.UserRole)
            status_id = item.data(0, Qt.UserRole + 1)
            if dev_ids.append(eid)
        else:
            prod_names.append(item.text(1))
        if prod_names:
            self.show_error(self.translator.tr('error_prod_entity_delete').format(' '.join(prod_names)))
        return
        if not dev_ids:
            if QMessageBox.question(self, self.translator.tr('confirm_delete'),
            QMessageBox.Yes | QMessageBox.No) == QMessageBox.Yes:
                for eid in dev_ids:
                    self.db.execute_query("DELETE FROM meta.entitytypes WHERE id=%s" % (eid,)), fetch=False)
                self.load_entities()
                if self.current_entity_id in dev_ids:
                    self.current_entity_id = None

```

Файл: /home/sergey/Documents/configurate/tabs/entity_subtabs/__init__.py (относительно

```

# tabs/entity_subtabs/__init__.py
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QTableWidget, QTableWidgetItem,
from :composition_tab import CompositionTab
from :routes_tab import RoutesTab # раскомментируем
all_tabs = [
    'CompositionTab',
    'RoutesTab',
    'ImportTab',
]

```

Файл: /home/sergey/Documents/configurate/tabs/entity_subtabs/bu_visibility_tab.py (относительно

```

# tabs/entity_subtabs/bu_visibility_tab.py
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QTableWidget, QTableWidgetItem,
from PyQt5.QtCore import Qt
from database import DatabaseService
from lang.local_translator import LocalTranslator

class BUVisibilityTab(QWidget):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.translator = LocalTranslator()
        layout = QVBoxLayout(self)
        layout.setContentsMargins(0, 0, 0, 0)
        self.crud = CrudButtons(self, with_add=True, with_edit=False, with_delete=True)
        layout.addWidget(self.crud)
        self.table = QTableWidget(0, 4)
        self.table.setSelectionBehavior(QTableWidget.SelectRows)
        layout.addWidget(self.table)
        self.crud.add.clicked.connect(self.add_visibility)
        self.crud.delete.clicked.connect(self.delete_selected)
        self.retranslate_ui()
    def retranslate_ui(self):
        headers = [
            self.translator.tr('field_id'),
            self.translator.tr('field_name'),
            self.translator.tr('field_policy'),
        ]
        self.table.setHorizontalHeaderLabels(headers)
        if hasattr(self, 'crud'):

```



```

self.child_crud.retranslate_ui()
def create_parent_tab(self):
    layout = QVBoxLayout()
    layout.addWidget(QPushButton('add', with_add=True, with_edit=True, with_delete=True))
    self.parent_table = QTableWidget(0, 5)
    self.parent_table.setHorizontalHeaderLabels([
        self.translator.tr('field_id'),
        self.translator.tr('field_name'),
        self.translator.tr('field_order')
    ])
    self.parent_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.parent_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.parent_table.clicked.connect(lambda: self.edit_composition('parent'))
    self.parent_crud.add_clicked.connect(lambda: self.add_composition('parent'))
    self.parent_crud.delete_clicked.connect(lambda: self.delete_composition('parent'))
    return layout

def create_child_tab(self):
    layout = QVBoxLayout()
    layout.addWidget(QPushButton('add', with_add=False, with_edit=True, with_delete=True))
    self.child_table = QTableWidget(0, 5)
    self.child_table.setHorizontalHeaderLabels([
        self.translator.tr('field_id'),
        self.translator.tr('field_name'),
        self.translator.tr('field_order')
    ])
    self.child_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
    self.child_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.child_table.clicked.connect(lambda: self.edit_composition('child'))
    self.child_crud.add_clicked.connect(lambda: self.add_composition('child'))
    self.child_crud.delete_clicked.connect(lambda: self.delete_composition('child'))
    return layout

def set_entity_id(self, entity_id):
    self.current_entity_id = entity_id
    self.load_parent()

def load_parent(self):
    if not self.parent_table.setRowCount(0):
        return
    sql = """SELECT c.id, et.cname, cr.cname, c.c_link_fieldname, c.isortorder
    FROM meta.entitytypes et JOIN c.child_entityid = et.id
    JOIN meta.composition_roles cr ON cr.role_id = cr.id
    ORDER BY c.id"""
    rows = self.db.execute_query(sql, (self.current_entity_id,))
    for i, parent_cname in enumerate(rows):
        self.parent_table.setItem(i, 0, QTableWidgetItem(str(parent_cname)))
        self.parent_table.setItem(i, 1, QTableWidgetItem(str(parent_cname)))
        self.parent_table.setItem(i, 2, QTableWidgetItem(str(parent_cname)))
        self.parent_table.setItem(i, 3, QTableWidgetItem(str(parent_cname)))
        self.parent_table.setItem(i, 4, QTableWidgetItem(str(parent_cname)))
    self.parent_table.item(i, 0).setData(Qt.UserRole, parent_cname)

def load_child(self):
    if not self.child_table.setRowCount(0):
        return
    sql = """SELECT c.id, et.cname, cr.cname, c.c_link_fieldname, c.isortorder
    FROM meta.entitytypes et JOIN c.child_entityid = et.id
    JOIN meta.composition_roles cr ON cr.role_id = cr.id
    ORDER BY c.id"""
    rows = self.db.execute_query(sql, (self.current_entity_id,))
    for i, child_cname in enumerate(rows):
        self.child_table.setItem(i, 0, QTableWidgetItem(str(child_cname)))
        self.child_table.setItem(i, 1, QTableWidgetItem(str(child_cname)))
        self.child_table.setItem(i, 2, QTableWidgetItem(str(child_cname)))
        self.child_table.setItem(i, 3, QTableWidgetItem(str(child_cname)))
        self.child_table.setItem(i, 4, QTableWidgetItem(str(child_cname)))
    self.child_table.item(i, 0).setData(Qt.UserRole, child_cname)

def add_composition(self, role):
    if not self.current_entity_id:
        QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('warning_select_entity'))
        return
    dlg = QDialog(self)
    dlg.setWindowTitle(self.translator.tr('title_add_composition'))
    layout = QVBoxLayout()
    entities = self.db.execute_query("SELECT id, cname FROM meta.entitytypes WHERE id != %s ORDER BY
    cname", (self.current_entity_id,))
    for i, entity in enumerate(entities):
        child_combo.addItem("%s (%s)" % (entity[1], entity[0]))
    for i, role in enumerate(entities):
        role_combo.addItem("%s (%s)" % (role[1], role[0]))
    form = QDialogForm()
    form.addWidget(QLabel(self.translator.tr('field_role'), 'role_combo'))
    form.addWidget(QLabel(self.translator.tr('field_link_field'), 'field_edit'))
    form.addWidget(QLabel(self.translator.tr('field_order'), 'order_edit'))
    layout.addWidget(form)
    btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
    btn_box.accepted.connect(dlg.accept)
    btn_box.rejected.connect(dlg.reject)
    dlg.exec_()
    if dlg.exec_() == QDialog.Accepted:
        role_id = role_combo.currentData()
        field_id = field_edit.text()
        order = int(order_edit.text()) if order_edit.text().isdigit() else 10
        success = self.composition_manager.add_composition(
            parent_entity_id=self.current_entity_id,
            child_entity_id=child_id,
            role_id=role_id,
            link_field=field_id,
            sortorder=order
        )
        if success:
            self.load_parent()
        else:
            QMessageBox.information(self, self.translator.tr('info'), self.translator.tr('info_add_as_child_not_supported'))
    def edit_composition(self, mode):
        table = self.parent_table if mode == 'parent' else self.child_table
        selected = table.selectedItems()
        if not selected:
            QMessageBox.warning(self, self.translator.tr('warning'), self.translator.tr('warning_select_record'))
            return
        row = selected[0].row()
        sql = """SELECT c.id, et.cname, cr.cname, c.c_link_fieldname, c.isortorder
        FROM meta.entitytypes et JOIN c.child_entityid = et.id
        JOIN meta.composition_roles cr ON cr.role_id = cr.id
        ORDER BY c.id"""
        rows = self.db.execute_query(sql, (child_id,))

```



```

rows = self.db.execute_query(sql, (self.current_entity_id,))
self.table.setRowCount(rows)
for idx, row in enumerate(rows):
    self.table.insertRow(idx)
    chk_item = QTableWidgetItem()
    chk_item.setFlags(Qt.ItemIsUserCheckable | Qt.ItemIsEnabled)
    if row[1] == self.flags(CheckItemFlags() & ~Qt.ItemIsEnabled)
    chk_item.setFlags(CheckItemFlags() & ~Qt.ItemIsEnabled)
    chk_item.setCheckState(Qt.Unchecked)
    self.table.setData(Qt.UserRole, row[0])
    # Отдельные колонки (1..8) - item
    for c_idx in range(8):
        if c_idx == row[7]: # колонка RefID - показываем имя сущности
            val = row[7] if row[7] else
        else:
            val = row[c_idx]
        item = QTableWidgetItem(str(val) if val is not None else "")
        self.setData(Qt.UserRole, row[8])
        self.table.setItem(r_idx, c_idx + 1, item)

def add_field(self):
    if QMessageBox.warning(self, self.translator.tr('error'), self.translator.tr('warning_select_entity')):
        dlg = FieldEditDialog(db=self.db)
        if dlg.exec_():
            self.save_field(data)

def edit_field(self):
    selected = self.table.selectedItems()
    if QMessageBox.warning(self, self.translator.tr('error'), self.translator.tr('warning_select_field')):
        return
    row = selected[0].row()
    prod_name = self.table.item(row, 2).data(Qt.UserRole)
    data = []
    prod_name = self.table.item(row, 3).text()
    self.table.item(row, 4).text()
    self.table.item(row, 5).text()
    self.table.item(row, 6).text()
    self.table.item(row, 7).text()
    self.table.item(row, 8).text()
    chk_item = self.table.item(row, 0)
    read_only = (status_id == Qt.UserRole) if chk_item else 1
    dlg = FieldEditDialog(data, status_id=status_id, read_only_mode=read_only, db=self.db)
    if dlg.exec_():
        self.save_field(new_data, is_update=True, old_name=old_name)

def delete_selected_fields(self):
    prod_names = []
    dev_ids = []
    for item in self.table.selectedItems():
        if row not in selected_rows:
            selected_rows.append(row)
            stat_item = chk_item.data(Qt.UserRole) if chk_item else 1
            fname = self.table.item(row, 1).text()
            if stat_item:
                dev_ids.append((self.current_entity_id, fname))
            else:
                prod_names.append(fname)
    if prod_names:
        QMessageBox.critical(self, self.translator.tr('error'),
            self.translator.tr('error_prod_field_delete') + "\n" + ", ".join(prod_names))
    if not dev_ids:
        if QMessageBox.question(self, self.translator.tr('confirm_delete'),
            self.translator.tr('confirm_delete_fields') % len(dev_ids)):
            for eid, fname in dev_ids:
                QMessageBox.Yes | QMessageBox.No == QMessageBox.Yes:
                    self.db.execute_query("DELETE FROM meta.fields WHERE entitytypeid=%s AND cfieldname=%s",
                        self.load_fields(eid, fname), fetch=False)
    self.load_fields()

def save_field(self, field_data, is_update=False, old_name=None):
    entitytypeid = self.current_entity_id
    if QMessageBox.warning(self, self.translator.tr('error'), self.translator.tr('warning_select_entity')):
        return
    cfieldname = field_data[0]
    alias = field_data[1]
    mcomment = field_data[3]
    mlength = field_data[4]
    mindexed = field_data[5]
    ref_entitytypeid = field_data[7]
    status_id = field_data[8]
    try:
        if is_update:
            # Проверка, не изменилось ли имя поля? Если изменилось - нужно переименовать колонку в ext-таблице
            if old_name != cfieldname:
                sql = self.db.execute_query("SELECT cfieldname FROM meta.fields WHERE id = %s" % old_name)
                if res = self.db.execute_query(sql, table, (entitytypeid,)):
                    # Ренем колонку
                    rename_exec = self.db.execute_query("RENAME COLUMN {old_name} TO {cfieldname}" %
                        (old_name, cfieldname))
                # Обновляем meta.fields
                sql = self.db.execute_query("UPDATE meta.fields SET cfieldname=%s, alias=%s, mcomment=%s, mlength=%s, mindexed=%s, ref_entitytypeid=%s, status_id=%s WHERE entitytypeid=%s AND cfieldname=%s" %
                    (cfieldname, alias, mcomment, mlength, mindexed, ref_entitytypeid, status_id, entitytypeid, old_name), fetch=False)
            else:
                # Вставка нового поля
                sql = self.db.execute_query("INSERT INTO meta.fields (entitytypeid, cfieldname, cfieldtype, alias, mcomment, mlength, mindexed, ref_entitytypeid, status_id) VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s) RETURNING id" %
                    (entitytypeid, cfieldname, cfieldtype, alias, mcomment, mlength, mindexed, ref_entitytypeid, status_id))
                res = self.db.execute_query(sql, (entitytypeid, cfieldname, cfieldtype, alias, mcomment, mlength, mindexed, ref_entitytypeid, status_id))
                if res:
                    field_id = res[0][0]
                    # Связываем с ext-таблицей
                    self.db.execute_procedure('meta.p_field_sync', (field_id,))
        except Exception as e:
            QMessageBox.critical(self, self.translator.tr('error'), self.translator.tr('error_save_field') + str(e))

```

Файл: /home/sergey/Documents/configurate/tabs/entity_subtabs/import_engine.py (относи

```

# import tab.py
from PyQt5.QtWidgets import (QWidget, QVBoxLayout, QHBoxLayout, QFormLayout,
    QLabel, QComboBox, QLineEdit, QPushButton,
    QTableWidgetItem, QTableWidgetItemHeader, QHeaderView,
    QDialog, QDialogButtonBox, QGroupBox, QCheckBox,
    QDateWidget)
from PyQt5.QtCore import Qt
from database import DatabaseService
from import_model import SourceSettings, Mapping
from import_utils import load_source_settings

```



```
try:
    from dbread import DBF
except ImportError:
    DBF_AVAILABLE = False
else:
    DBF_AVAILABLE = True


class SourceCheckDialog(QDialog):
    """
    Диалог выбора источника и выбора полей для маппинга.
    """
    def __init__(self, parent=None, db=None, entity_id=None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.source_fields = [] # список полей из источника
        self.mapping_table = QTableWidget(10, 2) # таблица сопоставления полей
        self.setWindowTitle("Проверка источника и сопоставление полей")
        self.init_ui()

    def init_ui(self):
        layout = QVBoxLayout(self)
        # Выбор файла/URL и формата
        self.source_type_combo = OComboBox()
        self.source_type_combo.addItem("Файл", "f")
        self.source_type_combo.addItem("Ссылка", "l")
        self.source_type_combo.currentIndexChanged.connect(self.on_source_type_changed)
        form.addRow("Тип источника:", self.source_type_combo)
        self.source_path_edit = QLineEdit()
        self.btn_browse.clicked.connect(self.browse_file)
        path_layout.addWidget(self.source_path_edit)
        form.addRow("Путь к URL:", path_layout)
        self.format_combo = OComboBox()
        self.format_combo.addItem("JSON", "j")
        self.format_combo.addItem("XML", "x")
        self.format_combo.addItem("DBF", "d")
        form.addRow("Формат:", self.format_combo)
        self.id_field_combo = OComboBox()
        form.addRow("Поле-идентификатор (sourceid):", self.id_field_combo)
        self.btn_check = QPushButton("Проверить источник")
        form.addRow(self.btn_check)
        layout.addLayout(form)
        # Таблица сопоставления полей
        self.mapping_table.setHorizontalHeaderLabels(["Поле сущности", "Поле источника", "Ссылка", "Целевая сущность"])
        self.mapping_table.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
        layout.addWidget(self.mapping_table)
        # Кнопки OK/Cancel
        btn_box = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        btn_box.accepted.connect(self.accept)
        btn_box.rejected.connect(self.reject)
        layout.addWidget(btn_box)

    def on_source_type_changed(self, index):
        current_data = self.source_type_combo.currentData()
        if current_data == "f":
            self.btn_browse.setEnabled(True)
            self.source_path_edit.clear()
        else:
            self.btn_browse.setEnabled(False)

    def browse_file(self):
        path = QFileDialog.getOpenFileName(self, "Выберите файл источника")
        self.source_path_edit.setText(path)

    def check_source(self):
        source_type = self.source_type_combo.currentData()
        fmt = self.format_combo.currentData()
        if not path:
            QMessageBox.warning(self, "Ошибка", "Укажите путь к источнику")
            return
        try:
            if fmt == "j":
                self.source_fields = self.read_json_structure(path)
            elif fmt == "x":
                self.source_fields = self.read_xml_structure(path)
            elif fmt == "d":
                self.source_fields = self.read_dbf_structure(path)
            else:
                QMessageBox.critical(self, "Ошибка", "Неподдерживаемый формат")
        except Exception as e:
            QMessageBox.critical(self, "Ошибка", f"Не удалось прочитать источник: {str(e)}")
            return
        if not self.source_fields:
            QMessageBox.warning(self, "Предупреждение", "Не удалось получить поля источника")
            return
        # Обновляем комбобокс для идентификатора
        self.id_field_combo.clear()
        self.id_field_combo.addItem("-- не выбрано --", None)
        self.id_field_combo.setEnabled(True)
        # Получаем список полей сущности (системные + из meta.fields)
        entity_types = self.get_entity_types()
        for i in range(self.mapping_table.rowCount()):
            self.mapping_table.setItem(i, 0, entity_types[i])
            self.mapping_table.setItem(i, 1, self.source_fields[i])
            self.mapping_table.setItem(i, 2, "")
            self.mapping_table.setItem(i, 3, "")
            self.mapping_table.setItem(i, 4, "")
        # Колонка 2 - выпадающий список полей источника
        for row in range(self.mapping_table.rowCount()):
            combo = OComboBox()
            combo.addItem("-- не сопоставлено --", None)
            for field in self.source_fields:
                combo.addItem(field["name"], field["id"])
            self.mapping_table.setCellWidget(row, 2, combo)
        # Колонка 3 - флажок ссылки
        chk = QCheckBox()
        chk.stateChanged.connect(lambda state, r=row: self.on_reference_toggled(r, state))
        self.mapping_table.setCellWidget(row, 3, chk)
        # Колонка 4 - выпадающий список целевых сущностей (изначально disabled)
        ref_combo = OComboBox()
        ref_combo.setEnabled(False)
        ref_combo.addItem("-- не выбрано --", None)
        entity_types_list = self.get_entity_types_list()
        for et in entity_types_list:
            ref_combo.addItem(et["name"], et["id"])
        self.mapping_table.setCellWidget(row, 4, ref_combo)
        # Сохраняем имя поля для дальнейшего использования
        self.mapping_table.item(row, 0).setData(Qt.UserRole, field_name)
        QMessageBox.information(self, "Информация", f"Найдено {len(self.source_fields)} полей в источнике")

    def on_reference_toggled(self, row, state):
        ref_combo = self.mapping_table.cellWidget(row, 4)
        ref_combo.setEnabled(state == Qt.Checked)

    def read_json_structure(self, path):
        with open(path, encoding='utf-8') as f:
            data = json.load(f)
            if isinstance(data, dict) and len(data) > 0:
                return list(data.keys())
            else:
                return []

    def read_xml_structure(self, path):
        root = etree.parse(path)
        return [root.get('name')]

    def read_dbf_structure(self, path):
        dbf = DbfFileWrapper(path)
        return dbf.getFieldsNames()
```

```

        return [child.tag for child in root[0]]
    return []

def read_dbf_structure(self, path):
    if not os.path.exists(path):
        raise IOError("Библиотека dbfread не установлена")
    table = dbf.Table(path, ignore_missing_tables=True)
    return [field.name for field in table.fields]

def get_entity_fields(self):
    """Возвращает список полей сущности в формате [(field_name, display_name), ...]."""
    fields = []
    fields.append(('cname', 'Наименование (cname)'))
    fields.append(('parentinstanceid', 'Родитель (parentinstanceid)'))
    sql = "WHERE entity_id = %s ORDER BY isortorder"
    rows = self.db.execute_query(sql, (self.entity_id,))
    for fieldname, alias in rows:
        fields.append((fieldname, f"{{alias}} ({{fieldname}})"))
    return fields

def get_entity_types_list(self):
    return self.db.execute_query("FROM meta.entitytypes ORDER BY cname")

def get_mappings(self):
    """Возвращает список объектов Mapping."""
    mappings = []
    # source_id
    if self.id == self.id_field combo.currentData():
        mappings.append(Mapping(target_field='sourceid', source_field=id_field))
    for row in range(self.mapping_table.rowCount()):
        combo = self.mapping_table.cellWidget(row, 1)
        source = combo.currentData()
        if source is None:
            chk = self.mapping_table.cellWidget(row, 2)
            is_ref = (chk.isChecked() == Qt.Checked)
            ref_field_name = None
            if is_ref:
                ref_combo = self.mapping_table.cellWidget(row, 3)
                # Генерируем имя ссылаемого поля автоматически
                ref_field_name, ref_source = mapping.append(Mapping(
                    source_field=source,
                    reference=is_ref,
                    ref_entitytype_id=ref_entitytype_id,
                    ref_field_name=ref_field_name
                ))
    return mappings

class ImportTab(QWidget):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.current_entity_id = None # id записи в import_sources для текущей сущности
        layout = QVBoxLayout(self)
        layout.setContentsMargins(5, 5, 5, 5)
        # Группа параметров источника
        source_layout = QFormLayout(self.source_group)
        self.source_name_edit = QLineEdit()
        source_layout.addRow("Название источника:", self.source_name_edit)
        self.source_type_combo = QComboBox()
        self.source_type_combo.addItem("Файл")
        self.source_type_combo.addItem("База данных")
        self.source_type_combo.addItem("Web")
        source_layout.addRow("Тип:", self.source_type_combo)
        self.source_path_edit = QLineEdit()
        self.btn_browse.clicked.connect(self.browse_file)
        path_layout = QHBoxLayout()
        path_layout.addWidget(self.source_path_edit)
        source_layout.addRow("Путь/URL:", path_layout)
        self.format_combo = QComboBox()
        self.format_combo.addItem("CSV")
        self.format_combo.addItem("DBF")
        source_layout.addRow("Формат:", self.format_combo)
        self.btn_check = QPushButton("Проверить и Настроить сопоставление")
        source_layout.addRow(self.btn_check)
        layout.addWidget(self.source_group)
        # Кнопки сохранения и импорта
        self.btn_save = QPushButton("Сохранить настройки")
        self.btn_import = QPushButton("Запустить импорт")
        self.btn_refresh = QPushButton("Обновить")
        btn_layout = QHBoxLayout()
        btn_layout.addWidget(self.btn_save)
        btn_layout.addWidget(self.btn_import)
        btn_layout.addWidget(self.btn_refresh)
        layout.addLayout(btn_layout)
        # Статус последнего импорта
        self.status_label = QLabel("Последний импорт: никогда")
        layout.addWidget(self.status_label)
        layout.addStretch()

    def on_source_type_changed(self, index):
        self.btn_browse.setEnabled(index == 'Файл')
        if self.source_path_edit.clear()

    def browse_file(self):
        path = QFileDialog.getOpenFileName(self, "Выберите файл источника")
        if path:
            self.source_path_edit.setText(path)

    def set_entity_id(self, entity_id):
        self.current_entity_id = entity_id
        self.load_settings()

    def load_settings(self):
        """Загружает настройки импорта для текущей сущности."""
        if not self.current_entity_id:
            return
        sql = "SELECT id, source_name, source_type, source_path, format, id_field, last_import"
        rows = self.db.execute_query(sql, (self.current_entity_id,))
        if rows:
            self.source_id = rows[0][0]
            self.source_name = rows[0][1]
            self.source_type = rows[0][2]
            self.source_path = rows[0][3]
            self.format_combo.setCurrentIndex(self.format_combo.findData(rows[0][4]))
            self.status_label.setText(f"Последний импорт: {rows[0][6]}")
        else:
            self.status_label.setText("Последний импорт: никогда")
        self.source_id = None
        self.source_name_edit.clear()
        self.source_path_edit.setCurrentIndex(0)
        self.status_label.setText("Последний импорт: никогда")

    def check_source(self):
        if not self.current_entity_id:
            QMessageBox.warning(self, "Внимание", "Сначала выберите сущность")

```



```

from dbfread import DBF
DBF_AVAILABLE = True
except ImportError:
    DBF_AVAILABLE = False

class BaseReader(ABC):
    """Абстрактный читатель источника данных."""
    @abstractmethod
    def get_fields(self) -> List[str]:
        """Возвращает список имен полей в источнике."""
        pass
    @abstractmethod
    def iter_(self) -> Iterator[Dict[str, Any]]:
        """Итератор по записям, каждая запись - словарь поле->значение."""
        pass

class JsonReader(BaseReader):
    def __init__(self, path: str):
        self.path = path
        with open(path, 'r', encoding='utf-8') as f:
            if isinstance(data, list):
                self.records = data
            else:
                self.records = [data]
    def get_fields(self) -> List[str]:
        if self.records:
            return list(self.records[0].keys())
        return []
    def iter_(self) -> Iterator[Dict[str, Any]]:
        for rec in self.records:
            yield rec

class XmlReader(BaseReader):
    def __init__(self, path: str):
        self.path = path
        self.root = etree.parse(path)
        # Предполагаем, что записи - это дочерние элементы корня
        self.records = list(self.root)
    def get_fields(self) -> List[str]:
        if self.records:
            # Собираем все возможные теги из первого элемента
            return [child.tag for child in self.records[0]]
        return []
    def iter_(self) -> Iterator[Dict[str, Any]]:
        for elem in self.records:
            yield {child.tag: child.text for child in elem}

class DbfReader(BaseReader):
    def __init__(self, path: str):
        if not DBF_AVAILABLE:
            raise ImportError("Библиотека dbfread не установлена. Установите: pip install dbfread")
        self.records = list(self.table)
    def get_fields(self) -> List[str]:
        return [field.name for field in self.table.fields]
    def iter_(self) -> Iterator[Dict[str, Any]]:
        for rec in self.records:
            yield dict(rec)

```

Файл: /home/sergey/Documents/configurate/tabs/entity_subtabs/import_tab.py (относительный)

```

# tabs/entity_subtabs/import_tab.py
# coding: utf-8
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QPushButton
from database import DatabaseService
from lang import Translator
from mozar import dialogs.profile_manager_dialog import ProfileManagerDialog

class ImportTab(QWidget):
    """Вкладка импорта для конкретной сущности."""
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.translator = Translator()
        self.current_entity_id = None
        layout = QVBoxLayout(self)
        layout.setContentsMargins(5, 5, 5, 5)
        self.btn_manage = QPushButton(self.translator.tr('btn_start_import_designer'))
        layout.addWidget(self.btn_manage)
        self.btn_manage.clicked.connect(self.open_profile_manager)
        self.retranslate_ui()
    def retranslate_ui(self):
        self.btn_manage.setText(self.translator.tr('btn_start_import_designer'))
    def set_entity_id(self, entity_id):
        self.current_entity_id = entity_id
    def open_profile_manager(self):
        """Открывает диалог управления профилями импорта."""
        dlg = ProfileManagerDialog(self.db, self)
        dlg.exec_()

```

Файл: /home/sergey/Documents/configurate/tabs/entity_subtabs/import_utils.py (относительный)

```

# import_utils.py
from typing import Dict, Optional, List, Tuple

def get_entity_cname(db: DatabaseService, entitytype_id: int) -> Optional[str]:
    """Возвращает системное имя сущности (tablename) по entitytype_id"""
    res = db.execute_query("SELECT cname FROM meta.entitytypes WHERE id=%s", (entitytype_id,))
    return res[0][0] if res else None

def ensure_sourceid_field(db: DatabaseService, entitytype_id: int, field_type: str = 'C', length: int = 100) -> None:
    """Проверяет наличие поля sourceid в ext-таблице и в meta.fields, создаёт при необходимости.
    # Проверяем в meta.fields
    (entitytype_id,) = db.execute_query("SELECT id FROM meta.entitytypes WHERE id=%s", (entitytype_id,))
    if not res:
        # Создаём поле
        sql = "ALTER TABLE %s ADD COLUMN sourceid %s(%s) NOT NULL" % (entitytype_id, field_type, length)
        new_id = db.execute_query(sql, (entitytype_id, field_type, length))
        if new_id:
            # Выполняем синхронизацию
            db.execute_query("CALL %s", (new_id[0][0],))
    # Поле уже существует - ничего не делаем

def load_reference_mappings(db: DatabaseService, ref_entitytype_id: int) -> Dict[str, int]:
    """Загружает mapping sourceid -> instanceid для указанной сущности.
    Возвращает словарь {sourceid: instanceid}.
    cname = get_entity_cname(db, ref_entitytype_id)
    if not cname:
        return {}
    ext_table = f"ext.ext_{cname}"
    sql = f"SELECT sourceid, instanceid FROM {ext_table}"

```

```

SELECT e.sourceid, v.instanceid
FROM text_table t, e
JOIN data_entity_versions v ON v.id = e.versionid
WHERE e.sourceid IS NOT NULL
"""
rows = db.execute_query(sql)
return [str(row[0]) for row in rows]

def load_source_settings(db: DatabaseService, import_source_id: int) -> Optional['SourceSettings']:
    """Загружает настройки из БД в объект SourceSettings.
    from import_models import SourceSettings, Mapping
    # Загружаем основную запись
    sql_src = FROM meta import sources WHERE id = %s
    src_row = db.execute_query(sql_src, (import_source_id,))
    if not src_row:
        return None
    source_type, source_path, fmt, source_name, id_field = src_row[0]
    # Загружаем маппинги
    sql_map = FROM meta import mappings
    WHERE import_source_id = %s
    map_rows = db.execute_query(sql_map, (import_source_id,))
    mappings = []
    for row in map_rows:
        target_source_id, ref, ref_etype, ref_fname = row
        mapping = Mapping(
            source_id=source,
            is_reference=True,
            ref_entitytype_id=ref_etype,
            ref_field_name=ref_fname
        )
    return SourceSettings(
        source_type=source_type,
        source_path=source_path,
        format=fmt,
        source_name=source_name,
        id_field=id_field,
        mappings=mappings
    )

```

Файл: /home/sergey/Documents/configurate/tabs/entity_subtabs/routes_tab.py (относительно: tabs/entity_subtabs/routes_tab.py)

```

# tabs/entity_subtabs/routes_tab.py
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QLabel
from database import DatabaseService
from lang.local_translator import LocalTranslator

class RoutesTab(QWidget):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.db = db or DatabaseService()
        self.translator = LocalTranslator()
        self.current_entity_id = None
        layout = QVBoxLayout(self)
        label = QLabel(self.translator.tr('routes_in_development'))
        layout.addWidget(self.label_status)
        self.retranslate_ui()

    def retranslate_ui(self):
        self.label_status.setText(self.translator.tr('routes_in_development'))

    def set_entity_id(self, entity_id):
        self.current_entity_id = entity_id

```

Файл: /home/sergey/Documents/configurate/tabs/relations_tab.py (относительно: tabs/relations_tab.py)

```

# tabs/relations_tab.py
from PyQt5.QtWidgets import QWidget, QTabWidget, QTreeWidget, QTreeWidgetItem, QTableWidgetItem,
    QMessageBox, QDialog, QFormLayout, QComboBox, QTableWidget,
    QLineEdit, QVBoxLayout,
    QPushButton, QLabel, QHeaderView
from PyQt5.QtCore import Qt
from crud_buttons import CrudButtons

class RefTypeEditDialog(QDialog):
    def __init__(self, parent=None, data=None):
        super().__init__(parent)
        self.setWindowTitle(Тип связи")
        layout = QVBoxLayout(self)
        form = QFormLayout(self)
        self.edit_name = QLineEdit(self.data.get("cname", ""))
        self.edit_comment = QLineEdit(self.data.get("comment", ""))
        form.addRow("Наименование:", self.edit_name)
        form.addRow("Комментарий:", self.edit_comment)
        layout.addLayout(form)
        btns = QHBoxLayout()
        btn_ok = QPushButton("OK")
        btn_cancel = QPushButton("Отмена")
        btns.addWidget(btn_ok)
        btns.addWidget(btn_cancel)
        layout.addLayout(btns)

    def get_data(self):
        return {
            "cname": self.edit_name.text(),
            "comment": self.edit_comment.text()
        }

class RefRuleEditDialog(QDialog):
    def __init__(self, parent=None, data=None, ref_types=None, entity_types=None):
        super().__init__(parent)
        self.setWindowTitle(Правило связи")
        self.data = data
        layout = QVBoxLayout(self)
        form = QFormLayout()
        self.combo_ref_type = QComboBox()
        for tid, name in ref_types.items():
            self.combo_ref_type.addItem(f"({tid})", tid)
        self.combo_ref_type.setCurrentIndex(0)
        self.combo_entity_type = QComboBox()
        for eid, name in entity_types.items():
            self.combo_entity_type.addItem(f"({eid})", eid)
        self.combo_role_source = QComboBox()
        for eid, name in entity_types.items():
            self.combo_role_source.addItem(f"({eid})", eid)
        if data:
            self.combo_ref_type.setCurrentIndex(self.combo_ref_type.findData(data.get("reftypeid")))
            self.combo_entity_type.setCurrentIndex(self.combo_entity_type.findData(data.get("entitytypeid")))
            self.combo_role_source.setCurrentIndex(self.combo_role_source.findData(data.get("role_source_entityid")))
        form.addRow("Тип связи:", self.combo_ref_type)
        form.addRow("Источник полей:", self.combo_role_source)

```



```

layout.addLayout(form)
btns = QHBoxLayout()
btn_ok = QPushButton("OK")
btn_cancel = QPushButton("Отмена")
btn_cancel.clicked.connect(self.reject)
btns.addWidget(btn_ok)
btns.addWidget(btn_cancel)
layout.addLayout(btns)
def get_data(self):
    return {
        "ref_type_id": self.combo_ref_type.currentData(),
        "role_source_entity": self.combo_entity.currentData(),
        "role_source_entity": self.combo_role_source.currentData()
    }
class RelationsTab(BaseTab):
def __init__(self, parent=None, db=None):
    super().__init__(parent, db)
    self.tabs = QWidget()
    self.layout.addWidget(self.tabs)
    # Вкладка "Типы связей"
    self.tabs.addTab(self.create_types_tab(), "Типы связей")
    # Вкладка "Правила"
    self.tabs.addTab(self.create_rules_tab(), "Правила")
    # Вкладка "Экземпляры связей" (заглушка)
    self.tabs.addTab(self.create_instances_tab(), "Экземпляры связей")
def create_types_tab(self):
    tab = QWidget()
    layout = QVBoxLayout(tab)
    layout.addWidget(self.type_crud)
    layout.addWidget(self.type_table)
    # Заголовки для таблицы
    self.type_table.setHorizontalHeaderLabels(["ID", "Алиас", "Наименование"])
    # Соединения для таблицы
    self.type_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.type_table.clicked.connect(self.edit_type)
    self.type_crud.add_clicked.connect(self.add_type)
    self.type_crud.delete_clicked.connect(self.delete_type)
    return tab
def create_rules_tab(self):
    tab = QWidget()
    layout = QVBoxLayout(tab)
    layout.addWidget(self.rule_crud)
    layout.addWidget(self.rule_table)
    # Заголовки для таблицы
    self.rule_table.setHorizontalHeaderLabels(["ID", "Тип связи", "Тип участника", "Источник ролей"])
    # Соединения для таблицы
    self.rule_table.setSelectionBehavior(QTableWidget.SelectRows)
    self.rule_table.clicked.connect(self.edit_rule)
    self.rule_crud.add_clicked.connect(self.add_rule)
    self.rule_crud.delete_clicked.connect(self.delete_rule)
    return tab
def create_instances_tab(self):
    tab = QWidget()
    layout = QVBoxLayout(tab)
    return tab
# r - Типы связей
def load_types(self):
    sql = "SELECT id, alias, cname FROM meta.entity_ref_types ORDER BY id"
    rows = self.db.execute_query(sql)
    self.type_table.setRowCount(0)
    for row in rows:
        pos = self.type_table.rowCount()
        self.type_table.insertRow(pos)
        if pos == 0:
            self.type_table.setItem(pos, 0, QTableWidgetItem(str(val)))
            self.type_table.setItem(pos, 1, QTableWidgetItem(str(val)))
        self.type_table.setItem(pos, 2, QTableWidgetItem(str(val)))
def add_type(self):
    dlg = RefTypeEditDialog(self)
    if dlg.exec_():
        data = dlg.get_data()
        self.db.execute_query(sql, (data["alias"], data["cname"], data["mcomment"],), fetch=False)
        self.load_types()
def edit_type(self):
    row = self.type_table.currentRow()
    if row < 0:
        self.show_warning("Выберите тип связи")
        return
    item = self.type_table.item(row, 0)
    tid = item.data(Qt.UserRole, "mcomment")
    sql = "UPDATE meta.entity_ref_types SET alias=%s, cname=%s, mcomment=%s WHERE id=%s"
    if not row_data:
        return
    data["alias"] = row_data[0][0]
    data["cname"] = row_data[0][1]
    data["mcomment"] = row_data[0][2]
    dlg = RefTypeEditDialog(self, data)
    if dlg.exec_():
        new_data = dlg.get_data()
        self.db.execute_query(sql, (new_data["alias"], new_data["cname"], new_data["mcomment"], tid), fetch=False)
        self.load_types()
def delete_type(self):
    row = self.type_table.currentRow()
    if row < 0:
        self.show_warning("Выберите тип связи")
        return
    item = self.type_table.item(row, 0)
    tid = item.data(Qt.UserRole, "mcomment")
    if QMessageBox.question(self, "Подтверждения", "Удалить тип связи?") == QMessageBox.Yes:
        self.db.execute_query(sql, (tid,), fetch=False)
        self.load_types()
# r - Правила
def load_rules(self):
    sql = "SELECT id, r.cname, et.cname FROM meta.entity_ref_types r JOIN meta.entity_ref_types et ON r.ref_type_id = et.id JOIN meta.entity_types rs ON r.role_source_entity_id = rs.id"
    rows = self.db.execute_query(sql)
    self.rule_table.setRowCount(0)
    for row in rows:
        pos = self.rule_table.rowCount()
        self.rule_table.insertRow(pos)
        if pos == 0:
            self.rule_table.setItem(pos, 0, QTableWidgetItem(str(val)))
            self.rule_table.setItem(pos, 1, QTableWidgetItem(str(val)))
            self.rule_table.setItem(pos, 2, QTableWidgetItem(str(val)))
        self.rule_table.setItem(pos, 3, QTableWidgetItem(str(val)))
def get_ref_types_list(self):
    return self.db.execute_query(sql)

```



```

self.data = data or {}
layout = QVBoxLayout(self)
form = QFormLayout(self)
self.edit_login = QLineEdit(self.data.get("login", ""))
self.edit_pass = QLineEdit(self.data.get("password", ""))
self.chk_active = QCheckBox(self.translator.tr('label_active'))
self.edit_person_instanceid = self.data.get('person_instanceid', "")
form.addRow(self.translator.tr('field_password_hash'), self.edit_pass)
form.addRow(self.translator.tr('field_person_instanceid'), self.edit_person_instanceid)
form.addRow(self.translator.tr('field_active'), self.chk_active)
layout.addLayout(form)

btns = QHBoxLayout()
btn_ok = QPushButton(self.translator.tr('btn_ok'))
btn_cancel = QPushButton(self.translator.tr('btn_cancel'))
btn_cancel.clicked.connect(self.reject)
btns.addWidget(btn_ok)
btns.addWidget(btn_cancel)
layout.addLayout(btns)

def get_data(self):
    return {
        "login": self.edit_login.text(),
        "password": self.edit_pass.text(),
        "chk_active": self.chk_active.isChecked(),
        "person_instanceid": self.edit_person_instanceid.text() or None
    }

class RoleEditDialog(QDialog):
    def __init__(self, parent=None, data=None, db=None):
        super().__init__(parent)
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('title_role_edit'))
        self.data = data or {}
        layout = QVBoxLayout(self)
        form = QFormLayout(self)
        self.edit_name = QLineEdit(self.data.get("name", ""))
        self.edit_alias = QLineEdit(self.data.get("alias", ""))
        # Ссылка на родительскую роль
        self.parent_role = auth_roles_editor(
            displayfield="name",
            db=self.db
        )
        if self.data.get("parentid"):
            sql = self.db.execute_query(
                "SELECT name FROM auth_roles WHERE id = %s" % self.data["parentid"]
            )
            if self.db.execute_query(sql, (self.data["parentid"],)):
                self.ref_parent.set_value(self.data["parentid"], res[0][0])
        self.chk_interface = QCheckBox(self.translator.tr('label_interface_role'))
        self.edit_path = QLineEdit(self.data.get("interface_path", ""))
        self.edit_comment = QLineEdit(self.data.get("comment", ""))
        form.addRow(self.translator.tr('field_name'), self.edit_name)
        form.addRow(self.translator.tr('field_alias'), self.edit_alias)
        form.addRow(self.translator.tr('field_parent_role'), self.ref_parent)
        form.addRow(self.translator.tr('field_interface'), self.chk_interface)
        form.addRow(self.translator.tr('field_path'), self.edit_path)
        form.addRow(self.translator.tr('field_comment'), self.edit_comment)
        layout.addLayout(form)

        btns = QHBoxLayout()
        btn_ok = QPushButton(self.translator.tr('btn_ok'))
        btn_cancel = QPushButton(self.translator.tr('btn_cancel'))
        btn_cancel.clicked.connect(self.reject)
        btns.addWidget(btn_ok)
        btns.addWidget(btn_cancel)
        layout.addLayout(btns)

    def get_data(self):
        return {
            "name": self.edit_name.text(),
            "alias": self.edit_alias.text(),
            "parent_role": self.ref_parent.get_value(),
            "interface": self.chk_interface.isChecked(),
            "interface_path": self.edit_path.text(),
            "comment": self.edit_comment.text()
        }

class ContextEditDialog(QDialog):
    def __init__(self, parent=None, users=None, roles=None, bus=None):
        super().__init__(parent)
        self.translator = LocalTranslator()
        self.setWindowTitle(self.translator.tr('title_context_assign'))
        form = QFormLayout(self)
        self.combo_user = QComboBox()
        for user in users:
            self.combo_user.addItem(f"{user['login']} ({user['uid']})", user['uid'])
        self.combo_role = QComboBox()
        for role in roles:
            self.combo_role.addItem(f"{role['name']} ({role['rid']})", role['rid'])
        self.combo_bus = QComboBox()
        for bus in bus:
            self.combo_bus.addItem(f"{bus['name']} ({bus['bid']})", bus['bid'])
        form.addRow(self.translator.tr('field_user'), self.combo_user)
        form.addRow(self.translator.tr('field_role'), self.combo_role)
        form.addRow(self.translator.tr('field_balance_unit'), self.combo_bus)
        layout.addLayout(form)

        btns = QHBoxLayout()
        btn_ok = QPushButton(self.translator.tr('btn_ok'))
        btn_cancel = QPushButton(self.translator.tr('btn_cancel'))
        btn_cancel.clicked.connect(self.reject)
        btns.addWidget(btn_ok)
        btns.addWidget(btn_cancel)
        layout.addLayout(btns)

    def get_data(self):
        return {
            "userid": self.combo_user.currentData(),
            "roleid": self.combo_role.currentData(),
            "balanceunitid": self.combo_bus.currentData()
        }

class UsersRolesTab(BaseTab):
    def __init__(self, parent=None, db=None):
        super().__init__(parent)
        self.tabs = QTabWidget()
        self.layout.addWidget(self.tabs)
        # Вкладка "Пользователи"
        self.users_tab = self.create_users_tab()
        self.tabs.addTab(self.users_tab, self.translator.tr('tab_users'))
        # Вкладка "Роли"
        self.roles_tab = self.create_roles_tab()
        self.tabs.addTab(self.roles_tab, self.translator.tr('tab_roles'))
        # Вкладка "Контексты (назначения)"
        self.context_tab = self.create_context_tab()
        self.tabs.addTab(self.context_tab, self.translator.tr('tab_context'))
        self.retranslate_ui()

    def retranslate_ui(self):
        # Обновляем заголовки внутренних вкладок
        self.tabs.setTabText(0, self.translator.tr('tab_users'))
        self.tabs.setTabText(2, self.translator.tr('tab_context'))

```

```

# Обновляем заголовки таблиц и деревьев
self.update_roles_tree_headers()
self.update_context_table_headers()

# Обновляем кнопки CRUD
if hasattr(self, crud.translate_ui()):
    if hasattr(self, crud.translate_ui()):
        self.context.crud.retranslate_ui()

def update_users_table_headers(self):
    self.user_table.setHorizontalHeaderLabels([
        self.translator.tr('field_login'),
        self.translator.tr('field_active'),
        self.translator.tr('field_person_instance')
    ])

def update_roles_tree_headers(self):
    self.role_tree.setHorizontalHeaderLabels([
        self.translator.tr('field_name'),
        self.translator.tr('field_interface_path')
    ])

def update_context_table_headers(self):
    self.context_table.setHorizontalHeaderLabels([
        self.translator.tr('field_user'),
        self.translator.tr('field_balance_unit')
    ])

def create_users_tab(self):
    layout = QVBoxLayout(tab)
    widget = QWidget(tab)
    widget.setObjectName('UsersTab')
    widget.setStyleSheet('background-color: #f0f0f0;')
    self.user_table.setHorizontalHeaderLabels([
        self.translator.tr('field_login'),
        self.translator.tr('field_person_instance')
    ])
    self.user_table.horizontalHeader().setStretchLastSection(True)
    widget.addWidget(self.user_table)
    widget.addWidget(self.add_clicked.connect(self.add_user))
    widget.addWidget(self.delete_clicked.connect(self.delete_user))
    return tab

def create_roles_tab(self):
    layout = QVBoxLayout(tab)
    widget = QWidget(tab)
    widget.setObjectName('RolesTab')
    widget.setStyleSheet('background-color: #f0f0f0;')
    self.role_tree.setHorizontalHeaderLabels([
        self.translator.tr('field_name'),
        self.translator.tr('field_interface_path')
    ])
    self.role_tree.itemDoubleClicked.connect(self.edit_role)
    widget.addWidget(self.role_tree)
    widget.addWidget(self.add_clicked.connect(self.add_role))
    widget.addWidget(self.delete_clicked.connect(self.delete_role))
    return tab

def create_context_tab(self):
    layout = QVBoxLayout(tab)
    widget = QWidget(tab)
    widget.setObjectName('ContextTab')
    widget.setStyleSheet('background-color: #f0f0f0;')
    self.context_table.setHorizontalHeaderLabels([
        self.translator.tr('field_user'),
        self.translator.tr('field_balance_unit')
    ])
    self.context_table.horizontalHeader().setStretchLastSection(True)
    widget.addWidget(self.context_table)
    widget.addWidget(self.add_clicked.connect(self.add_context))
    widget.addWidget(self.delete_clicked.connect(self.delete_context))
    return tab

# Загрузка данных
def load_users(self):
    sql = 'SELECT u.clogin, u.lactive, u.person_instanceid FROM auth.users ORDER BY id'
    rows = self.db.execute_query(sql, ('auth.users', 'id', 'person_instanceid'))
    for row in rows:
        pos = self.user_table.rowCount()
        for col, val in enumerate(row):
            item = QTableWidgetItem(str(val) if val is not None else '')
            self.user_table.setItem(pos, col, item)
        self.user_table.setItem(pos, 0, item)

def load_roles(self):
    sql = 'SELECT r.parentid, r.level, r.data FROM get_hierarchy_iterative(%s, %s, %s)'
    rows = self.db.execute_query(sql, ('auth.roles', 'id', 'parentid'))
    self.role_tree.clear()
    for row in rows:
        node_id, parent_id, level, data = row
        alias = data.get('alias')
        interface_path = data.get('interface_path', False)
        item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
        if item:
            item.setText(0, alias)
        else:
            item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
            if item:
                item.setText(0, alias)
            else:
                item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
                if item:
                    item.setText(0, alias)
                else:
                    item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
                    if item:
                        item.setText(0, alias)
                    else:
                        item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
                        if item:
                            item.setText(0, alias)
                        else:
                            item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
                            if item:
                                item.setText(0, alias)
                            else:
                                item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
                                if item:
                                    item.setText(0, alias)
                                else:
                                    item = self.role_tree.item(self.role_tree.indexOfTopLevelItemFromPath(interface_path, alias))
                                    if item:
                                        item.setText(0, alias)
                                        self.role_tree.addTopLevelItem(item)
                                    else:
                                        self.role_tree.addTopLevelItem(item)
                                else:
                                    self.role_tree.addTopLevelItem(item)
                            else:
                                self.role_tree.addTopLevelItem(item)
                        else:
                            self.role_tree.addTopLevelItem(item)
                    else:
                        self.role_tree.addTopLevelItem(item)
                else:
                    self.role_tree.addTopLevelItem(item)
            else:
                self.role_tree.addTopLevelItem(item)
        else:
            self.role_tree.addTopLevelItem(item)
    self.role_tree.expandAll()

def load_context(self):
    sql = 'FROM auth u, self pr context urc u clogin, r cname, bu cname'
    sql = 'JOIN auth.users u ON urc.userid = u.id'
    sql = 'JOIN auth.roles r ON urc.roleid = r.id'
    sql = 'JOIN auth.balanceunits bu ON urc.balanceunitid = bu.id'
    rows = self.db.execute_query(sql)

```

```

self.context_table.setRowCount(0)
for row in rows:
    pos = self.context_table.rowCount()
    self.context_table.insertRow(pos)
    item = QTableWidgetItem(str(val))
    if col == Qt.UserRole:
        self.context_table.setItem(pos, col, item)

# CRUD для пользователей
def add_user(self):
    dlg = UserAddDialog(self)
    if dlg.exec_():
        data = dlg.get_data()
        sql = """INSERT INTO auth.users (clogin, cpassword_hash, lactive, person_instanceid)
VALUES (%s, %s, %s, %s)"""
self.db.execute_query(sql, (data["clogin"], data["cpassword_hash"], data["lactive"], data["person_instanceid"]),
fetch=False)
self._load_users()

def edit_user(self):
    row = self.user_table.currentRow()
    if row >= self.user_table.rowCount():
        self.show_warning(self.translator.tr('warning_select_user'))
        return
    item = self.user_table.item(row, 0)
    user_id = item.data(Qt.UserRole)
    sql = """SELECT clogin, cpassword_hash, lactive, person_instanceid FROM auth.users WHERE id=%s"""
    if not data := self.db.execute_query(sql, (user_id,)):
        return
    data = ["clogin": row_data[0][0],
            "cpassword_hash": row_data[0][1],
            "lactive": row_data[0][2],
            "person_instanceid": row_data[0][3]]
    dlg = UserEditDialog(self, data)
    if dlg.exec_():
        data = dlg.get_data()
        sql = """UPDATE auth.users SET clogin=%s, cpassword_hash=%s, lactive=%s, person_instanceid=%s WHERE
id=%s"""
self.db.execute_query(sql, (new_data["clogin"], new_data["cpassword_hash"], new_data["lactive"],
new_data["person_instanceid"], user_id), fetch=False)
self._load_users()

def delete_user(self):
    row = self.user_table.currentRow()
    if row >= self.user_table.rowCount():
        self.show_warning(self.translator.tr('warning_select_user'))
        return
    item = self.user_table.item(row, 0)
    user_id = item.data(Qt.UserRole)
    if QMessageBox.question(self, self.translator.tr('confirm_delete'), self.translator.tr('confirm_delete_user')) ==
QMessageBox.Yes:
        sql = """DELETE FROM auth.users WHERE id=%s"""
        self.db.execute_query(sql, (user_id,), fetch=False)
        self._load_users()

# CRUD для ролей
def add_role(self):
    dlg = RoleAddDialog(self, db=self.db)
    if dlg.exec_():
        data = dlg.get_data()
        sql = """INSERT INTO auth.roles (cname, calias, parentid, lisinterface, cinterface_path, mcomment)
VALUES (%s, %s, %s, %s, %s, %s)"""
self.db.execute_query(sql, (data["cname"], data["calias"], data["parentid"], data["lisinterface"],
data["cinterface_path"], data["mcomment"])), fetch=False)
self._load_roles()

def edit_role(self):
    if not item := self.role_tree.currentItem():
        self.show_warning(self.translator.tr('warning_select_role'))
        return
    role_id = item.data(Qt.UserRole)
    sql = """SELECT cname, calias, parentid, lisinterface, cinterface_path, mcomment FROM auth.roles WHERE
id=%s"""
    if not data := self.db.execute_query(sql, (role_id,)):
        return
    data = ["cname": row_data[0][0],
            "calias": row_data[0][1],
            "parentid": row_data[0][2],
            "lisinterface": row_data[0][3],
            "cinterface_path": row_data[0][4],
            "mcomment": row_data[0][5]]
    dlg = RoleEditDialog(self, data, db=self.db)
    if dlg.exec_():
        data = dlg.get_data()
        sql = """UPDATE auth.roles SET cname=%s, calias=%s, parentid=%s, lisinterface=%s, cinterface_path=%s,
mcomment=%s WHERE id=%s"""
self.db.execute_query(sql, (new_data["cname"], new_data["calias"], new_data["parentid"],
new_data["lisinterface"], new_data["cinterface_path"], new_data["mcomment"], role_id), fetch=False)
self._load_roles()

def delete_role(self):
    if not item := self.role_tree.currentItem():
        self.show_warning(self.translator.tr('warning_select_role'))
        return
    role_id = item.data(Qt.UserRole)
    if QMessageBox.question(self, self.translator.tr('confirm_delete'), self.translator.tr('confirm_delete_role')) ==
QMessageBox.Yes:
        sql = """DELETE FROM auth.roles WHERE id=%s"""
        self.db.execute_query(sql, (role_id,), fetch=False)
        self._load_roles()

# CRUD для контекста
def get_users_list(self):
    return self.db.execute_query(sql="""SELECT * FROM auth.users ORDER BY clogin""")

def get_roles_list(self):
    return self.db.execute_query(sql="""SELECT * FROM auth.roles ORDER BY cname""")

def get_bus_list(self):
    return self.db.execute_query(sql="""SELECT * FROM auth.balanceunits ORDER BY cname""")

def add_context(self):
    users = self.get_users_list()
    roles = self.get_roles_list()
    bus = self.get_bus_list()
    if not users or not roles or not bus:
        self.show_warning(self.translator.tr('warning_context_required'))
        return
    dlg = ContextEditDialog(self, users, roles, bus)
    if dlg.exec_():
        data = dlg.get_data()
        sql = """INSERT INTO auth.userrolecontext (userid, roleid, balanceunitid)
VALUES (%s, %s, %s)"""
self.db.execute_query(sql, (data["userid"], data["roleid"], data["balanceunitid"])), fetch=False)
self._load_context()

def delete_context(self):
    row = self.context_table.currentRow()
    if row >= self.context_table.rowCount():
        self.show_warning(self.translator.tr('warning_select_context'))
        return
    item = self.context_table.item(row, 0)
    ctx_id = item.data(Qt.UserRole)
    if QMessageBox.question(self, self.translator.tr('confirm_delete'), self.translator.tr('confirm_delete_context')) ==
QMessageBox.Yes:
        sql = """DELETE FROM auth.userrolecontext WHERE id=%s"""
        self.db.execute_query(sql, (ctx_id,), fetch=False)
        self._load_context()

```


Файл: /home/sergey/Documents/configurate/tensor_crypto.py (относительно: tensor_crypto.py)

```
# tensor_crypto.py
import random
from hashlib import sha256

# Арифметика в конечном поле GF(p)
class GFp:
    """Элемент поля Fp (простое p)."""
    def __init__(self, value, p):
        self.p = p
        self.value = value % p

    def __add__(self, other):
        return GFp(self.value + other.value) % self.p, self.p
    def __sub__(self, other):
        return GFp(self.value - other.value) % self.p, self.p
    def __mul__(self, other):
        return GFp(self.value * other.value) % self.p, self.p
    def __pow__(self, other):
        return GFp(self.value ** other) % self.p, self.p
    def __inv__(self):
        return GFp(self.value * inv) % self.p, self.p
    def __eq__(self, other):
        return self.value == other.value and self.p == other.p
    def __str__(self):
        return str(self.value)

def random_element(p):
    return GFp(random.randrange(p), p)

# Матрицы n x n над GF(p)
class Matrix:
    def __init__(self, n, p, data=None):
        self.p = p
        if data is not None:
            self.data = data
        else:
            self.data = [[GFp(0, p) for _ in range(n)] for _ in range(n)]

    def __getitem__(self, idx):
        return self.data[idx]
    def __setitem__(self, idx, val):
        self.data[idx] = val

    @staticmethod
    def identity(n, p):
        m = Matrix(n, p)
        for i in range(n):
            m[i][i] = GFp(1, p)
        return m

    @staticmethod
    def random_invertible(n, p):
        """Сгенерация случайной обратимой матрицы через LU-подобный метод.
        # Создаем верхнюю треугольную матрицу с единицами на диагонали и случайными наддиагональными
        # Перемножаем получаем обратимую.
        U = Matrix.identity(n, p)
        for i in range(1, n):
            U[i][i] = random_element(p)
        # Перемножаем L * U
        M = Matrix(n, p)
        for i in range(n):
            for j in range(n):
                for k in range(i):
                    M[i][j] += U[i][k] * U[k][j]
                M[i][j] += U[i][j]
        return M

    def inverse(self):
        """Обратная матрица методом Гаусса-Жордана.
        # Расширенная матрица [A | I]
        inv = [[GFp(1, p) if i == j else GFp(0, p) for j in range(n)] for i in range(n)]
        for col in range(n):
            # Поиск ненулевого элемента в столбце col
            while pivot_row < n and a[pivot_row][col] == GFp(0, self.p):
                pivot_row += 1
            if pivot_row == n:
                raise ValueError("Matrix is singular")
            if pivot_row != col:
                a[col], a[pivot_row] = a[pivot_row], a[col]
                inv[col], inv[pivot_row] = inv[pivot_row], inv[col]
            pivot = a[col][col]
            inv[pivot] = GFp(1, self.p) / pivot
            for i in range(n):
                a[col][i] = a[col][i] * inv[pivot]
                inv[col][i] = inv[col][i] * inv[pivot]
            for i in range(n):
                if i != col:
                    factor = a[i][col]
                    for j in range(n):
                        inv[i][j] = inv[i][j] - factor * inv[col][j]
            result = Matrix(n, self.p)
            for i in range(n):
                result[i][i] = inv[i][i]
            return result

    def __mul__(self, other):
        """Умножение матриц.
        n = self.p
        m = Matrix(n, self.p)
        for i in range(n):
            for k in range(n):
                if a[i][k] != GFp(0, self.p):
                    for j in range(n):
                        result[i][j] = result[i][j] + a[i][k] * other[k][j]
        return result

    def to_bytes(self):
        """Сериализация матрицы в байты (для хэширования).
        data = range(self.p)
        for i in range(self.n):
            data += int(self[i][i]).to_bytes(2, 'big') # т.к. p < 2^16
        return data

# 3-тензоры n x n x n над GF(p)
class Tensor3:
    def __init__(self, n, p, data=None):
        self.p = p
        if data is not None:
            self.data = data
        else:
            self.data = np.empty((n, n, n), dtype=object)
            for i in range(n):
                for j in range(n):
                    self.data[i, j, k] = GFp(0, p)

    @staticmethod
```

```

def fixed_tensor(n, p):
    """Эталонный тензор T0 с ненулевыми элементами только при i=j и k=1.
    Индексы: 0..n-1.
    T = Tensor3(n, p)
    for i in range(n):
        T[i][i][1] = GFp(1, p)
    return T"""

def apply_isometry(self, A, B, C):
    """Применяет (A, B, C) к тензору: возвращает новый тензор."""
    p = self.p
    # Преобразование по первому измерению
    T1 = np.empty((n, n, n), dtype=object)
    for i in range(n):
        for j in range(n):
            coeff = GFp(0, p):
                for k in range(n):
                    if i == 0 and k == 0:
                        T1[i, j, k] = coeff * self.data[i, j, k]
                    else:
                        T1[i, j, k] = T1[i, j, k] + coeff * self.data[i, j, k]
    # по второму измерению
    T2 = np.empty((n, n, n), dtype=object)
    for i in range(n):
        for j in range(n):
            coeff = GFp(0, p):
                for k in range(n):
                    if i == 0 and k == 0:
                        T2[i, j, k] = coeff * T1[i, j, k]
                    else:
                        T2[i, j, k] = T2[i, j, k] + coeff * T1[i, j, k]
    # по третьему измерению
    T3 = np.empty((n, n, n), dtype=object)
    for i in range(n):
        for j in range(n):
            coeff = GFp(0, p):
                for k in range(n):
                    if i == 0 and k == 0:
                        T3[i, j, k] = coeff * T2[i, j, k]
                    else:
                        T3[i, j, k] = T3[i, j, k] + coeff * T2[i, j, k]
    result = Tensor3(n, p)
    result.data = T3
    return result

def repr_(self):
    return f"Tensor3({self.n}, {self.p})"

def to_bytes(self):
    """Сериализация тензора в байты."""
    data = []
    for i in range(self.n):
        for j in range(self.n):
            for k in range(self.p):
                data.append(int(self.data[i, j, k]).to_bytes(2, 'big'))
    return data

# Параметры системы (должны быть одинаковы у всех участников)
N = 2521 # простое число (2^16 - 1)
p0 = 1 # размерность тензора (можно увеличить до 32 для безопасности)

# Основные функции криптосистемы
def generate_keypair():
    """Генерирует пару (публичный тензор, секретные матрицы)."""
    B = Matrix.random_invertible(N, p)
    T = Matrix.random_invertible(N, p)
    return T, B, C

def message_to_matrix(message: bytes, n: int, p: int) -> Matrix:
    """Преобразует байтовое сообщение в матрицу n x n над GF(p).
    Сообщение дополняется хэшем для обеспечения целостности и произвольной длины.
    # Вычисляем хэш сообщения (32 байта) и добавляем в начало
    data = b'\x00' * 32 + hashlib.sha256(message).digest()
    # Дополняем нулями до n*n*2 байт (т.к. каждый элемент матрицы кодируется 2 байтами)
    total_cells = n * n * 2
    if len(data) < total_cells:
        data += b'\x00' * (total_cells - len(data))
    else:
        data = data[:total_cells]
    # Добавляем по парам байт и превращаем в числа
    M = Matrix(n, p)
    for i in range(n):
        for j in range(n):
            val = int.from_bytes(data[i*2:j*2], 'big')
            M[i, j] = GFp(val, p)
    return M

def matrix_to_message(M: Matrix) -> bytes:
    """Извлекает сообщение из матрицы.
    Ожидается, что первые 32 байта - хэш, затем само сообщение, затем дополнение.
    n = M.n
    data = b''
    for i in range(n):
        for j in range(n):
            data += M[i, j].to_bytes(2, 'big')
    # Извлекаем хэш (первые 32 байта) и остальное
    rest = data[32:]
    # Проверка хэша (хэш должен совпасть)
    if not hashlib.sha256(rest).digest() == data[:32]:
        raise ValueError("Контрольная сумма не совпадает - сообщение повреждено")
    return rest

def encrypt(public_tensor: Tensor3, message: bytes) -> Tensor3:
    """Шифрует сообщение с использованием публичного тензора.
    X = matrix_to_matrix(message, public_tensor.n, public_tensor.p)
    # Выводим Y = (X * I) * B
    return Y

def decrypt(secret_matrices, cipher_tensor: Tensor3) -> bytes:
    """Дешифрует сообщение, используя секретные матрицы (B, C).
    B = secret_matrices[0]
    C = secret_matrices[1]
    p = cipher_tensor.p
    n = cipher_tensor.n
    Y = cipher_tensor.data
    # Z = (Y * B) * C
    # Инициализируем X из слоя k=1 (индекс 1)
    X = Matrix(n, p)
    for i in range(n):
        for j in range(n):
            X[i, j] = Y[i, j][1]
    return matrix_to_message(X)

```

Файл: /home/sergey/Documents/configurate/view_sqlite.py (относительно: view_sqlite.py)

```
#!/usr/bin/env python3
coding: utf-8
import sys
import sqlite3
from PyQt5.QtWidgets import (QApplication, QMainWindow, QWidget, QVBoxLayout,
                             QMessageBox, QPushButton, QTableWidget, QHeaderView,
                             QAbstractTableModel, QModelIndex)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont

class SQLiteTableModel(QAbstractTableModel):
    """Модель для отображения данных из SQLite с поддержкой фильтрации."""
    def __init__(self, data, headers):
        self._data = data # список записей (кортежи)
        self._headers = headers # список имен колонок
        self._filters = {} # {col: pattern}
        self._filtered_rows = list(range(len(data)))

    def rowCount(self, parent=QModelIndex()):
        return len(self._filtered_rows)

    def columnCount(self, parent=QModelIndex()):
        return len(self._headers)

    def data(self, index, role=Qt.DisplayRole):
        if not index.isValid():
            return None
        if role == Qt.DisplayRole:
            row = self._filtered_rows[index.row()]
            col = index.column()
            value = self._data[row][col]
            return None if value is None else ""
        if role == Qt.EditRole:
            return None

    def headerData(self, section, orientation, role=Qt.DisplayRole):
        if orientation == Qt.Horizontal and role == Qt.DisplayRole:
            return self._headers[section]
        return None

    def setFilter(self, col, pattern):
        if self._filters[col] == pattern:
            return
        self._filters[col] = pattern
        self.updateFilter()

    def clearFilters(self):
        self._filters = {}
        self.updateFilter()

    def updateFilter(self):
        self.beginResetModel()
        for row_idx, row in enumerate(self._data):
            for col, pattern in self._filters.items():
                if pattern and row[col] and not None else ""
                regex = re.compile(pattern.replace(r'?', '.').replace(r'*', '.').replace(r'+', '.'))
                if not re.match(regex, row[col]):
                    break
            else:
                self._filtered_rows.append(row_idx)
        self.endResetModel()

class SQLiteViewer(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Просмотрщик SQLite")
        self.resize(900, 700)
        central = QWidget()
        self.setCentralWidget(central)
        main_layout = QVBoxLayout(central)

        # Верхняя панель
        top_layout = QHBoxLayout()
        self.btn_open = QPushButton("Открыть SQLite файл")
        top_layout.addWidget(self.btn_open)
        self.btn_apply = QPushButton("Применить фильтр")
        top_layout.addWidget(self.btn_apply)
        self.btn_clear = QPushButton("Сбросить фильтры")
        top_layout.addWidget(self.btn_clear)
        self.info_label = QLabel("Файл не открыт")
        top_layout.addWidget(self.info_label)
        main_layout.addLayout(top_layout)

        # Панель фильтров (горизонтальные QLineEdit)
        self.filter_layout = QHBoxLayout()
        main_layout.addLayout(self.filter_layout)

        # Таблица
        self.table_view = QTableWidget()
        font = QFont()
        font.setPointSize(12) # увеличенный шрифт
        self.table_view.setHeaderSection(0, True)
        main_layout.addWidget(self.table_view)

        self.model = None
        self.conn = None

    def open_file(self):
        file_path = QFileDialog.getOpenFileName(
            self, "Выберите SQLite файл", "", "SQLite files (*.sqlite *.db);;All files (*)"
        )
        if not file_path:
            return

        try:
            # Подключаемся к базе
            self.conn = sqlite3.connect(file_path)
            # Получаем список таблиц
            cursor = self.conn.cursor()
            cursor.execute("SELECT name FROM sqlite_master WHERE type='table'")
            tables = [row[0] for row in cursor.fetchall()]
            if not tables:
                QMessageBox.warning(self, "Ошибка", "В базе нет таблиц")
                return
            # Для простоты используем первую таблицу (kladr или любую)
            table_name = tables[0]
            # Загружаем данные
            cursor.execute("SELECT rowid, * FROM \"{}\"".format(table_name))
            headers = [description[0] for description in cursor.description]
            # Создаём модель
            self.table_view.setModel(SQLiteTableModel(cursor.fetchall(), headers))
            # Создаём фильтры
            self.create_filter_inputs(headers)
            self.info_label.setText(f"Файл: {file_path.split('/')[-1]} | Таблица: {table_name} | Записей: {len(cursor.fetchall())}")
        except Exception as e:
            QMessageBox.critical(self, "Ошибка", f"Не удалось прочитать файл: {str(e)}")
            self.conn = None
```

```

def create_filter_inputs(self, headers):
    self.clear_filter_layout()
    for col, header in enumerate(headers):
        line_edit = QLineEdit(header)
        line_edit.returnPressed.connect(self.apply_filters)
        self.filter_edits.append((col, line_edit))

def clear_filter_layout(self):
    for widget in reversed(self.filter_layout_widgets()):
        if widget:
            self.filter_layout.removeItem(widget)
            widget.deleteLater()

def apply_filters(self):
    if self.model is None:
        return
    for col, line_edit in self.filter_edits:
        self.model.set_filter(col, text if text else None)

def clear_filters(self):
    if self.model is None:
        return
    self.model.clear_filters()
    for line_edit in self.filter_edits:
        line_edit.clear()

def closeEvent(self, event):
    if self.conn:
        self.conn.close()
    event.accept()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    viewer = SQLTableView()
    viewer.show()
    sys.exit(app.exec_())

```

Файл: /home/sergey/Documents/configurate/widgets/__init__.py (относительно: widgets/

```

# widgets/ __init__.py
from .reference_editor import ReferenceEditor
from .reference_selector import ReferenceSelector
__all__ = ['ReferenceEditor', 'ReferenceSelector']

```

Файл: /home/sergey/Documents/configurate/widgets/reference_editor.py (относительно: w

```

#reference_editor.py
from PyQt5.QtWidgets import QWidget, QHBoxLayout, QLineEdit, QToolButton, QMessageBox
from database import DatabaseService

class ReferenceEditor(QWidget):
    valueChanged = pyqtSignal(object)
    def __init__(self, parent=None, table_name=None, display_field='cname',
                 id_field='id', id_dialog=None, db=None):
        super().__init__(parent)
        self.table_name = table_name
        self.display_field = display_field
        self.id_dialog = id_dialog
        self.db = DatabaseService()
        self.current_id = None
        layout = QHBoxLayout(self)
        layout.setContentsMargins(0, 0, 0, 0)
        self.line_edit = QLineEdit()
        self.line_edit.setReadOnly(True)
        layout.addWidget(self.line_edit)
        self.btn_select = QToolButton()
        self.btn_select.setText('...')
        layout.addWidget(self.btn_select)
        self.btn_clear = QToolButton()
        self.btn_clear.setText('X')
        layout.addWidget(self.btn_clear)
        self.btn_clear.clicked.connect(self.clear_value)
        def set_value(self, item_id, display_text=None):
            if display_text is not None:
                self.line_edit.setText(display_text)
            else:
                self.line_edit.setText(str(item_id) if item_id else "")
            self.valueChanged.emit(item_id)
        def get_value(self):
            return self.current_id
        def setReadOnly(self, ro):
            self.btn_clear.setEnabled(not ro)
        def select_value(self):
            if not self.table_name:
                QMessageBox.warning(self, "Ошибка", "Не указана таблица-источник")
            if self.id_dialog:
                dlg = self.id_dialog(self.table_name, self.display_field, self.id_field, db=self.db)
            else:
                from widgets.reference_selector import ReferenceSelector
                dlg = ReferenceSelector(self.table_name, self.display_field, self.id_field, db=self.db, parent=self)
            if dlg.exec_():
                selected_id, selected_display = dlg.get_selected()
                self.set_value(selected_id, selected_display)
        def clear_value(self):
            self.set_value(None, "")

```

Файл: /home/sergey/Documents/configurate/widgets/reference_selector.py (относительно:

```

# widgets/reference_selector.py
from PyQt5.QtWidgets import QDialog, QVBoxLayout, QTableWidgetItem, QTableWidgetItem, QPushButton,
QHBoxLayout, QLineEdit, QHeaderView
from PyQt5.QtCore import Qt
from database import DatabaseService

class ReferenceSelector(QDialog):
    """Универсальный диалог выбора записи из указанной таблицы.
    def __init__(self, table_name, display_field='cname', id_field='id',
                 parent=None, db=None):
        super().__init__(parent)
        self.setWindowTitle("Выбор значения")
        self.resize(600, 400)
        self.table_name = table_name
        self.display_field = display_field
        self.id_field = id_field
        self.db = DatabaseService()
        self.selected_id = None
        self.selected_display = None
        layout = QVBoxLayout(self)

```

```

self.search_edit = QLineEdit()
self.search_edit.setPlaceholderText("Поиск")
layout.addWidget(self.search_edit)

self.table = OTableWidget()
self.table.setHorizontalHeaderLabels(["ID", "Наименование"])
self.table.setSelectionBehavior(OTableWidget.SelectRows)
self.table.setSelectionMode(OTableWidget.SingleSelection)
layout.addWidget(self.table)

btn_layout = QHBoxLayout()
btn_ok.clicked.connect(self.accept_selection)
btn_cancel.clicked.connect(self.reject)
btn_layout.addWidget(btn_ok)
btn_layout.addWidget(btn_cancel)
layout.addLayout(btn_layout)

self.load_data()

def load_data(self):
    filter_text = self.search_edit.text().strip()
    sql = "SELECT {self.id_field}, {self.display_field} FROM {self.table_name} WHERE {self.display_field} ILIKE %s"
    ORDER BY {self.id_field} LIMIT 200"
    params = ("%" + filter_text + "%",)
    else:
    sql = "SELECT {self.id_field}, {self.display_field} FROM {self.table_name} ORDER BY {self.display_field} LIMIT
200"
    params = None
    rows = self.db.execute(query(sql, params))
    self.table.setRowCount(0)
    for row in rows:
        pos = self.table.rowCount()
        self.table.insertRow(pos)
        for col, val in enumerate(row):
            if col == 0:
                item = QTableWidgetItem(str(val))
                self.table.setItem(pos, col, item)
            else:
                item = QTableWidgetItem(str(val))
                self.table.setItem(pos, col, item)

def accept_selection(self):
    current_row = self.table.currentRow()
    if current_row >= 0:
        item = self.table.item(current_row, 0)
        self.selected_id = item.data(Qt.UserRole)
        self.selected_display = self.table.item(current_row, 1).text()
        self.accept()

def get_selected(self):
    return self.selected_id, self.selected_display

```
